

캡스톤 디자인 I 종합설계 프로젝트

프로젝트 명	길벗
팀 명	리트리버 팀
문서 제목	결과보고서

Version	1.0
Date	2026-MAY-14

팀원	한여진 (조장)
	김예지
	양나래
	이일환
	전예찬
	황연주

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

CONFIDENTIALITY/SECURITY WARNING

이 문서에 포함되어 있는 정보는 국민대학교 소프트웨어융합대학 소프트웨어학부 및 소프트웨어학부 개설 교과목 캡스톤 디자인 수강 학생 중 프로젝트 “길벗”을 수행하는 팀 “리트리버”의 팀원들의 자산입니다. 국민대학교 소프트웨어학부 및 팀 “리트리버”의 팀원들의 서면 허락없이 사용되거나, 재가공 될 수 없습니다.

문서 정보 / 수정 내역

Filename	결과보고서-길벗.doc
원안작성자	한여진, 김예지, 양나래, 이일환, 전예찬, 황연주
수정작업자	한여진, 김예지, 양나래, 이일환, 전예찬, 황연주

수정날짜	대표수정자	Revision	추가/수정 항목	내 용
2026-05-14	한여진	1.0	최초 작성	최초 작성

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

목 차

1	개요	4
1.1	프로젝트 개요.....	4
1.2	추진 배경 및 필요성.....	5
2	개발 내용 및 결과물	9
2.1	목표.....	9
2.2	연구/개발 내용 및 결과물	9
2.2.1	연구/개발 내용.....	9
2.2.2	시스템 기능 요구사항	38
2.2.3	시스템 비기능(품질) 요구사항	45
2.2.4	시스템 구조 및 설계도	46
2.2.5	활용/개발된 기술	53
2.2.6	현실적 제한 요소 및 그 해결 방안.....	59
2.2.7	결과물 목록.....	62
2.3	기대효과 및 활용방안	64
3	자기평가.....	65
4	참고 문헌.....	75
5	부록	77
5.1	사용자 매뉴얼.....	77
5.2	운영자 매뉴얼.....	79
5.3	배포 가이드.....	80
5.4	현장 조사 및 인터뷰.....	82
5.5	멘토링.....	89
5.5.1	1차 멘토링	89
5.5.2	2차 멘토링	92
5.6	테스트 케이스.....	97
5.7	백엔드 API 명세서	100
5.8	데이터셋 상세 클래스 정보.....	103
5.9	AI 안내 판단 기준.....	105
5.10	DB 스키마	112
5.11	개발 환경 상세	113
5.12	Git 협업 방식.....	117

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

1 개요

1.1 프로젝트 개요

본 프로젝트 “길벗”은 시각장애인 및 저시력자의 안전한 보행을 지원하기 위한 **모바일 기반 보행 보조 애플리케이션**을 개발하는 것을 목표로 한다.

스마트폰 카메라와 인공지능 기반 객체 인식 기술을 활용하여 사용자의 주변 환경을 실시간으로 분석하고, 보행 중 발생할 수 있는 장애물을 탐지한다. 단순히 장애물을 감지하는 것에 그치지 않고, **장애물의 위치를 분석하여 사용자에게 최적의 방향을 안내하는 기능**을 제공한다. 예를 들어 장애물이 사용자의 정면에 위치한 경우 “왼쪽으로 이동하세요.” 와 같은 음성 안내를 제공하여 사용자가 안전하게 장애물을 피할 수 있도록 지원한다.

또한, 사용자가 목적지를 입력하면 **음성 기반 길찾기 기능**을 통해 경로를 안내하며, 시각장애인을 고려한 **접근성 중심 사용자 인터페이스**와 TTS(Text-to-Speech) 기반 음성 피드백 시스템을 통해 정보를 전달한다.

본 시스템은 다음과 같은 기능을 중심으로 구성된다.

- 시각장애인 및 저시력자를 고려한 **접근성 중심 UI**
- 카메라 기반 **실시간 장애물 탐지**
- 장애물 위치 분석 기반 **최적의 방향 안내**
- 음성 안내 기반 **길찾기 기능**
- TTS 기반 **음성 피드백 시스템**
- **햅틱 안내 시스템**

이를 통해 시각장애인의 독립적인 이동성을 향상시키고, 보다 안전한 보행 환경을 제공하는 것을 목표로 한다.



안내견 리트리버는 ‘길벗’ 서비스의 따뜻한 동반자로, 시각장애인과 저시력자의 안전한 보행을 함께 돕는 안내 캐릭터이다.

사용자의 시선이 되어 주변 환경을 대신 인식하고, 위험 요소를 빠르게 전달하며 올바른 방향으로 안내하는 역할을 상징한다. 친근하고 신뢰감 있는 이미지로 설계되어 사용자가 언제나 안심하고 의지할 수 있는 존재를 표현한다. 또한 기술과 배려가 결합된 ‘디지털 안내견’이라는 의미를 담아, 서비스의 핵심 가치를 직관적으로 전달한다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

1.2 추진 배경 및 필요성

1.2.1 기술의 시장 현황

최근 인공지능 기술과 모바일 기술의 발전으로 다양한 분야에서 **컴퓨터 비전 기반 서비스**가 빠르게 확대되고 있다. 특히 객체 인식 기술을 활용한 서비스는 자율주행, 스마트 시티, 보안 시스템 등 다양한 산업 분야에서 활용되고 있으며, 모바일 환경에서도 이러한 기술을 적용하려는 시도가 증가하고 있다.

또한 고령화 사회로의 진입과 함께 **보행 보조 기술에 대한 사회적 관심과 수요**가 증가하고 있다. 시각 장애인 뿐만 아니라 노약자 등 이동 약자를 위한 기술 개발이 활발하게 이루어지고 있으며, 스마트폰과 같은 개인 모바일 기기를 활용한 보행 보조 서비스의 필요성이 점차 증가하고 있다.

특히 스마트폰은 대부분의 사용자가 이미 보유하고 있는 장치이며 카메라, GPS, 센서 등 다양한 기능을 포함하고 있어 별도의 장비 없이도 다양한 보행 보조 기능을 제공할 수 있는 플랫폼으로 활용될 수 있다. 이러한 환경 속에서 **모바일 기반 보행 보조 시스템의 개발 가능성과 시장성이 확대**되고 있다.

1.2.2 기술 발전 현황

최근 딥러닝 기반 컴퓨터 비전 기술의 발전으로 객체 탐지 정확도와 처리 속도가 크게 향상되었다. 특히 **YOLO 계열과 같은 실시간 객체 탐지 모델**은 단일 프레임 내에서 차량, 보행자, 장애물 등 다양한 클래스를 동시에 탐지할 수 있으며, 신뢰도 점수(Confidence Score)와 바운딩 박스(Bounding Box, BBox) 정보를 함께 제공하여 탐지 결과를 기반으로 한 위험도 분석 및 우선순위 산출이 가능해졌다.

또한 객체 탐지를 넘어 이미지 내 각 영역을 픽셀 단위로 구분하는 **segmentation 기술**도 빠르게 발전하고 있다. **Semantic segmentation 모델**은 보도, 차도, 횡단보도와 같이 장면을 구성하는 영역을 분할할 수 있으며, **instance segmentation 모델**은 객체의 위치뿐 아니라 실제 형태에 가까운 mask 정보를 제공할 수 있다. 이러한 기술은 단순히 장애물의 존재 여부를 파악하는 수준을 넘어, 사용자가 이동할 수 있는 보행 가능 영역과 위험 영역을 구분하는 데 활용될 수 있다. 특히 시각장애인 보행 보조 서비스에서는 보도, 점자블록, 횡단보도, 차도, 주의 노면 등을 구분하는 것이 중요하므로, segmentation 기술은 보행 환경 이해를 위한 핵심 기술로 활용될 수 있다. 최근에는 YOLO 계열 모델에서도 segmentation 기능을 지원하면서, 객체 탐지와 영역 분할을 하나의 모델 구조 안에서 처리할 수 있게 되었다.

FastAPI 와 같은 비동기 기반 Python 웹 프레임워크의 발전은 딥러닝 모델을 별도 추론 서버로 분리하여 운영하는 구조를 용이하게 하였다. 이를 통해 모바일 클라이언트에서 전송된 이미지를 서버에서 즉시 추론하고 결과를 반환하는 실시간 파이프라인 구축이 가능해졌다. 또한 PyTorch 의

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

GPU 자동 감지 및 inference mode 최적화 기술은 서버사이드 추론의 지연 시간을 단축하는 데 기여하고 있으며, 실제 서비스 환경에서 컴퓨터 비전 모델을 API 형태로 연동하는 데 활용되고 있다.

백엔드 분야에서는 비동기 처리와 실시간 통신 기술이 빠르게 발전하였다. **Spring Boot** 와 같은 프레임워크는 **WebSocket** 및 **STOMP** 프로토콜을 통한 **실시간 양방향 통신**을 안정적으로 지원하며, AI 추론 결과를 특정 사용자에게 즉시 전달하는 개인화 메시지 브로커 구성이 가능해졌다. Spring WebFlux 기반의 리액티브 프로그래밍 모델은 대규모 동시 연결에서도 효율적인 자원 관리를 가능하게 한다. JWT 기반 무상태(Stateless) 인증 방식은 모바일 환경에서의 세션 관리 부담을 줄이고 확장성을 높이는 표준적인 수단으로 자리잡았으며, WebSocket 연결 시점에도 JWT 를 검증하는 방식으로 실시간 통신 구간의 보안을 확보할 수 있게 되었다. 또한 Tmap 보행자 경로 API, 카카오 Local API, OpenWeather API 등 외부 서비스와의 연동이 REST 표준화로 인해 용이해져, 경로 탐색·장소 검색·기상 정보 제공 등의 기능을 짧은 개발 기간 내에 통합 구현할 수 있게 되었다.

프론트엔드 분야에서는 **Flutter** 와 같은 크로스플랫폼 프레임워크의 발전으로 Android 와 iOS 를 단일 코드베이스에서 실무 수준으로 개발하는 것이 가능해졌다. TTS(Text-to-Speech) 및 STT(Speech-to-Text) API 의 품질 향상으로 음성 기반 인터페이스의 완성도가 높아졌으며, 이를 활용한 시각장애인 대상 접근성 앱 개발이 활성화되고 있다. 또한 Android TalkBack, iOS VoiceOver 와 같은 운영체제 내장 스크린리더와 앱의 접근성 트리를 연동하는 기술이 발전하여, 별도의 보조 기기 없이도 스마트폰만으로 시각장애인이 앱을 조작할 수 있는 환경이 마련되고 있다.

1.2.3 기존 서비스 현황

현재 시각장애인을 위한 다양한 보행 보조 서비스가 존재한다. 대표적으로 음성 기반 길찾기 서비스, 스마트 지팡이와 같은 보조 장치, 그리고 카메라 기반 객체 인식 기술을 활용한 보행 보조 애플리케이션 등이 있다.

예를 들어 일부 스마트폰 애플리케이션은 카메라를 활용하여 주변 환경을 분석하고 특정 물체를 인식하는 기능을 제공하고 있으며, 지도 기반 서비스에서는 음성 안내를 통해 사용자가 목적지까지 이동할 수 있도록 지원하고 있다.

그러나 이러한 서비스들은 대부분 특정 기능에만 초점을 맞추고 있어 실제 보행 상황에서 발생하는 다양한 위험 요소를 충분히 대응하지 못하는 경우가 많다. 또한 별도의 장비가 필요한 경우도 있어 접근성이 제한될 수 있다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

1.2.4 기존 서비스 문제점 및 개발 필요성

1) 기존 서비스 분석

서비스	특징	한계
Be My Eyes	자원봉사자와의 실시간 영상 통화 를 통해 시각장애인이 카메라로 비춘 주변 환경을 사람이 대신 확인하고 설명해주는 시각 보조 서비스	실시간 보행 상황에서 즉각적인 장애물 대응 등의 도움을 받기 어려움
Seeing AI	Microsoft가 개발한 AI 기반 시각 보조 앱으로 텍스트, 사물, 사람, 장면 등을 인식하여 음성으로 설명 하는 기능 제공	주변 환경 설명 중심 서비스로 보행 중 안전한 이동을 위한 실시간 장애물 탐지 및 방향 안내 기능이 부족함
Sullivan+	스마트폰 카메라와 AI 객체 인식 기술 을 활용하여 텍스트, 사물, 얼굴 등을 인식하고 그 결과를 음성으로 안내하는 시각 보조 서비스	다양한 인식 기능을 제공하지만 실제 이동 상황에서 장애물 회피를 위한 보행 보조 기능은 제한적임
Lazarillo	GPS 기반 음성 안내 를 통해 현재 위치와 주변 장소 정보를 제공하여 시각장애인의 보행 경로 탐색을 지원하는 길찾기 서비스	지도 기반 위치 안내 중심으로 제공되며 보행 중 발생하는 장애물을 직접 인식하거나 회피 방향을 안내하는 기능이 부족함
G-EYE+	시각장애인 보행에 맞춰 음성 안내, 제스처 조작, 출발 방향·건물 입구·횡단보도·계단 같은 보행 정보를 제공하는 전용 내비게이션	서비스 가능 지역이 제한적이고, 실시간으로 변하는 장애물 대응에는 한계가 있어 범용성과 현장 반응성은 부족함

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

2) “길벗”과 기존 서비스 차별점

기능	Be My Eyes	Seeing AI	Sullivan+	Lazarillo	G-EYE+	길벗 (본 프로젝트)
영상 기반 도움 요청	O	X	X	X	X	O (실시간 이미지 스트림)
GPS 기반 길 찾기	X	X	X	O	O	O
주변 장소 안내	X	X	X	O	O	O
사물 인식	X	O	O	X	X	O
장애물 탐지	X	제한적	제한적	X	제한적	O
실시간 보행 지원	X	X	X	제한적	O	O
장애물 회피 방향 안내	X	X	X	X	제한적	O (핵심 기능)

기존 서비스들은 시각장애인을 위해 주변 환경을 설명하거나 길찾기 기능을 제공하는 데 초점을 맞추고 있다. 그러나 실제 보행 환경에서는 다양한 장애물과 같은 위험 요소가 존재하며, 이러한 요소를 실시간으로 인식하고 사용자가 안전하게 이동할 수 있도록 장애물이 존재하지 않는 **최적의 방향을 안내하는 기능은 충분히 제공되지 않고 있다.**

따라서 본 프로젝트에서는 인공지능 기반 객체 탐지 기술을 활용하여 보행 경로 상의 장애물을 실시간으로 탐지하고, 장애물의 위치를 분석하여 사용자가 안전하게 이동할 수 있도록 **이동 방향을 안내하는 보행 보조 시스템을 개발하고자 한다.**

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

2 개발 내용 및 결과물

2.1 목표

본 프로젝트의 목표는 다음과 같다.

“시각장애인(저시력자)의 안전한 보행을 지원하기 위한 모바일 기반 보행 보조 애플리케이션을 개발한다.”

특히 스마트폰 카메라와 인공지능 기반 객체 인식 기술을 활용하여 보행 중 발생할 수 있는 장애물을 실시간으로 탐지하고, 장애물의 위치를 분석하여 사용자가 안전하게 이동할 수 있도록 최적의 방향을 안내하는 기능을 구현하는 것을 핵심 목표로 한다.

이를 위해 다음과 같은 세부 목표를 설정한다.

1. 스마트폰 카메라를 활용한 실시간 보행 위험 요소 탐지 시스템 구현
2. 탐지된 장애물의 위치를 분석하여 최적의 방향을 안내하는 기능 구현
3. 보행자 신호등 상태를 인식하여 음성으로 안내하는 기능 구현
4. 목적지 기반 음성 안내 길찾기 기능 구현
5. 시각장애인을 고려한 접근성 중심 사용자 인터페이스 설계
6. TTS(Text-to-Speech) 기반 음성 안내 시스템 구현
7. 모바일 환경에서 안정적인 성능을 위한 서버 기반 AI 추론 및 결과 전달 구조 구현

2.2 연구/개발 내용 및 결과물

2.2.1 연구/개발 내용

본 프로젝트에서는 시각장애인의 안전한 보행을 지원하기 위해 다음과 같은 기능을 중심으로 서비스를 개발한다.

1) 장애물 탐지 기능

스마트폰 카메라 영상 이미지를 기반으로 보행 중 발생할 수 있는 장애물을 실시간으로 탐지하는 인공지능 시스템을 구현한다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

본 기능에서는 스마트폰 카메라로 입력되는 보행 영상 이미지를 기반으로, 보행자의 안전을 위협할 수 있는 객체를 실시간으로 인식하는 인공지능 모듈을 개발한다. 이를 위해 YOLOv11s 객체 탐지 모델을 활용하여 사람, 자전거, 킥보드, 차량, 오토바이, 휠체어, 신호등 등 보행 환경에서 위험한 객체를 탐지한다. 장애물 탐지 정확도와 위험 판단의 신뢰성을 높이기 위해, 단순 객체 탐지 모델만 사용하는 것이 아니라 YOLO26-Seg 기반 segmentation 모델과 Depth Estimation 모델을 함께 활용한다.

객체 탐지 모델은 차량, 사람, 볼라드, 오토바이 등 개별 장애물의 위치와 종류를 bounding box (bbox) 형태로 탐지하고, YOLO26-Seg 모델은 도로, 보행 가능 영역, 경계 영역 등을 mask 형태로 분할한다. 이를 통해 탐지된 객체가 실제 사용자의 보행 경로상에 위치한 위험 요소인지, 혹은 진행 방향과 무관한 주변 배경 객체인지 구분할 수 있다. 예를 들어 화면 내에 차량이 존재하더라도 보행 가능 영역 밖에 있거나 사용자의 진행 경로와 겹치지 않는 경우에는 위험도를 낮게 판단하고, 반대로 보행 경로상 가까운 위치에 존재하는 객체는 우선순위가 높은 위험 요소로 판단하여 음성 안내를 제공하도록 설계한다.

또한 객체의 실제 위험도를 보다 정확하게 판단하기 위해 Depth Estimation 기반 거리 추정 기능을 추가하였다. 기존의 bbox 크기 기반 거리 추정 방식은 객체의 실제 거리보다 화면 내 크기에 크게 영향을 받기 때문에, 작지만 가까운 객체와 크지만 먼 객체를 정확히 구분하기 어렵다는 한계가 있었다. 이를 보완하기 위해 본 프로젝트에서는 Depth Anything V2 Metric Outdoor 모델을 활용하여 입력 이미지의 depth map 을 생성하고, 객체 bbox 와 결합하여 사용자와 장애물 간의 거리를 추정한다. 거리 계산 시에는 bbox 전체 영역을 그대로 사용하는 것이 아니라, 객체가 지면과 맞닿는 하단 영역을 중심으로 depth 값을 추출하고, 그중 가까운 픽셀 일부의 중앙값을 사용하여 거리값을 계산한다. 이를 통해 차량 하단의 도로면이나 배경 픽셀이 거리 계산에 섞이는 문제를 줄이고, 실제 객체와 사용자 간 거리를 보다 안정적으로 추정할 수 있도록 하였다.

Depth Estimation 결과는 단순 시각화에 그치지 않고, 위험도 판단과 음성 안내 여부 결정에 직접 활용된다. 시스템은 추정된 거리값을 기준으로 3m 이내를 긴급 거리, 5.5m 이내를 경고 거리, 8m 이내를 안내 가능 거리로 구분한다. 이를 통해 사용자와 실제로 가까운 장애물은 우선적으로 안내하고, 멀리 있거나 진행 경로와 무관한 객체는 불필요하게 음성 안내하지 않도록 설계하였다. 또한 모델의 거리 예측값과 실제 거리 사이의 오차를 줄이기 위해 선형 보정을 적용하였다. 즉, depth 모델이 출력한 raw distance 에 scale factor 와 offset 을 적용하여 실제 보행 환경에 더 적합한 거리값으로 보정함으로써, 가까운 객체와 먼 객체를 보다 정확하게 구분할 수 있도록 하였다.

연구 개발 방법은 다음과 같다.

- ① 데이터 수집 및 전처리 단계: 스마트폰 촬영 영상 이미지와 시뮬레이션 환경 데이터를 수집하고, 장애물 객체와 보행 영역에 대한 라벨링 데이터를 구축한다.
- ② 객체 탐지 모델 학습 단계: YOLOv11s 모델을 기반으로 장애물 탐지 학습을 수행하고, 보행 안전과 직접적으로 관련된 클래스 중심으로 성능을 최적화한다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

③ 영역 분할 모델 학습 단계: YOLO26-Seg 를 활용하여 도로/보행 가능 영역/경계선을 분할하는 모델을 학습한다.

④ Depth Estimation 기반 거리 추정 단계: Depth Anything V2 Metric Outdoor 모델을 활용하여 입력 이미지의 깊이 정보를 추정하고, 객체 bbox 하단 영역의 depth 값을 기반으로 사용자와 장애물 간 거리를 계산한다. 또한 실측 결과를 반영한 선형 보정을 적용하여 거리 추정 정확도를 개선한다.

⑤ 융합 추론 단계: 객체 탐지 결과, 영역 분할 결과, depth 기반 거리 추정 결과를 결합하여, 단순히 “무엇이 있는가”뿐 아니라 “사용자 경로 상에서 얼마나 가깝고 위험한가”를 판단한다.

⑥ 실시간 추론 및 경량화 단계: 스마트폰 기반 서비스에 적용하기 위해 입력 해상도, 추론 속도, 응답 시간을 최적화하고, 실시간 동작 가능 여부를 검증한다.

최종 결과물은 실시간 장애물 탐지 및 거리 기반 위험 판단 AI 모듈이며, 카메라 입력 영상 이미지에서 주요 장애물을 탐지하고 각 객체의 위치, 보행 경로 침범 여부, 사용자와의 추정 거리, 위험도를 산출하여 후속 안내 기능으로 전달하는 형태로 구현한다. 이를 통해 단순히 화면에 보이는 모든 객체를 안내하는 것이 아니라, 실제 사용자와 가까운 위험 요소를 우선적으로 선별하여 보다 정확하고 실용적인 보행 보조 안내를 제공할 수 있도록 한다.

데이터셋

본 프로젝트에서는 AIHub 의 인도 보행 영상 데이터셋을 활용한다. 해당 데이터셋은 시각장애인 및 보행약자의 안전한 이동 지원을 목적으로 구축된 보행 환경 특화 데이터로, 보행 중 마주칠 수 있는 다양한 장애물 객체와 노면 정보를 포함하고 있다.

객체 탐지용 데이터는 bbox 35 만 건, 노면 상태 분석용 Surface Masking 5 만 건 등으로 구성되어 있어 탐지와 분할 모델 학습에 모두 적합하다. 객체 클래스는 사람, 자동차, 버스, 트럭, 자전거, 스쿠터, 휠체어, 유모차 같은 이동체와 불라드, 바리케이드, 신호등, 전신주, 정류장 등을 포함한다. 또한 세그멘테이션을 위해 인도, 점자블록, 차도, 이면도로, 자전거도로, 주의구역 등의 노면 클래스와 보도블록, 시멘트, 파손, 횡단보도, 계단, 맨홀 등 세부 속성 정보도 함께 제공한다.

이를 통해 본 프로젝트는 보행자 전방의 장애물 탐지뿐 아니라 노면 상태와 주의구역까지 함께 인식하여, 보다 안전한 보행 보조 인공지능 시스템을 구현하고자 한다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

분류	객체 클래스	
이동체	• person (사람) • car (승용차, 승합차) • bus (버스) • truck (트럭) • bicycle (자전거) • motorcycle (오토바이, 스쿠터) • stroller (유모차) • wheelchair (휠체어) • dog (개) • cat (고양이) • carrier (리어카, 손수레)	
고정체	• movable_signage (이동식 홍보 전시물/안내판) • scooter (변속 기어가 없는 이륜차, 두 발로 탈 수 있는 기구) • tree_trunk (가로수 기둥) • potted_plant (화분) • traffic_light (신호등) • traffic_sign (교통 표지판) • pole (대/기둥) • bench (2인 이상이 앉는 기구) • chair (1인이 앉을 수 있는 기구) • table (탁자) • stop (버스/택시 정류장) • kiosk (한쪽이 열린 점포) • fire_hydrant (소화전) • parking_meter (주차요금정산기) • bollard (볼라드) • barricade (바리케이드) • power_controller (전력제어함) • traffic_light_controller (신호등제어기)	
노면 클래스		속성
Sidewalk (인도)		• blocks (보도블럭) • cement (시멘트) • urethane (우레탄) • asphalt (아스팔트) • soil_stone (흙, 돌, 비포장도로) • damaged (인도의 파손) • other (정의된 인도의 속성 외의 경우)
braille_guiding_block (점자블럭)		• normal • damaged (점자블럭 파손)
roadway (차도)		• normal • crosswalk (횡단보도)
alley (이면도로)		• normal • crosswalk (횡단보도) • speed_bump (과속방지턱) • damaged (이면도로 파손)
bike_lane (자전거도로)		(해당없음)
caution_zone (주의구역)		• stairs (계단) • manhole (맨홀) • tree_zone (가로수 한그루 식재를 위한 바닥면 구역) • grating (그레이팅) • repair_zone (보수구역)

최적의 모델 선정 과정

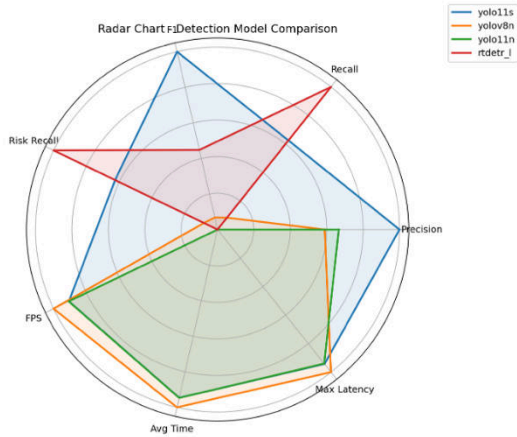
본 실험에서는 객체 탐지, 신호등 인식, 보행 환경 분석을 위한 다양한 데이터셋을 활용하였다. 먼저 객체 탐지를 위해 AIHub 에서 제공하는 인도 보행 영상 데이터셋 중 bbox 데이터 약 35 만 개 중 일부를 활용하여 약 10GB 규모의 이미지 데이터셋을 구성하고 학습을 진행하였다. 해당 데이터셋은 차량, 사람, 자전거, 킥보드, 볼라드 등 보행 시 위험 요소가 될 수 있는 다양한 객체를 포함하고 있으며, 실제 인도 환경을 반영한 데이터로 구성되어 있다. 특히 인도 보행 데이터셋은 보행 중 발생할 수 있는 다양한 장애물과 환경 정보를 포함하고 있어 실환경 적용에 적합한 특징을 가진다. 객체 탐지 모델은 총 50 epoch 동안 학습을 수행하였다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

보행 환경 분석을 위해서는 AIHub 인도 보행 영상 데이터셋 중 Surface Masking 데이터 약 5 만 개 중 일부를 활용하여 약 30GB 규모의 데이터셋을 구성하고 학습을 진행하였다. 해당 데이터는 인도, 차도, 횡단보도, 점자블록 등 보행과 밀접한 환경 요소를 폴리곤 형태로 어노테이션한 데이터로, 보행 가능 영역 및 위험 요소를 정밀하게 분석하는 데 활용된다. 보행 환경 분석 모델은 총 50 epoch 동안 학습을 수행하였다.

1-1) Object Detection 모델 테스트 결과





모델	F1	Risk Recall	Avg Inference(ms)	FPS	Max Latency(ms)	mAP50
YOLOv8n	0.5867	0.5353	12.53	79.84	14.33	0.3386
YOLOv11n	0.5773	0.5099	13.06	76.55	14.95	0.3617
YOLOv11s	0.7167	0.6998	13.07	76.51	14.92	0.4861
RT-DETR-L	0.6397	0.8137	22.64	44.17	24.71	0.5013

본 실험에서는 다양한 object detection 모델에 대해 정확도와 실시간성을 함께 고려하여 성능을 비교하였다. 실험 결과, YOLOv11s 는 F1 0.7167, Risk Recall 0.6998, mAP50 0.4861 로 전반적으로 가장 균형 잡힌 성능을 보였다. 평균 추론 시간은 13.07ms, FPS 는 76.51, 최대 지연 시간은 14.92ms 로 측정되어, 정확도와 실시간성을 동시에 만족하는 모델로 확인되었다. 특히 실제 보행 보조 환경에서 중요한 위험 상황 탐지 성능 측면에서도 높은 수준을 보여, 우선적으로 고려할 수 있는 후보로 판단된다.

YOLOv8n 은 F1 0.5867, Risk Recall 0.5353, mAP50 0.3386 으로 YOLOv11s 보다는 정확도가 다소 낮았지만, 평균 추론 시간 12.53ms, FPS 79.84, 최대 지연 시간 14.33ms 로 가장 빠른 추론 속도를 기록하였다. 경량 모델 특성상 모바일 및 실시간 서비스 환경에 적용하기에 유리하며, 제한된 연산 자원에서도 안정적인 동작 가능성이 높다는 점에서 실용성이 큰 모델로 평가되었다.

반면, RT-DETR-L 은 Risk Recall 0.8137, mAP50 0.5013 으로 위험 객체 탐지 성능 자체는 가장 높게 나타났으나, 평균 추론 시간 22.64ms, FPS 44.17, 최대 지연 시간 24.71ms 로 실시간성 측면에서 상대적으로 불리한 결과를 보였다. 또한 YOLOv11n 은 YOLOv8n 과 유사한 경량 계열 모델이지만, F1 0.5773, Risk Recall 0.5099 로 YOLOv8n 대비 뚜렷한 성능 우위를 보이지 못해 최종 우선 후보에서는 제외하였다.

종합적으로, 높은 정확도와 안정적인 탐지 성능을 고려할 때는 YOLOv11s 가 가장 적합하며, 빠른 추론 속도와 경량성을 중시하는 경우에는 YOLOv8n 이 효율적인 선택으로 판단된다. 이에

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

따라 본 프로젝트에서는 object detection 모델의 최종 상위 2 개 후보로 YOLOv11s 와 YOLOv8n 을 선정하였다. 이후에는 segmentation 모델 및 LLM 기반 상황 설명 모듈과의 조합 실험을 추가적으로 수행하여, 전체 시스템 관점에서 정확도·실시간성·확장성을 종합 평가한 뒤 최종 detection 모델을 선정할 예정이다.

종합적으로, 본 실험에서는 객체 탐지 정확도와 실시간 추론 성능을 함께 고려하여 최종 object detection 모델을 선정하였다. YOLOv11s 는 F1 0.7167, Risk Recall 0.6998, mAP50 0.4861 로 비교 모델 중 전반적으로 가장 균형 잡힌 성능을 보였으며, 평균 추론 시간 13.07ms, FPS 76.51, 최대 지연 시간 14.92ms 로 실시간 보행 보조 서비스에 적용 가능한 수준의 처리 속도를 확보하였다. 특히 본 프로젝트에서 중요한 요소인 위험 객체 탐지 성능 측면에서도 YOLOv8n 과 YOLOv11n 대비 높은 성능을 보였고, RT-DETR-L 에 비해서는 정확도 일부 지표가 낮았지만 훨씬 빠른 추론 속도를 보여 전체 시스템 적용 가능성이 더 높다고 판단하였다.

따라서 본 프로젝트에서는 정확도, 위험 상황 탐지 성능, 실시간성, 모바일 환경 적용 가능성을 종합적으로 고려하여 YOLOv11s 를 최종 object detection 모델로 선정하였다. YOLOv11s 는 경량 모델 수준의 빠른 추론 속도를 유지하면서도 YOLOv8n 보다 높은 탐지 정확도를 제공하므로, 보행 중 장애물과 위험 객체를 안정적으로 인식해야 하는 시각장애인 보행 보조 시스템에 적합하다. 최종 시스템에서는 YOLOv11s 를 기반으로 탐지된 객체 정보를 후속 모듈인 거리 추정, 위험도 판단, 상황 설명 및 음성 안내 생성 과정과 연계하여 사용자에게 필요한 위험 정보를 실시간으로 제공하도록 구성하였다.

최종 선정된 YOLOv11s 모델을 보행 보조 시스템 환경에 더욱 적합하게 만들기 위해 추가 학습을 수행하였다. 실제 보행 환경에서는 화면 가장자리에서 일부가 잘린 객체(Edge-touching)와 다른 물체에 가려진 객체(Occluded)가 빈번하게 발생하며, 이러한 어려운 케이스(Hard Sample)에 대한 탐지 누락은 시각장애인 안전과 직결되는 사고로 이어질 수 있다. 따라서 본 프로젝트에서는 Hard Sample Mining 기법과 2 단계 Curriculum Learning 을 결합한 추가 학습 전략을 적용하였다.

Hard Sample 은 다음 두 가지 기준으로 자동 식별하였다. (1) Edge-touching: bbox 의 좌표가 프레임 가장자리로부터 4px 이내에 위치한 경우 "잘린 객체"로 분류 (2) Occluded: AI HUB 원본 XML 라벨의 occluded=1 속성을 추출하여 "가려진 객체"로 분류 식별된 Hard Sample 을 포함한 이미지는 학습 매니페스트 단계에서 가중 오버샘플링(Weighted Oversampling)을 적용하였다.

- 일반 이미지 : ×1
- Occluded 포함 이미지: ×2
- Edge 차량 포함 이미지: ×3
- 두 조건 모두 만족 : ×4

이를 통해 어려운 케이스의 학습 노출 빈도를 최대 4 배까지 증가시켰다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

학습은 난이도에 따라 점진적으로 진행하는 Curriculum Learning 방식을 채택하여 2 단계로 구성하였다. Stage 1 (Generalization)에서는 전체 데이터를 사용하여 모델의 기본 표현 능력을 학습하는 단계로, 학습률 0.001, Mosaic 1.0, MixUp 0.05 의 강한 데이터 증강을 적용하여 다양한 보행 환경에 대한 일반화 성능을 확보하였다. Stage 2 (Hard Sample Fine-tuning)에서는 Stage 1 에서 산출된 best.pt 가중치를 출발점으로, Edge-Occluded 케이스만으로 구성된 Hard Subset 으로 미세조정을 수행하였다. 기존에 학습된 일반 표현을 손상시키지 않도록 학습률을 0.0003 으로 낮추고, Mosaic 을 0.3 으로 줄여 원본 데이터에 가까운 형태로 학습하였다.



1-2) Segmentation 모델 테스트 결과

본 프로젝트에서는 시각장애인의 보행 보조를 위해 보도, 점자블록, 횡단보도, 차도 등 보행 환경의 주요 영역을 인식하는 segmentation 모델을 개발하였다. 초기에는 SegFormer 기반 semantic segmentation 모델을 중심으로 보행 가능 영역을 분할하는 방식을 검토하였으나, 실시간 보행 안내 서비스에서는 추론 속도와 배포 구조의 단순성이 중요하다고 판단하였다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

이에 따라 최종적으로는 YOLO26-seg 기반 segmentation 모델을 사용하여 보도, 점자블록, 주의 노면, 차도, 횡단보도 영역을 분할하는 구조로 변경하였다.

최종 모델의 클래스는 실제 보행 안내에 필요한 정보를 중심으로 재구성하였다. 기존 데이터셋의 세부 라벨을 그대로 사용하는 대신, 서비스에서 직접 활용 가능한 **walkable_surface**, **braille_guide_blocks**, **caution_surface**, **roadway**, **crosswalk** 의 5 개 클래스로 정리하였다. 이를 통해 모델이 모든 배경 정보를 세밀하게 이해하는 것보다, 사용자의 안전한 이동 판단에 필요한 핵심 영역을 우선적으로 인식하도록 설계하였다.

최종 YOLO26-seg 모델 성능

구분	Box mAP50	Box mAP50-95	Mask mAP50	MaskmAP50-95
전체성능	0.750	0.600	0.730	0.548

클래스별 Mask 성능

클래스	설명	Mask mAP50	Mask mAP50-95
walkable_surface	보행 가능 영역	0.832	0.703
braille_guide_blocks	점자블록	0.776	0.609
caution_surface	주의 노면	0.664	0.412
roadway	차도	0.729	0.534
crosswalk	횡단보도	0.649	0.483

실험 결과, 최종 YOLO26-seg 모델은 전체 기준 Mask mAP50 0.730, Mask mAP50-95 0.548 의 성능을 보였다. 특히 보행 가능 영역인 **walkable_surface** 클래스는 Mask mAP50 0.832, 점자블록인 **braille_guide_blocks** 클래스는 Mask mAP50 0.776 을 기록하여, 실제 보행 안내에 중요한 핵심 클래스에서 비교적 안정적인 분할 성능을 확인할 수 있었다. 차도인 **roadway** 클래스 또한 Mask mAP50 0.729 로 양호한 성능을 보였으며, 보행 가능 영역과 차도 영역을 구분하는 데 활용 가능함을 확인하였다.

반면 **caution_surface** 와 **crosswalk** 클래스는 상대적으로 낮은 성능을 보였다. 이는 주의 노면과 횡단보도의 경우 촬영 각도, 조도, 노면 마모 상태, 가림 현상 등에 따라 형태가 다양하게 나타나기 때문으로 판단된다. 따라서 향후에는 해당 클래스에 대한 추가 데이터 확보와 라벨 품질 개선이 필요하다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14



점자블록 끊김 및 가림 감지 로직

본 프로젝트에서는 점자블록을 단순히 인식하는 것에서 나아가, 점자블록이 실제 보행 경로로 이어지고 있는지 판단하기 위한 후처리 로직을 추가하였다. YOLO26-seg 모델의 **braille_guide_blocks** segmentation mask 를 활용하여 점자블록 영역을 추출한 뒤, 현재 보이는 점자블록의 방향과 폭을 기반으로 예상 점자블록 corridor 를 생성하였다. 이후 예상 corridor 내부에서 실제 점자블록 mask 가 연속적으로 나타나는지 분석하여, 점자블록의 정상 연결 여부를 판단하였다.

점자블록 상태는 크게 **clear**, **blocked_or_occluded**, **broken**, **no_tactile** 로 구분하였다. **clear** 는 점자블록이 비교적 정상적으로 이어져 있는 경우이며, **blocked_or_occluded** 는 예상되는 점자블록 경로 중간에 큰 빈 구간이 발생하여 장애물에 의해 가려졌거나 알 수 없는 물체가 점자블록 위에 존재할 가능성이 있는 경우이다. **broken** 은 화면 하단에는 점자블록이 감지되지만 전방으로 이어지는 점자블록이 확인되지 않는 경우로, 사용자가 점자블록을 따라 이동하다가 경로가 끊길 수 있는 상황으로 판단하였다. **no_tactile** 은 점자블록 mask 자체가 거의 감지되지 않는 경우로 처리하였다.



또한 객체 탐지 결과와 점자블록 mask 를 함께 활용하여, 장애물이 점자블록 위에 위치하는지도 함께 판단하였다. 객체 바운딩 박스 전체가 아니라 하단 영역을 기준으로

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

점자블록 mask 와의 겹침 비율을 계산하였으며, 일정 비율 이상 겹치는 경우 해당 객체가 점자블록을 막고 있는 것으로 판단하였다. 이를 통해 단순히 점자블록이 존재하는지 여부만 확인하는 것이 아니라, 점자블록 위 장애물, 점자블록 가림, 점자블록 끊김 상황을 구분하여 안내할 수 있도록 하였다. 코드에서는 bbox 하단 영역과 점자블록 mask 가 5% 이상 겹치는 경우 점자블록 위 장애물로 판단하도록 설정하였다.



추론 속도 테스트 결과

추론 속도는 최종 서버 파이프라인 기준 전체 처리 시간과 입력 크기별 모델 추론 속도를 함께 확인하였다. 최종 서버 파이프라인 기준 평균 전체 처리 시간은 약 0.0827 초/장으로 측정되었으며, 이는 약 12.10 FPS 수준에 해당한다. 내부 모델 처리 기준으로는 preprocess 2.84ms, inference 12.51ms, postprocess 1.13ms 가 소요되었다.

내부 모델 처리 시간과 평균 전체 처리 시간에 차이가 발생한 이유는 모델 연산 외에도 이미지 로딩, 데이터 변환, 결과 시각화, 후처리 로직, 서버 응답 구성 등의 시간이 전체 처리 시간에 포함되기 때문이다. 따라서 내부 속도는 순수 모델 추론 성능을 확인하기 위한 지표로, 평균 전체 처리 시간과 FPS 는 실제 서비스 파이프라인에서의 처리 가능성을 판단하기 위한 지표로 활용하였다.

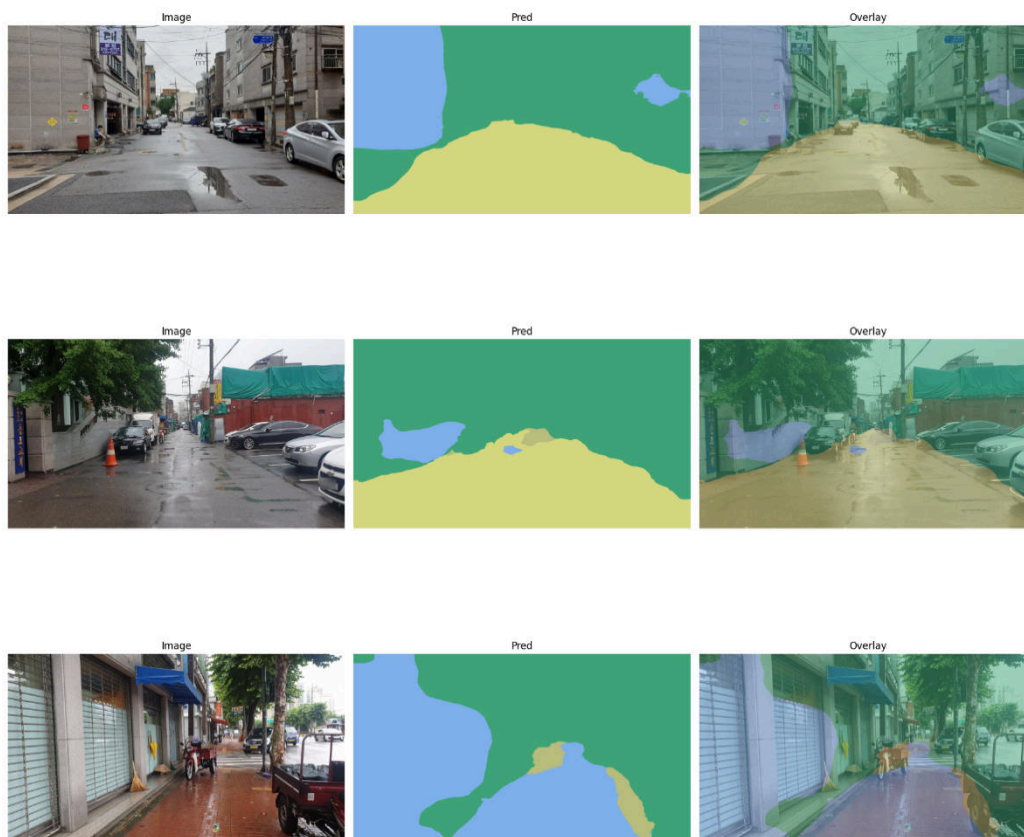
입력 크기별 추론 속도 비교

모델	imgsz	Avg Time	Inference	Boxes	Mask
YOLO26-seg-n	320	0.1334 초	48.18ms	6	o
YOLO26-seg-n	480	0.1190 초	40.69ms	5	o
YOLO26-seg-n	640	0.1369 초	55.91ms	6	o
YOLO26-seg-n	960	0.1743 초	92.19ms	6	o

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

추가로 YOLO26-seg-n 모델에 대해 입력 크기별 추론 속도를 비교하였다. 실험 결과 imgsz 480 에서 평균 처리 시간 0.1190 초, inference 40.69ms 로 가장 빠른 결과를 보였으며, 모든 입력 크기에서 mask 출력이 정상적으로 생성되는 것을 확인하였다. 일반적으로 입력 크기가 커질수록 더 많은 픽셀 정보를 활용할 수 있어 분할 결과가 정교해질 가능성이 있지만, 동시에 추론 시간이 증가하는 경향이 있다. 본 실험에서도 imgsz 960 의 경우 inference 시간이 92.19ms 까지 증가하여 입력 크기 확대가 실시간성 측면에서 부담이 될 수 있음을 확인하였다.

따라서 본 프로젝트에서는 정확도와 실시간성의 균형을 고려하여 imgsz 480 을 주요 테스트 기준으로 활용하였다. 이를 통해 YOLO26-seg 모델이 기존 SegFormer 기반 이중 구조보다 파이프라인을 단순화하면서도, 보행 안내에 필요한 핵심 노면 영역을 실시간에 가깝게 인식할 수 있는 가능성을 확인하였다.



후보 Segmentation 모델 비교 실험

최종 모델을 선정하기 전에는 다양한 semantic segmentation 모델을 대상으로 정확도와 실시간성을 함께 비교하였다. 실험 대상은 BiSeNetV2, SegFormer, DDRNet-23-slim, PIDNet-S 였으며, mIoU, Mean Accuracy, Pixel Accuracy, 평균 지연 시간, FPS 등을 기준으로 성능을 평가하였다.

모델	Epoch	mIoU	Mean Accuracy	Pixel Accuracy	Avg Latency	FPS	Max Latency
----	-------	------	---------------	----------------	-------------	-----	-------------

					(ms)		(ms)
BiSeNetV2 (후보)	20	0.1888	0.2652	0.7351	11.48	87.08	-
SegFormer (후보)	20	0.4479	0.5168	0.8667	18.73	53.38	27.59
SegFormer	2	0.1299	0.1820	0.7920	5.8812	170.0331	9.3252
DDRNet-23-slim	2	0.0509	0.0775	0.5993	-	-	-
PIDNet-S	2	0.0018	0.0468	0.0193	43.2641	23.1139	60.5805

후보 모델 비교 결과, SegFormer 는 mIoU 0.4479, Mean Accuracy 0.5168, Pixel Accuracy 0.8667 로 가장 높은 정확도를 보였다. 그러나 Transformer 기반 모델 특성상 연산량이 크고, 객체 탐지 모델과 함께 사용할 경우 전체 시스템 구조가 복잡해진다는 한계가 있었다. 특히 본 프로젝트는 보행 중 실시간 안내를 목표로 하기 때문에, 높은 정확도뿐만 아니라 추론 속도와 배포 가능성도 중요한 평가 기준이었다.

BiSeNetV2 는 SegFormer 보다 정확도는 낮았지만, 평균 지연 시간 11.48ms, FPS 87.08 로 빠른 추론 속도를 보였다. 따라서 실시간성이 중요한 환경에서는 적합한 후보가 될 수 있었으나, 보행 안내에 필요한 점자블록, 횡단보도, 보행 가능 영역을 안정적으로 구분하기에는 정확도 측면에서 한계가 있었다. DDRNet-23-slim 과 PIDNet-S 는 초기 실험에서 낮은 정확도를 보여, 추가적인 설정 보완이나 학습 조건 개선이 필요한 상태로 판단하였다.

이러한 비교 실험을 바탕으로 초기에는 SegFormer 를 주요 후보로 검토하였고, 이를 토대로 보행 환경 인식 파이프라인을 구축하였다. 그러나 4 월 11 일에 진행된 1 차 멘토링 이후 실시간성과 배포 구조를 고려하여 YOLO segmentation 단일 모델 구조로 방향을 변경하였다. 멘토링 과정에서는 SegFormer 가 Transformer 기반 모델이기 때문에 모바일·실시간 서비스 환경에서 연산량 부담이 크고, YOLO 와 SegFormer 를 함께 사용하는 이중 모델 구조는 시스템 자원 소모와 배포 복잡도를 높일 수 있다는 피드백을 받았다. 이에 따라 기존의 YOLO + SegFormer 구조를 재검토하고, YOLO 계열 segmentation 모델을 활용하여 주요 보행 환경 영역을 하나의 모델에서 처리하는 방향으로 구조를 단순화하였다.

다만 YOLO segmentation 은 semantic segmentation 모델처럼 화면 전체의 도로·보도 영역을 정밀하게 분할하는 데에는 한계가 있을 수 있다고 판단하였다. 따라서 본 프로젝트에서는 모든 배경 영역을 세밀하게 분할하는 방식보다, 실제 보행 안내에 직접적으로 필요한 정보를 우선적으로 인식하는 방향으로 전략을 수정하였다. 이에 따라 점자블록, 횡단보도, 보행 가능 영역, 주의가 필요한 노면 영역 등 안내 생성에 핵심이 되는 클래스를 중심으로 라벨 체계를 재구성하였다. 이는 모델의 목적을 단순한 장면 이해가 아니라, 시각장애인의 안전한 이동 판단을 보조하기 위한 핵심 영역 인식으로 명확히 설정한 것이다.

최종 판단

종합적으로, 최종 YOLO26-seg 모델은 보행 가능 영역, 점자블록, 차도, 횡단보도 등 시각장애인 보행 안내에 필요한 주요 노면 클래스를 하나의 모델에서 분할할 수 있음을

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

확인하였다. 특히 보행 가능 영역과 점자블록 클래스에서 비교적 안정적인 성능을 보여, 길벗 서비스의 보행 환경 인식 기능에 적용 가능한 가능성을 확인하였다.

기존 SegFormer 기반 구조는 후보 모델 중 정확도 측면에서 장점이 있었으나, 실시간 서비스 적용을 고려했을 때 연산량과 이중 모델 구성에 따른 배포 복잡도 문제가 있었다. 이에 비해 YOLO26-seg 기반 구조는 단일 모델로 핵심 보행 환경을 인식할 수 있어 최종 결과물에 더 적합한 구조로 판단하였다.

다만 횡단보도와 주의 노면 클래스는 상대적으로 낮은 성능을 보여, 향후 추가 데이터 확보와 라벨 품질 개선이 필요하다. 또한 다양한 날씨, 조도, 촬영 각도, 보행 환경에서의 추가 검증과 모바일 환경에서의 추론 속도 최적화가 필요하다. 따라서 현재 결과물은 상용 서비스 수준의 완성 모델이라기보다는, 길벗 서비스에서 보행 환경 인식 기능의 실현 가능성을 확인한 기능성 프로토타입으로 평가할 수 있다.

1-3) Depth Estimation 모델 테스트 결과

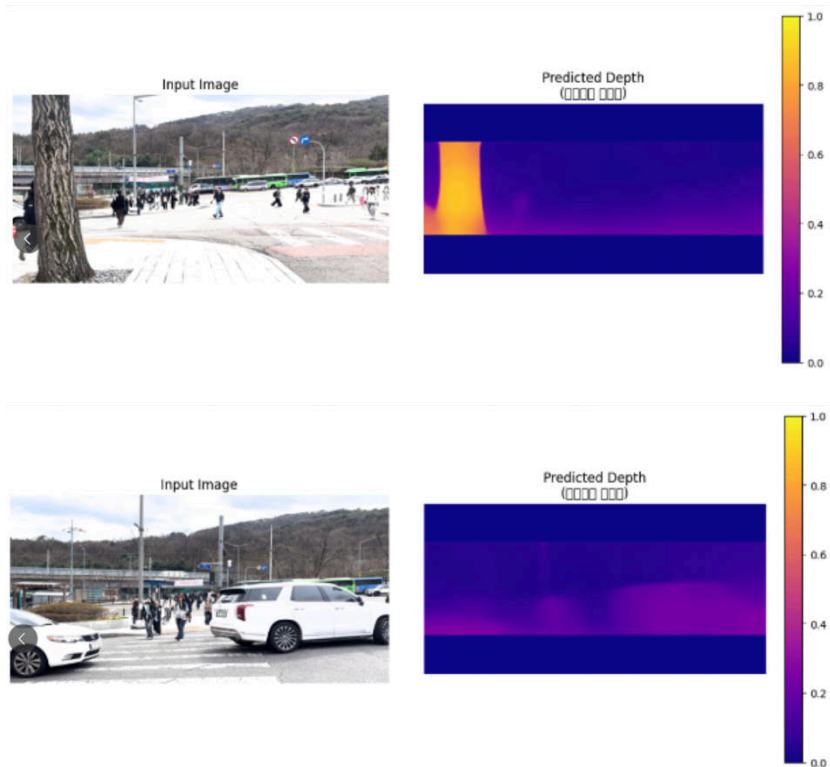
본 프로젝트에서는 객체 탐지 결과만으로는 실제 장애물과 사용자 간 거리를 정확히 판단하기 어렵다는 문제를 해결하기 위해 Depth Estimation 모델을 추가로 테스트하였다. 기존에는 bbox 의 크기나 화면 내 위치를 기반으로 상대적인 거리를 추정할 수 있었지만, 이 방식은 “작지만 가까운 객체”와 “크지만 먼 객체”를 구분하는 데 한계가 있었다. 특히 보행 보조 서비스에서는 단순히 화면에 객체가 보이는지보다, 해당 객체가 실제로 사용자와 충돌할 가능성이 있는 가까운 위험 요소인지 판단하는 것이 중요하다. 따라서 본 실험에서는 monocular depth estimation 기반으로 입력 이미지의 깊이값을 추정하고, 이를 객체 탐지 결과와 결합하여 실제 거리 기반 위험 판단을 수행하도록 구성하였다.

본 실험에서는 FastDepth, LiteMono, Depth Anything V2 Metric Outdoor 모델을 비교하였다.

FastDepth 실험 결과

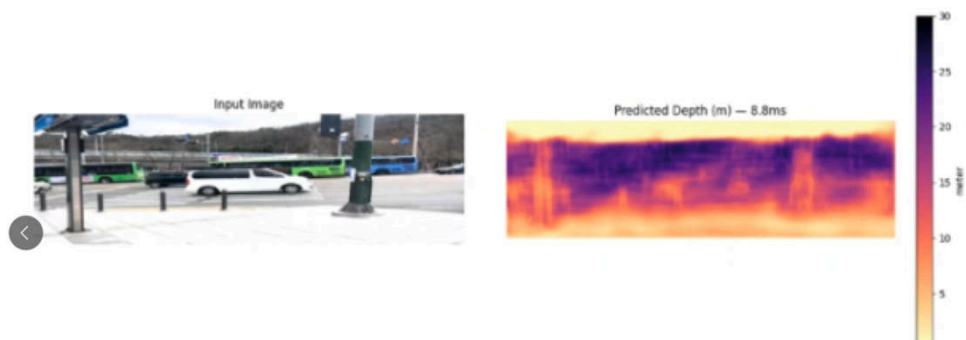
먼저 FastDepth 모델은 경량 모델이라는 장점이 있어 실시간 추론에는 유리했지만, 실험 결과 depth map 이 전체적으로 균일하게 출력되는 경향이 나타났다. 아래 실험 이미지에서 확인할 수 있듯이 객체별 깊이 차이가 뚜렷하게 드러나지 않았고, 차량이나 사람과 같은 개별 장애물의 형상이 명확히 분리되지 않았다. 이 경우 객체가 가까운지 먼지를 판단하기 어렵기 때문에, 보행 중 가까운 위험 객체만 선별하여 안내해야 하는 본 프로젝트의 목적에는 한계가 있었다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

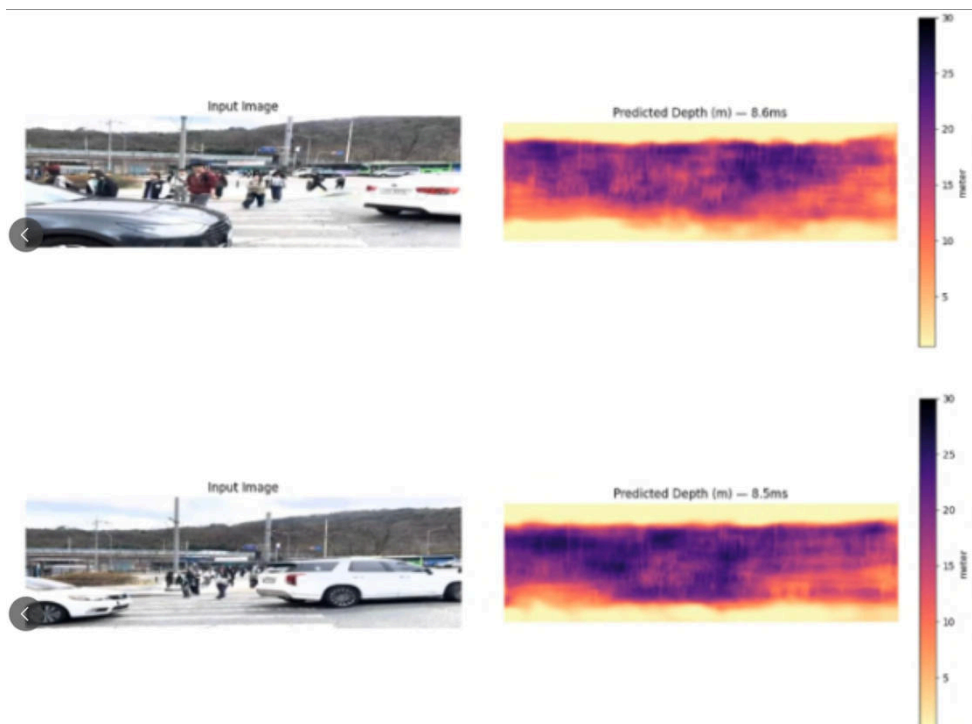


LiteMono 테스트 결과

두 번째로 LiteMono 모델을 테스트하였다. LiteMono 는 FastDepth 에 비해 장면 내 깊이 변화가 더 잘 표현되었고, 일정 수준의 거리값을 출력할 수 있었다. 그러나 아래 실험 이미지에서 확인할 수 있듯이 테스트 결과 일부 객체의 윤곽이 흐리게 나타나거나, 가까운 객체와 먼 객체의 상대적 거리 관계가 항상 안정적으로 유지되지는 않았다. 특히 도로 위 차량이나 사람처럼 크기와 위치가 다양한 객체에 대해 깊이값이 불안정하게 나타나는 경우가 있어, 본 프로젝트의 목적에는 한계가 있었다.

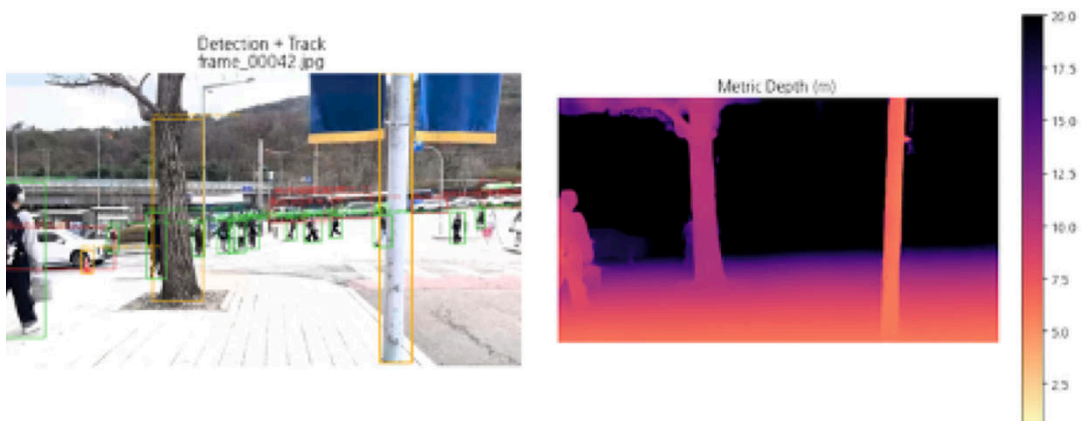


 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

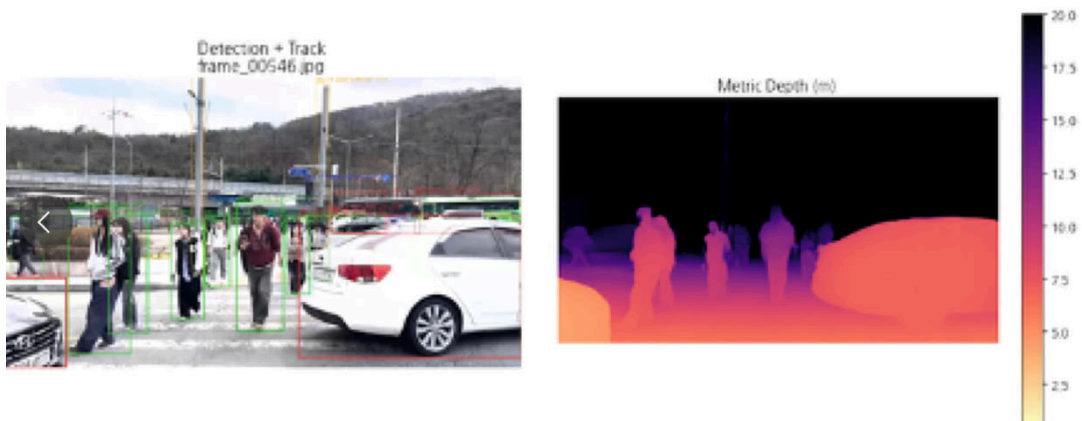


Depth Anything V2 Metric Outdoor 테스트 결과

최종적으로는 Depth Anything V2 Metric Outdoor 모델을 사용하였다. 해당 모델은 실외 환경에 적합한 metric depth 모델로, 실험 결과 FastDepth 와 LiteMono 에 비해 객체와 배경의 깊이 차이가 더 선명하게 나타났으며, 사람·차량·기둥 등 주요 객체의 형태가 비교적 명확하게 분리되었다. 아래 이미지에서 확인할 수 있듯이, Depth Anything V2 Metric Outdoor 모델은 전방 객체와 배경 사이의 깊이 차이를 더 안정적으로 표현하였고, 객체별 거리 추정 결과도 실제 보행 환경에서 활용 가능한 수준으로 나타났다. 따라서 본 프로젝트에서는 depth map 의 시각적 선명도뿐 아니라, 객체별 거리 판단의 정확성과 안정성을 종합적으로 고려하여 Depth Anything V2 Metric Outdoor 모델을 최종 depth estimation 모델로 선정하였다.



 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

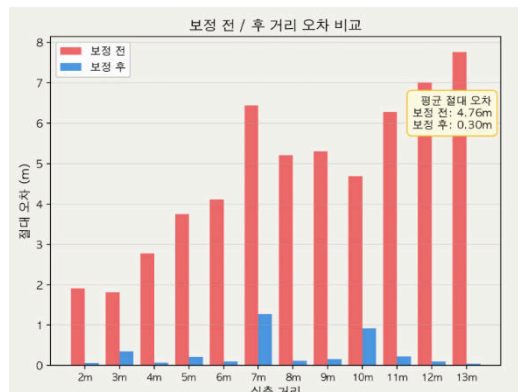
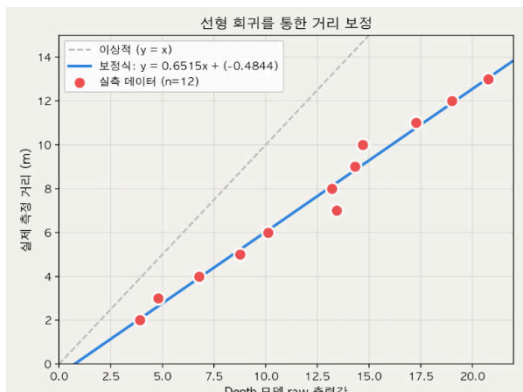


최종 시스템에서는 depth 모델명을 Depth-Anything-V2-Metric-Outdoor-Small-hf 로 설정하여 사용하였다. 또한 detection, segmentation 모델과 동일하게 설정 파일에서 입력 크기, 거리 임계값, 보정 계수 등을 통합 관리하도록 구성하였다. 입력 이미지 크기는 480px480 으로 설정하였으며, 거리 기준은 3.0m 이내를 긴급 거리, 5.5m 이내를 경고 거리, 8.0m 이내를 최대 안내 거리로 설정하였다. 이를 통해 너무 멀리 있는 객체는 음성 안내 대상에서 제외하고, 실제 보행 중 충돌 가능성이 있는 가까운 객체를 우선적으로 안내하도록 하였다. URGENT_DISTANCE_M, NEAR_M, MID_M 의 거리 기준이 정의되어 있으며, depth 모델명과 선형 보정 계수도 함께 관리된다.

거리 계산은 객체 탐지 모델의 bbox 와 depth map 을 결합하는 방식으로 수행하였다. 단순히 bbox 전체 영역의 평균 depth 를 사용하는 경우, 차량 아래 도로면이나 배경 픽셀이 함께 포함되어 실제 객체 거리보다 부정확한 값이 나올 수 있다. 이를 보완하기 위해 객체 bbox 전체가 아니라 하단부 영역을 중심으로 depth 값을 추출하였다. 구체적으로 bbox 하단 10% 영역을 사용하고, 해당 영역 안에서도 가장 가까운 25% 픽셀만 선별한 뒤 중앙값을 계산하였다. 이 방식은 객체 하단부가 실제 지면과 맞닿는 위치에 가깝다는 점을 활용한 것으로, 차량이나 사람의 실제 위치를 더 안정적으로 추정하기 위한 방법이다.

또한 Depth Anything V2 Metric Outdoor 모델의 출력값을 그대로 사용하지 않고, 실측 결과를 반영한 선형 보정을 적용하였다. 실제 테스트 과정에서 모델이 예측한 거리와 실제 거리 사이에 일정한 오차가 존재하는 것을 확인하였고, 이를 줄이기 위해 2m~13m 구간에서 12 개 지점의 실측 데이터를 수집해 “거리 = raw depth * scale factor + offset” 형태의 선형 보정식을 사용하였다. 현재 설정에서는 DEPTH_SCALE_FACTOR = 0.6515, DEPTH_OFFSET_M = -0.4844 를 적용하였다. 이를 통해 모델이 예측한 raw depth 값을 실제 보행 환경에 더 가까운 거리값으로 변환하였으며, 결과적으로 가까운 객체와 먼 객체를 더 정확히 구분할 수 있도록 하였다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14



작은 bbox 에 대한 보정도 추가하였다. bbox 영역이 작으면, 그 안에 있는 depth 픽셀 값 중 일부가 노이즈로 인해 잘못 추정될 수 있는 경다. 화면에서 작게 탐지된 객체는 depth 값 일부가 가까운 픽셀로 잘못 추정될 경우 실제보다 지나치게 가까운 위험 객체로 판단될 수 있다. 이를 방지하기 위해 bbox 높이가 전체 이미지 높이의 일정 비율보다 작은 경우에는 최소 신뢰 거리 기준을 적용하였다. 현재 설정에서는 작은 bbox 높이 비율 기준을 0.15 로 설정하고, 작은 객체가 지나치게 가까운 거리로 판단될 경우 최소 8.0m 이상의 거리로 보정하도록 하였다. 이를 통해 멀리 있는 작은 표지판이나 배경 객체가 불필요하게 음성 안내 대상으로 포함되는 문제를 줄이고자 하였다.

Depth map 은 객체별 거리 계산뿐 아니라 화면 영역별 안전도 분석에도 활용하였다. depth map 의 하단 ROI 영역을 좌측, 중앙, 우측 세 구역으로 나누고, 각 영역에서 3.0m 이내의 픽셀 수와 5.5m 이내의 픽셀 수를 계산하였다. 가까운 픽셀이 일정 개수 이상 존재하면 해당 영역을 urgent 또는 alert 상태로 판단하고, 가까운 장애물이 충분히 감지되지 않는 영역은 safe 상태로 분류하였다. 이후 객체의 방향과 안전한 영역 정보를 함께 고려하여, 사용자가 어느 방향으로 우회할 수 있는지를 판단하였다. 예를 들어 12 시 방향에 가까운 차량이 있다고 판단되면, “12 시 방향 차량, 2 시 우회”와 같은 안내가 가능하도록 구성하였다.

추론 속도도 함께 측정하였다. 총 93 장의 프레임을 대상으로 단계별 처리 시간을 측정한 결과, depth inference 는 평균 32.0ms, 최소 30.9ms, 중앙값 32.1ms, 최대 33.1ms 로 비교적 안정적인 처리 시간을 보였다. 전체 파이프라인의 평균 처리 시간은 117.7ms 였다.

1-4) 위험도 판단 로직

본 프로젝트에서는 객체 탐지, segmentation, depth estimation 결과를 종합하여 사용자에게 음성으로 안내할 위험 객체를 선정하는 단계가 필요하다. 그러나 화면에 탐지된 모든 객체를 단순히 거리 순으로 안내할 경우, 실제 보행과 무관한 객체까지 모두 경고하게 되어 알림이 과도하게 발생하는 문제가 발생한다. 특히 차도 위 차량, 멀리 보이는 표지판, 보행 시야 바깥의 객체와 같이 사용자의 직접적인 보행 안전에는 영향이 적은 객체까지 모두 안내될 경우, 안내 정보의 신뢰도가 떨어지고 사용자가 정작 중요한 위험 상황을 인지하지 못할 수 있다. 이러한 문제를 해결하기 위해 본 프로젝트에서는 단순한 거리 기반 판단이 아니라, 다차원

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

가중치 기반의 위험도 스코어링 방식을 도입하여 각 객체의 위험도를 정량화하고, 가장 위험한 단 하나의 객체를 우선적으로 안내하도록 설계하였다.

위험도 점수는 객체의 종류, 거리, 화면 점유 비율, 시야 방향, 보행 경로 점유 여부, 차도 점유 여부, 차량 접근 속도, 점자블록 점유 여부의 7 가지 요인을 점수화하여 합산하는 방식으로 산정하였다. 최종 위험도 점수는 다음과 같이 계산된다.

$$\text{risk_score} = \text{base} + \text{dist_bonus} + \text{size_bonus} + \text{view_bonus} + \text{path_bonus} + \text{road_penalty} + \text{approach_bonus} + \text{tactile_bonus}$$

먼저 base 점수는 객체의 종류에 따라 부여하였다. 차량은 5.0 점, 일반 장애물은 3.0 점, 신호등은 1.0 점으로 설정하여, 충돌 시 사용자에게 더 큰 위협이 될 수 있는 객체에 더 높은 기본 점수를 부여하였다. 사람의 경우 일반 보행자는 적극적인 회피 안내 대상이 아니라고 판단하여 위험도 산정 대상에서 제외하였다.

dist_bonus 는 사용자와 객체 사이의 거리값에 따라 부여하였다. Depth Anything V2 Metric Outdoor 모델로부터 추정된 거리값을 기준으로, 3m 이내인 긴급 거리에서는 10.0 점, 5.5m 이내인 경고 거리에서는 6.0 점, 8.0m 이내인 최대 안내 거리에서는 2.0 점을 부여하고, 그 이상의 거리에서는 추가 점수를 부여하지 않았다. 거리 자체가 위험도를 결정하는 가장 직접적인 요인이기 때문에 다른 항목보다 가중치를 크게 설정하였다.

size_bonus 는 객체의 화면 점유 비율과 거리값을 결합하여 부여하였다. bbox 면적이 전체 화면 면적의 20% 이상이면서 거리가 4m 이내인 경우 4.0 점을 추가로 부여하였다. 이는 단순히 거리값만으로 판단할 때 발생할 수 있는 오차를 보완하기 위한 항목으로, 화면을 크게 차지한다는 것은 사용자에게 매우 가까이 존재할 가능성이 높음을 의미하기 때문이다.

view_bonus 는 객체가 사용자의 보행 시야 내에 있는지 여부를 반영하였다. bbox 중심점의 x 좌표를 시계 방향(clock direction)으로 변환한 뒤, 9시부터 3 시 사이에 해당하는 정면 보행 시야 영역에 객체가 위치하면 2.0 점을 부여하고, 그 외 시야 바깥에 위치한 객체에는 -3.0 점의 페널티를 적용하였다. 이를 통해 사용자의 진행 방향과 무관한 측후방 객체가 불필요하게 우선순위에 포함되는 문제를 줄였다.

path_bonus 와 road_penalty 는 segmentation 결과를 활용하여 부여하였다. 객체의 bbox 하단부가 walkable_surface 영역과 일정 비율 이상 겹치면 보행 경로 상에 위치한 것으로 판단하여 1.0 점을 부여하였고, 보행 경로와 거리가 먼 객체에는 -1.0 점의 페널티를 적용하였다. 반대로 차량이 roadway 영역과 30% 이상 겹치고 거리가 4m 를 초과하는 경우에는 -3.0 점의 road_penalty 를 적용하여, 차도 위에서 정상적으로 주행 중인 차량이 무차별적으로 경고 대상이 되는 문제를 방지하였다.

approach_bonus 는 차량의 접근 속도를 반영한 항목이다. 단순히 화면에 차량이 보이는지 여부보다, 실제로 사용자에게 빠르게 접근하고 있는 차량이 훨씬 위험하다는 점을 반영한 것이다. 객체 추적 결과로부터 차량의 프레임 간 거리 변화량을 계산하여 접근 속도(approach

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

speed, m/s)를 추정하고, 접근 속도가 임계값을 초과하면서 최소 추적 프레임 수를 만족하고, 거리가 6m 이내인 경우에 3.0 점을 부여하였다. 이를 통해 정지해 있는 차량과 사용자에게 다가오는 차량을 구분하여 안내할 수 있도록 하였다.

마지막으로 tactile_bonus 는 점자블록 위에 객체가 위치하는지 여부를 반영한 항목이다. 점자블록은 시각장애인 보행에서 가장 핵심적인 안내 수단이므로, 점자블록 위에 장애물이 위치하는 상황은 일반적인 보행 경로 침범보다 더 높은 위험으로 판단해야 한다. 따라서 객체 bbox 하단부와 점자블록 mask 의 겹침 비율이 5% 이상인 경우 4.0 점의 가산점을 부여하였다.

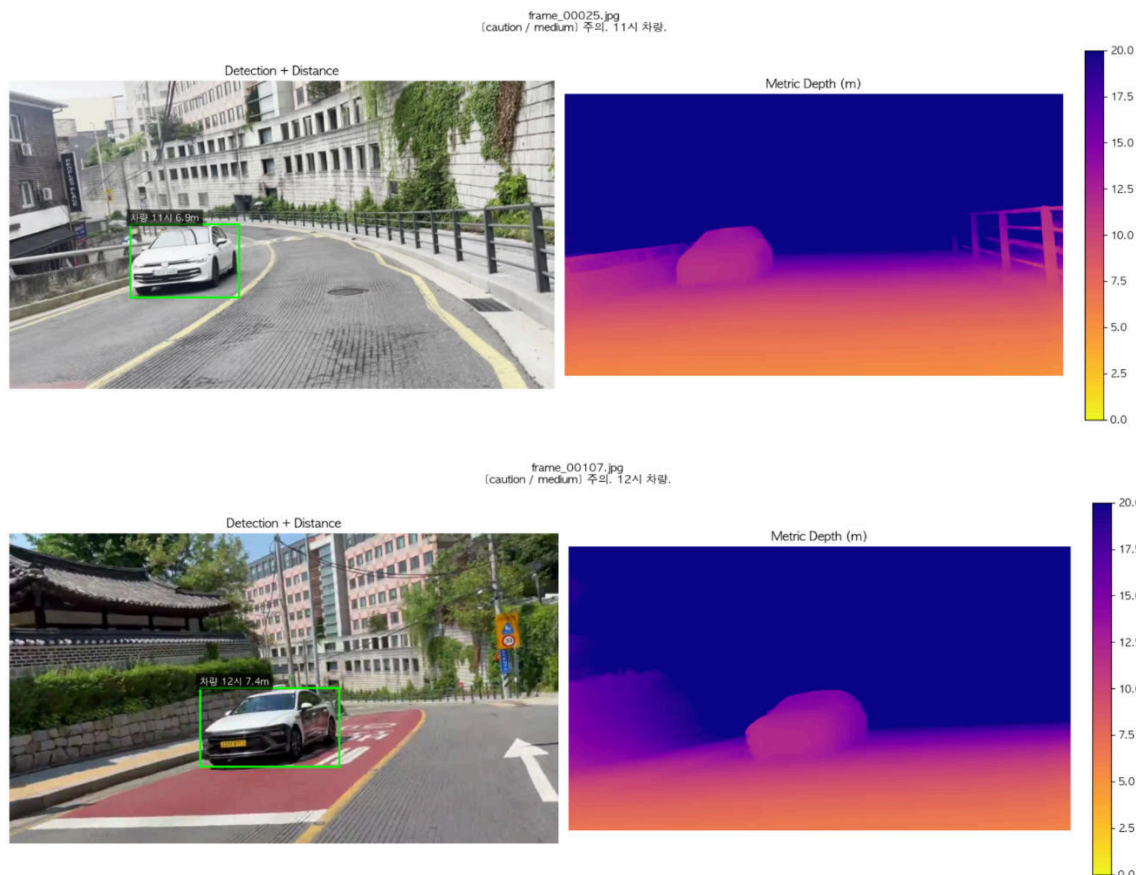
산정된 위험도 점수를 바탕으로 안내 등급은 3 단계로 결정하였다. risk_score 가 4.0 점 이상이면 긴급 조건(3m 이내 / 화면을 크게 차지하는 객체 / 빠르게 접근 중인 차량)을 만족하는 경우에는 STOP(high) 등급으로 분류하여 “정지!”로 시작하는 강한 경고 문장을 생성하였다. risk_score 가 5.0 점 이상이면 위험 조건을 만족하는 경우에는 DETOUR / CAUTION(medium) 등급으로 분류하여 우회 방향을 함께 안내하였다. 그 외의 경우에는 GO(safe / low) 등급으로 분류하여 별도의 경고를 발생시키지 않도록 하였다. 우회 방향은 segmentation 기반 좌·중·우 영역별 안전도 분석 결과(zones)에서 safe 상태인 영역을 우선적으로 선택하도록 구성하여, 단순히 반대 방향으로 우회하는 것이 아니라 실제로 가까운 장애물이 적은 방향을 안내하도록 하였다.

또한 점자블록 위에 장애물이 감지된 경우에는 risk_score 계산 과정을 건너뛰고 즉시 최우선 안내로 전환하도록 구성하였다. 구체적으로는 (1) 점자블록 위 장애물 감지, (2) 점자블록 가림·막힘, (3) 점자블록 끊김의 세 가지 상황에 대해 별도의 안내 분기를 두었다. 이는 시각장애인 보행에서 점자블록 관련 이슈가 일반 장애물보다 직접적인 안전 위협으로 이어질 수 있다는 점을 반영한 설계이다.

동일한 객체에 대한 안내가 반복되는 문제를 방지하기 위해 객체별 cooldown 로직도 함께 적용하였다. 동적 객체(차량, 사람 등)는 10 프레임, 정적 객체(볼라드, 화분 등)는 20 프레임의 cooldown 을 부여하여, 한 번 안내된 객체는 일정 시간 동안 다시 안내되지 않도록 하였다. 또한 동일한 voice_guide 문장이 10 초 이내에 재생되지 않도록 음성 텍스트 단위의 중복 방지 로직도 함께 적용하여, 사용자가 짧은 시간 동안 같은 안내를 반복적으로 듣는 알림 피로 문제를 최소화하였다.

종합적으로, 본 프로젝트의 위험도 판단 로직은 단순한 거리 기반 안내가 가지는 “모든 탐지 객체에 대해 경고가 발생하는” 한계를 극복하기 위해, 객체 종류·거리·크기·시야·보행 경로·차도 위치·접근 속도·점자블록 점유 여부의 7 가지 요인을 가중치로 결합한 다차원 스코어링 구조로 설계되었다. 이를 통해 화면 내 다수의 객체 중 실제로 사용자의 보행 안전에 직결되는 단 하나의 객체를 선별하여 안내할 수 있게 되었으며, cooldown 과 중복 안내 방지 로직을 결합하여 알림 빈도와 사용자 피로도를 동시에 관리할 수 있도록 구성하였다. 결과적으로 본 위험도 판단 로직은 시각장애인 보행 보조 시스템에서 “무엇을, 언제, 어떻게” 안내할지를 결정하는 핵심 모듈로서, 앞 단계의 detection·segmentation·depth 결과를 사용자에게 의미 있는 안내 정보로 변환하는 역할을 수행한다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14



최종 판단

종합적으로, 본 실험에서는 FastDepth, LiteMono, Depth Anything V2 Metric Outdoor 모델을 비교한 결과, Depth Anything V2 Metric Outdoor 모델이 가장 안정적인 depth map 과 객체별 거리 추정 결과를 보였다. FastDepth 는 경량성은 우수하지만 깊이 표현이 지나치게 단순하여 객체별 거리 구분이 어려웠고, LiteMono 는 FastDepth 보다 개선된 결과를 보였으나 객체 윤곽과 거리값의 안정성 측면에서 한계가 있었다. 반면 Depth Anything V2 Metric Outdoor 모델은 실외 보행 환경에서 객체와 배경의 깊이 차이를 더 명확하게 표현하였고, 선형 보정을 통해 실제 거리와의 오차를 줄일 수 있었다.

따라서 본 프로젝트에서는 최종 depth estimation 모델로 Depth Anything V2 Metric Outdoor 모델을 선정하였다. 최종 시스템에서는 해당 모델의 depth map 을 YOLOv11s 기반 객체 탐지 결과, YOLO26-seg 기반 보행 영역 분할 결과와 결합하여 객체별 거리, 좌·중·우 안전 영역, 위험도 판단, 회피 방향 안내에 활용하였다. 또한 단순한 거리 기반 안내가 가지는 한계를 보완하기 위해, 객체 종류·거리·화면 점유 비율·시야 방향·보행 경로·차도 점유·차량 접근 속도·점차블록 점유 여부의 7 가지 요인을 가중치로 결합한 다차원 위험도 스코어링 로직을 적용하여 가장 위험한 단 하나의 객체를 선별하도록 구성하였다. 이를 통해 단순히 화면에 보이는 모든 객체를 안내하는 것이 아니라, 실제로 사용자와 가까운 위험 요소를 우선적으로 판단하고, cooldown 및 중복 안내 방지 로직과 결합하여 불필요한 원거리 객체 안내와 알람 피로 문제를 동시에 줄이는 방향으로 보행 안내의 정확성과 실용성을 개선하였다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

2) 장애물 회피 방향 안내 기능

본 프로젝트에서는 객체 탐지, 보행 가능 영역 분석, 깊이 추정 결과를 통합하여 사용자가 실제로 회피해야 하는 위험 상황을 판단하고, 이에 따른 음성 안내 문장을 생성하는 장애물 회피 방향 안내 기능을 구현하였다. 해당 기능은 단순히 화면 내 객체를 탐지하는 수준을 넘어, 객체의 위치, 거리, 이동 여부, 보행 경로 침범 여부, 점자블록 가림 여부 등을 종합적으로 분석하여 사용자에게 필요한 안내만 제공하도록 설계하였다.

먼저 객체 탐지 모델로는 최종 선정된 YOLOv11s 기반 detection 모델을 사용하였으며, segmentation 모델과 depth estimation 모델을 함께 연동하였다. 시스템 설정 파일에서는 detection 모델 가중치, segmentation 모델 가중치, depth 모델명을 각각 관리하도록 구성하였고, 입력 이미지 크기는 480px 으로 설정하였다. 또한 안내 생성에 사용되는 confidence 기준, 거리 임계값, 추적 및 cooldown 관련 설정값을 별도로 정의하여 실험 및 실제 서비스 환경에 따라 유연하게 조정할 수 있도록 하였다. 특히 위험 안내 기준은 3m 이내를 긴급 거리, 5.5m 이내를 경고 거리, 8m 이내를 최대 안내 거리로 설정하여, 너무 멀리 있는 객체까지 불필요하게 안내하지 않도록 하였다.

장애물의 방향은 탐지된 객체의 bbox 중심 좌표를 기준으로 계산하였다. 화면의 가로 위치를 기준으로 객체가 사용자 시야에서 어느 방향에 있는지를 판단하고, 이를 9 시, 10 시, 11 시, 12 시, 1 시, 2 시, 3 시와 같은 시계 방향 표현으로 변환하였다. 이를 통해 사용자가 “왼쪽/오른쪽”보다 더 직관적으로 장애물의 위치를 인지할 수 있도록 하였다. 또한 객체의 위치를 left, center, right 영역으로 나누어 보행 방향 판단에 활용하였다.

거리 판단의 경우, 기존의 bbox 크기 기반 추정 방식만으로는 실제 거리 판단에 한계가 있었기 때문에 depth map 을 함께 활용하였다. 객체 bbox 내부에서 전체 영역을 그대로 사용하는 것이 아니라, 객체의 하단부를 중심으로 depth 값을 추출하고, 그중 가까운 픽셀 일부를 선별하여 중앙값을 계산하였다. 이를 통해 차량 하단의 도로면처럼 실제 객체가 아닌 배경 픽셀이 거리 계산에 섞이는 문제를 줄이고자 하였다. 또한 실측 결과를 반영하기 위해 depth scale factor 와 offset 을 적용하여 거리 값을 선형 보정하였다. 작은 bbox 가 과도하게 가깝게 판단되는 문제를 완화하기 위해 작은 객체에 대해서는 최소 신뢰 거리 기준도 함께 적용하였다.

보행 가능 경로 판단에는 segmentation 결과를 활용하였다. segmentation 클래스는 보행 가능 영역, 점자블록, 주의 영역, 차도, 횡단보도로 구분하였으며, 각 mask 를 기반으로 path mask, road mask, tactile mask 를 생성하였다. 이후 객체 bbox 와 보행 가능 영역 mask 의 겹침 비율을 계산하여 해당 객체가 실제 보행 경로 위에 있는지 판단하였다. 동시에 road mask 와의 겹침도 함께 확인하여, 차도에 있는 차량처럼 사용자의 보행 경로와 직접적으로 충돌하지 않는 객체가 과도하게 위험 객체로 판단되지 않도록 하였다.

위험도 산정은 객체 종류, 거리, 방향, 크기, 보행 경로 침범 여부, 차도 위치 여부, 접근 여부를 종합하여 수행하였다. 차량, 사람, 고정 장애물, 신호등을 서로 다른 위험 유형으로 분류하고, 위험 점수 계산 단계에서는 차량 계열 객체에 가장 높은 기본 점수를 부여하고, 고정

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

장애물, 신호등, 기타 객체 순으로 위험도를 차등 적용하였다. 또한 3m 이내의 객체에는 높은 거리 가중치를 부여하고, 5.5m 이내 또는 8m 이내 객체에는 단계적으로 낮은 가중치를 적용하였다. 화면 중앙 보행 시야에 해당하는 9 시부터 3 시 방향의 객체는 위험도를 높이고, 보행 경로와 겹치지 않는 객체는 위험도를 낮추었다. 이와 함께 tracking 정보를 이용하여 객체의 이동 여부와 접근 속도를 계산하고, 일정 거리 이내에서 접근 중인 차량은 정지 안내 대상으로 우선 처리하였다.

회피 방향은 depth map 기반으로 좌측, 중앙, 우측 영역의 안전도를 분석한 뒤 결정하였다. 각 영역에서 가까운 픽셀이 일정 개수 이상 존재하면 해당 영역을 urgent 또는 alert 상태로 판단하고, 그렇지 않은 영역은 safe 상태로 분류하였다. 이후 위험 객체의 방향과 안전한 영역 정보를 비교하여, 위험 객체를 피할 수 있는 우회 방향을 선택하였다. 예를 들어 12 시 방향에 장애물이 있고 오른쪽 영역이 안전하다고 판단되면 “12 시 방향에 차량이 있습니다. 2 시 방향으로 우회하세요.”와 같은 안내를 생성한다. 안전한 우회 방향이 없는 경우에는 무리한 이동을 유도하지 않고 “정지” 또는 “주의” 안내를 출력하도록 구성하였다.

또한 시각장애인 보행 환경에서 중요한 점자블록 관련 위험을 별도로 우선 처리하였다. 객체 bbox 의 하단 영역과 점자블록 mask 가 5% 이상 겹치는 경우, 해당 객체가 점자블록 위에 놓여 있는 것으로 판단하였다. 이 경우 일반 장애물 안내보다 우선하여 “주의. 12 시 점자블록 위 스쿠터, 오른쪽 우회.”와 같은 안내 문장을 생성한다. 만약 객체 탐지 결과가 없더라도 점자블록 mask 가 중간에서 끊기거나 가려진 경우에는 점자블록 위 장애물 또는 가림 상황으로 판단하고 별도의 경고를 제공하였다. 이를 통해 탐지 모델이 특정 장애물을 명확히 분류하지 못한 경우에도, 점자블록이 가려진 위험 상황을 보완적으로 안내할 수 있도록 하였다.

음성 안내 문장은 규칙 기반으로 생성하였다. 안내 결과는 warning_needed, primary_object_class, clock_direction, distance, action, priority, reason, voice_guide 등의 구조화된 필드로 구성되며, 최종적으로 백엔드에 전달할 수 있는 payload 형태로 변환된다. 안내 수준은 행동 유형에 따라 high, medium, low, safe 로 구분하였다. stop 은 high, detour 와 caution 은 medium, 단순 장면 정보는 low, 안내가 필요 없는 경우는 safe 로 처리하였다. 또한 동일 객체나 동일 장면 정보가 반복적으로 안내되는 문제를 줄이기 위해 dynamic object 와 static object 에 서로 다른 cooldown frame 을 적용하였다.

최종적으로 본 기능은 YOLOv11s 기반 객체 탐지 결과, segmentation 기반 보행 가능 영역, depth 기반 거리 추정, tracking 기반 이동 판단을 결합하여 실제 보행 중 위험도가 높은 객체만 선별적으로 안내하도록 구현되었다. 이를 통해 사용자가 모든 탐지 객체에 대해 과도한 음성 안내를 받는 문제를 줄이고, 정지해야 하는 상황과 우회 가능한 상황을 구분하여 보다 실용적인 보행 보조 안내를 제공할 수 있었다. 특히 점자블록 위 장애물, 접근 중인 차량, 보행 경로를 막는 가까운 객체를 우선적으로 처리함으로써 시각장애인 보행 환경에서 중요한 안전성과 사용성을 함께 고려한 안내 기능을 구현하였다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

3) 길찾기 기능

사용자의 실시간 GPS 위치와 설정된 목적지 데이터를 기반으로, 보행 환경에 최적화된 최단 및 안전 경로를 산출하고 안내하는 백엔드 시스템을 구축한다.

본 기능에서는 SK Open API(Tmap)의 보행자 전용 경로 탐색 엔드포인트를 활용하여, 복잡한 도심 환경에서도 이면도로, 계단, 횡단보도 등 보행자 전용 도로 특성을 반영한 경로 데이터를 확보한다. 단순히 좌표 간 직선거리를 계산하는 것이 아니라, 실제 보행이 가능한 네트워크 링크 정보를 분석하여 사용자에게 전달하는 것이 핵심이다.

특히 API로부터 반환되는 방대한 GeoJSON 데이터를 가공하여, 프론트엔드에서 즉시 시각화하거나 음성 안내로 변환하기 적합한 핵심 정보(이동 거리, 예상 소요 시간, 회전 지점 명칭, 구간별 안내 메시지 등)만을 선별하여 최적화된 인터페이스(Restful API)를 구현한다.

연구·개발 방법은 다음과 같다.

- ① API 인터페이스 설계 및 인증 단계: SK Open API 서버와의 안정적인 통신을 위해 인증 키 관리 로직을 구현하고, 보행자 경로 전용 엔드포인트를 호출하는 HTTP 통신 모듈을 개발한다.
- ② 데이터 요청 및 파라미터 최적화 단계: 사용자의 현재 위도·경도와 목적지 위도·경도를 기반으로 출발지/목적지 명칭, 좌표계(WGS84GEO), 경로 탐색 옵션 등을 Request Parameter로 구성하여 전달한다.
- ③ GeoJSON 파싱 및 데이터 정제 단계: API 응답으로 수신되는 다량의 좌표 데이터와 속성 정보(Point, LineString) 중 보행 안내에 필수적인 요소를 추출하는 파싱 알고리즘을 구현한다.
- ④ 프론트엔드 최적화 응답 생성 단계: 클라이언트(앱)의 통신 부하를 줄이기 위해, 정제된 데이터를 사용자 맞춤형 JSON 구조로 재구성한다.

4) 음성 및 햅틱 안내 시스템

본 서비스는 음성 안내와 햅틱 피드백을 결합한 멀티모달 안내 방식을 통해 사용자의 상황 인지 능력을 높이고 보행 안전성을 향상시키는 것을 목표로 한다.

① 음성 안내 시스템 (TTS)

AI는 입력된 실시간 영상 이미지를 기반으로 객체 탐지와 보행 가능 영역 분할을 수행하고, 그 결과를 종합하여 현재 상황을 텍스트 형태의 안내 정보로 생성한다. 이 과정에서 위험 객체,

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

방향, 회피 가능 경로를 우선적으로 판단하여 사용자에게 필요한 짧고 명확한 문장으로 정리한다. 최종 안내 문장은 음성 합성(TTS)과 햅틱 피드백 시스템에 전달되어 화면 확인 없이도 정보를 인지할 수 있도록 지원한다.

안내 음성은 TTS(Text-to-Speech) 기술을 활용하여 생성하며, 다음과 같은 상황에서 사용자에게 정보를 전달한다.

- 장애물 탐지 시 위험 안내 — 객체 종류·방향·거리를 결합한 안내 문장
(예: "정지! 12 시 방향 자전거")
- 장애물이 회피 방향 안내 — stop·caution·detour·go 판단 결과에 따른 행동 안내
(예: "2 시 방향으로 우회")
- 길찾기 경로 안내 — 구간별 방향 및 거리 TTS (예: "100m 후 좌회전.")
- 신호등 상태 안내 — 신호등 존재 여부, 음향신호기 존재 여부 안내

② 햅틱 안내 시스템 (진동)

보행 중 소음 환경에서도 사용자가 위험 상황을 인지할 수 있도록 햅틱(진동) 피드백 기능을 함께 제공하여 다음과 같은 패턴으로 진동을 차별화하여 제공한다.

- 장애물 감지 시 짧은 경고 진동 — caution 수준 장애물 탐지 시 짧은 단속 진동
- 긴급 위험 상황 발생 시 연속 진동 — stop 판단 시 강하고 연속적인 진동

노원복지관 인터뷰에서 진동 강도 조절 기능에 대해 긍정적인 반응을 받았으며, 가까운 장애물일수록 강한 진동을 제공하는 현행 방식도 충분히 효과적이라는 평가를 받았다.

5) 사용자 인터페이스

시각장애인의 사용성을 고려한 접근성 중심 모바일 사용자 인터페이스를 설계하고 구현한다. 사용자 인터페이스는 Flutter 기반 모바일 UI 로 구현하며, 다음과 같은 접근성 원칙을 적용한다.

① 최소 조작 인터페이스

시각장애인의 사용 편의를 위해 큰 버튼 중심의 단순한 인터페이스를 설계한다. 또한 주요 기능이 최소한의 조작 단계로 실행될 수 있도록 화면 구조를 단순화하고, 불필요한 화면 전환을 최소화한다.

② 음성 중심 인터랙션

사용자가 화면을 직접 확인하지 않고도 서비스를 이용할 수 있도록 음성 안내 중심의 사용자 경험(UX)을 제공한다.

주요 기능 수행 시 음성 안내를 통해 현재 상태와 결과를 전달하며, 버튼을 클릭할 경우에도 음성 피드백을 제공하여 사용자가 현재 기능을 수행 상태를 인지할 수 있도록 설계한다.

③ 접근성 가이드라인 적용

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

UI 설계 시 WCAG(Web Content Accessibility Guidelines)의 접근성 기준을 참고하여 다음 요소를 적용한다.

- 텍스트와 배경 사이의 명도 대비 최소 4.5:1 (큰 글씨는 3:1) 이상
(저시력자 및 색맹·색약 사용자를 모두 고려)
- 정보 전달 시 색상에만 의존하지 않고 아이콘 또는 텍스트를 병행하여 제공
- 충분한 크기의 버튼 인터페이스
- 햅틱 피드백(Haptic Feedback) 제공
- 큰 글자 크기(최소 18pt 이상) 적용
- 명확한 의미를 전달하는 아이콘 사용
- 단순하고 직관적인 레이아웃 및 화면 구성

④ 보행 중 사용을 고려한 인터페이스

보행 중 사용 환경을 고려하여 실시간 안내 화면을 단순화하고, 사용자가 화면을 지속적으로 확인하지 않아도 되도록 설계한다. 특히 위험 탐지 및 경로 안내 정보는 음성 중심으로 제공하여 사용자가 안전하게 이동할 수 있도록 지원한다.

⑤ 스크린 리더(Screen Reader) 지원

시각장애인 사용자가 스크린 리더(Screen Reader)를 통해 인터페이스를 탐색할 수 있도록 접근성을 고려한 UI 구조를 설계한다. Android 환경에서 TalkBack 을 통해 UI 요소의 정보를 음성으로 안내받을 수 있도록 지원한다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

UI 디자인 개선 과정

본 UI 는 총 세 차례에 걸쳐 개선되었으며, 각 변경은 기술적 요인 및 실사용자 피드백을 근거로 수행되었다.

[최초 디자인]



 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

[1 차 변경] 장애물 탐지 화면 세로→가로(랜드스케이프) 모드 전환



초기 디자인은 세로(포트레이트) 모드 기반으로 구성되었다. 그러나 수행계획서 검토 과정에서 세로 모드 카메라는 좌우 시야각이 협소하여 보행 방향 양측에 위치한 장애물을 동시에 인식하기 어렵다는 기술적 한계가 확인되었다. 이에 따라 장애물 탐지 화면을 가로(랜드스케이프) 모드로 전환하여 카메라 시야각을 확보하고 좌·우 방향 장애물 탐지 성능을 향상시켰다. 단순한 UI 변경이 아닌 탐지 정확도와 보행 안전성 향상을 위한 기능적 개선이다.

[2 차 변경] 홈 화면 기능 선택형 구조 도입 및 통합 모드 신설

초기 홈 화면은 '장애물 탐지'와 '스마트 길찾기' 두 가지 기능만 제공하였다. 성북시각장애인복지관 외부 사용자 테스트에서 "장애물 탐지와 길찾기를 동시에 제공하면 실제 보행 시 인지 부담이 크고, 지팡이·앱·길찾기를 동시에 사용하는 것은 현실적으로 어렵다"는 핵심 피드백이 수집되었다. 이를 반영하여 사용자가 상황에 따라 원하는 기능만 선택하여 단독 실행할 수 있는 기능 선택형 구조로 홈 화면을 개편하였다. 최종 홈 화면은 다음 4개 버튼으로 구성된다:

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

- 장애물 탐지 — 카메라 기반 실시간 장애물 탐지 단독 실행
- 길찾기 — 경로 탐색 및 구간별 TTS 안내 단독 실행
- 통합 모드 — 장애물 탐지 + 길찾기를 동시에 제공 (기존 방식 유지)
- 설정 — 사용자 맞춤 개인화 설정

[3 차 변경] 설정 화면 슬라이더 → 세그먼트 및 스텝퍼 기반 단계 선택 버튼 UI로 개선

2 차 개선과 동일하게 성북시각장애인복지관 사용자 테스트에서 “슬라이더 기반 조작은 시각장애인이 원하는 값을 정확하게 입력하기 어렵다”는 피드백이 확인되었다. 드래그 방식의 슬라이더는 화면을 직접 확인하지 않는 사용자가 정밀하게 제어하기 어렵고, TalkBack 환경에서도 현재 값과 조작 범위를 직관적으로 인지하기 어려운 한계가 있었다. 이에 따라 안내 문장 길이, 안내 재생 속도, 진동 강도 설정 방식을 단계 선택 기반 UI로 개선하였다. 안내 문장 길이는 ‘간결 · 보통 · 상세’ 중 하나를 직접 선택할 수 있는 세그먼트(Segmented Control) 방식으로 변경하였고, 재생 속도와 진동 강도는 - / + 버튼을 통해 단계 값을 조절하는 스텝퍼(Stepper) 방식으로 전환하였다. 특히 연속적인 드래그 입력 대신 단계 단위의 명확한 선택 구조를 적용하여, 비시각 환경에서도 현재 설정 값을 쉽게 인지하고 안정적으로 설정을 변경할 수 있도록 개선하였다. 이를 통해 시각장애인의 접근성과 조작 정확도를 함께 향상시켰다.



 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

2.2.2 시스템 기능 요구사항

기능	구현 내용 및 세부 사항	상태
장애물 탐지 기능	- YOLOv11s 기반 실시간 장애물 탐지 - 탐지 클래스: 사람·자전거·킥보드·차량·오토바이·휠체어·신호등 등 - AIHub 인도 보행 영상 BB 데이터 약 10GB, 50 epoch 학습 500ms(0.5 초)마다 이미지 캡처 후 서버 전송 - 탐지 이벤트 발생 시 카드 UI 표시 및 TTS, 진동 햅틱 안내 (*TTS 안내 기준은 TTS 음성 안내 시스템-채널 우선순위 구조 참고, 진동 햅틱 안내 기준은 햅틱(진동)안내-햅틱(진동) 안내 우선순위 및 동작 로직 참고.) - 장애물 안내 카드 UI: 탐지 결과를 경보 레벨(high/medium/low)에 따라 카드 UI 차등 표시 (테두리 두께·색·경고 아이콘 분리, 자세한 기준은 접근성 중심 UI 참고.) 8 초 이상 미수신 장애물 카드 자동 제거로 현재 탐지 중인 객체만 표시	완료
보행 가능 영역 분할	YOLOv26-seg 기반 보도/점자블록/보행가능영역/횡단보도 분할 [최초 계획] YOLOv11s + SegFormer 이중 모델 구조 설계 → YOLOv11s 기반 보행 중 장애물 탐지 및 SegFormer 기반 보도·점자블록·횡단보도·보행가능영역 분할 구조 계획 → Transformer 기반 SegFormer의 높은 연산량 및 이중 모델 사용에 따른 추론 속도·배포 복잡도 문제 확인 [1 차 변경] 1 차 멘토링(4/11) 피드백 반영, YOLO segmentation 단일 모델 구조로 전환 → SegFormer 제외 후 YOLO 계열 segmentation 모델 기반 보행 환경 분할 구조로 변경 → 파이프라인 단순화를 통한 실시간성 및 배포 가능성 확보 → 점자블록·횡단보도·보행가능영역·주의노면 등 실제 보행 안내 핵심 클래스 중심의 라벨 재구성 [2 차 변경] 2 차 멘토링(5/1) 피드백 반영, YOLO26-seg 기반 최종 구조 확정 → 단일 모델의 영역 분할 정밀도 한계 보완을 위한 YOLOv11s와 YOLO26-seg 비교 → 보행 가능 영역 및 노면 클래스 분할 성능, 추론 속도, 서비스 적용 가능성 종합 검토 → YOLO26-seg 기반 세그멘테이션 구조 최종 적용 → YOLO 계열 모델 중심의 구조 통일 및	완료 (변경)

	국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
		프로젝트 명	길벗	
		팀 명	리트리버	
		Confidential Restricted	Version 1.0	2026-MAY-14

	보도·점자블록·횡단보도·보행가능영역 분할 기능 구현	
거리 추정	• Depth Anything V2 Metric Outdoor 모델 기반 depth map 생성 • YOLOv11s 로 탐지된 객체 bbox 하단 영역의 depth 값 추출 • scale factor 와 offset 을 적용한 선형 보정으로 실제 거리 오차 감소 • 3m 이내 긴급, 5.5m 이내 경고, 8m 이내 안내 가능 거리로 구분 • 가까운 위험 객체를 우선 선별하여 불필요한 원거리 음성 안내 감소	완료
장애물 회피 방향 안내	• bbox 중심 좌표 기반 시계 방향 변환 (11 시·12 시·1 시 방향) • bbox 점유율로 가까움/중간/멀리 거리 추정 • 안내 예: "12 시 방향에 사람이 가까이 있습니다. 오른쪽으로 이동하세요." • 거리 레벨별 카드 시각화: 가까움(solid 3px 강조 테두리), 중간(solid 2px), 멀리(dashed 테두리) • 경보 레벨에 따라 진동 패턴 분리 적용 (high: 350ms×3 회 최대 강도 / medium: 200ms×2 회 / low: 100ms×1 회)	완료
길찾기 기능	• SK OpenAPI Tmap 보행자 전용 경로 탐색 엔드포인트 활용 • GeoJSON 파싱 → 이동거리·예상시간·회전 지점·구간별 안내 메시지 추출 • 음성 입력(STT)으로 목적지 주소 입력 가능 • 위치 권한 허용 여부 확인 후 안내 기능 동작 • 위치 초기화 진행: • 중복 실행을 막고 최초 1 회만 실행 • 앱이 포그라운드로 복귀할 때 _currentPosition == null 이면 재시도 진행 • 스텝 카드 UI: 방향 아이콘·이동거리·안내 문장 표시, 횡단보도 스텝 시 음성신호기 안내 박스 포함 • 출발 시 경로 개요 카드(총 거리·예상 시간·출발 방위) 5 초간 표시 후 스텝 전환 • 연속 3 회 이상 20m 이탈 감지 시 자동 재탐색, 재탐색 중 배너 표시, 재탐색 시 현재 좌표와 기존 목적지 좌표로 새 경로 요청 • 목적지 15m 이내 진입 시 도착 판정 및 TTS 안내 후 경로 안내 종료 • TTS, 진동 햅틱 안내: 카드 UI 표시와 동시 진행 (*TTS 안내 기준은 TTS 음성 안내 시스템·채널 우선순위 구조 참고, 진동 햅틱 안내 기준은 햅틱(진동)안내-햅틱(진동) 안내 우선순위 및 동작 로직 참고.)	완료 (변경)
통합 안내 기능 (길찾기+장애물 탐지)	• 장애물 탐지 전용 모드와 동일한 연결 로직으로 동작 (카메라 스트리밍, STOMP WebSocket 구동) • 경로 안내와 실시간 장애물 감지 병행 실행 • IndexedStack 으로 탭 전환 시 각 화면 상태 유지; 길찾기(index 2)·통합(index 3)은 NavigationScreen 을 각각 별도	완료

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

	인스턴스로 생성하고 GlobalKey 로 참조 • 두 NavigationScreen 인스턴스가 동시에 Geolocator 위치 스트림을 구독하면 충돌 발생 → 길찾기 시작 전 onBeforeNavigationStart 콜백으로 상대 탭의 cancelPositionStream()을 먼저 호출하여 스트림 충돌 방지 • 스텝 카드 UI: 방향 아이콘·이동거리·안내 문장 표시, 횡단보도 스텝 시 음성신호기 안내 박스 포함  • 출발 시 경로 개요 카드(총 거리·예상 시간·출발 방위) 5 초간 표시 후 스텝 전환 • 연속 3 회 이상 20m 이탈 감지 시 자동 재탐색, 재탐색 중 배너 표시, 재탐색 시 현재 좌표와 기존 목적지 좌표로 새 경로 요청 • 목적지 15m 이내 진입 시 도착 판정 및 TTS 안내 후 경로 안내 종료 • 장애물 안내 카드 UI: 탐지 결과를 경보 레벨(high/medium/low)에 따라 카드 UI 차등 표시 (테두리 두께·색·경고 아이콘 분리, 자세한 기준은 접근성 중심 UI 참고.) • 8 초 이상 미수신 장애물 카드 자동 제거로 현재 탐지 중인 객체만 표시 • 탐지 이벤트 발생 및 길찾기 스텝 변환 시 카드 UI 표시 및 TTS, 진동 햅틱 안내 (*TTS 안내 기준은 TTS 음성 안내 시스템·채널 우선순위 구조 참고, 진동 햅틱 안내 기준은 햅틱(진동)안내·햅틱(진동) 안내 우선순위 및 동작 로직 참고.)	
날씨 안내 기능 (TTS)	• 앱 시작(로그인/재시작) 후 홈화면 진입 시 위치 권한 확인 및 TTS 기능 on/off 값 확인 • 앱 시작(로그인/재시작) 후 홈화면 진입 시 현재 위치 GPS 좌표로 /api/weather?lat=&lon= 호출 • 응답의 description, temperature, windSpeed 필드를 바탕으로 문장 완성 후 TTS 로 출력 • 응답 예시: • "오늘 날씨는 맑음, 기온은 24 도입니다." • "오늘 날씨는 가벼운 비, 기온은 18 도입니다. 우산을 챙기세요." • "오늘 날씨는 눈, 기온은 영하 3 도입니다. 눈길에 미끄러움을"	완료

	국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
		프로젝트 명	길벗	
		팀 명	리트리버	
		Confidential Restricted	Version 1.0	2026-MAY-14

	주의하세요. 따뜻하게 입고 외출하세요." • "오늘 날씨는 맑음, 기온은 31 도입니다. 바람이 강하게 불고 있습니다. 더위에 대비해 수분을 충분히 섭취하세요."	
TTS 음성 안내 시스템	• flutter_tts 기반 4 가지 상황 음성 안내 (장애물·방향·길찾기·신호등) • 설정 변경 시 TTS 배속에 따른 배속 변경 안내 • 홈화면 진입 시 TTS 안내 설정 값 및 위치 권한 확인 후 날씨, 온도, 바람 강도에 따른 안내 (*자세한 내용은 날씨 안내 기능(TTS) 참고.) • 인앱 전용 TTS 안내 제외 동작 및 스크린 리더 TTS 안내는 Semantics를 활용하여 OS 스크린 리더에 권한 위임 - 인앱 TTS와 겹침 방지 • 안내 문장 1~2 문장 이내, 동일 상황 반복 제한 적용 • TTS 기본값: 속도 - 2.0 (2 배속), 범위 - 0.5 배속~7.0 배속, 변경 단위 - 0.25 (0.25 배속) • 채널 우선순위 구조: ① 장애물 탐지 모드 • 장애물 high alert: 모든 채널 끊고 즉시 출력 (interrupt: true) • 장애물 medium / low alert: 출력 중이면 스킵 • guideText 없음: TTS 출력 없음 (이벤트 수신 자체는 처리) • 탐지 중지 (stop 버튼): TTS 전체 즉시 중단 및 대기 큐 초기화 ② 길찾기 모드 • 길안내 시작 (개요 안내): 즉시 출력 (interrupt: true) • 개요 안내 직후 step[0] 서버 안내문: 개요 음성 완료 후 자동 재생 (큐 삽입) • step 전환: 즉시 출력 (interrupt: true) • 직진 step 거리 임계값 도달: TTS 없음 (직진은 거리 안내 생략) • 비직진 step 40m 이하 도달: 출력 중이면 큐 대기 → 완료 후 출력 • 비직진 step 20m 이하 도달: 출력 중이면 큐 대기 → 완료 후 출력 (40m 이미 출력 시 스킵) • 비직진 step 8m 이하 도달: 즉시 출력 (interrupt: true), 이후 20m~40m 임계값 자동 스킵 • 목적지 도착: 즉시 출력 (interrupt: true) ③ 통합 모드 • 길안내 step 전환 / 8m 이하: 탐지 음성 포함 모든 채널 끊고 즉시 출력 (interrupt: true) • 길안내 20m / 40m 안내: 탐지 음성 출력 중이면 큐 대기 → 완료 후 출력 • 장애물 high alert: 길안내 음성 출력 중이어도 끊고 즉시 출력 • 장애물 medium / low: 출력 중이면 스킵 • 목적지 도착: 탐지 음성 끊고 즉시 출력	완료
효과음 시스템	• audioplayers 라이브러리, 3 종 효과음: button_click.wav, action_start.wav, action_stop.wav	완료

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

	• 플레이어 2 개 분리 (_tapPlayer / _actionPlayer) → buttonTap 이 actionStart/Stop 과 동시 재생 가능 • iOS: AVAudioSessionOptions.mixWithOthers → TTS 재생 중에도 효과음 출력 • Android: audioFocus - gainTransientMayDuck (actionStart/Stop) • soundEffectEnabled 설정값으로 전체 ON/OFF 제어	
햅틱(진동) 안내	• vibration 라이브러리 기반 3 가지 진동 패턴 • 설정 변경 시 진동 강도에 따른 진동 변경 피드백 • 장애물 감지: 짧은 경고 / 긴급 위험: 연속 • 사용자 설정 진동 강도(0~100%)에 따라 모든 패턴 강도 비율 조정 • 햅틱(진동) 안내 우선순위 및 동작 로직: ▶ 우선순위 등급 (숫자 높을수록 우선) 0 — buttonTap: 버튼 터치 확인 1 — actionStart: 탐지·길안내 시작 1 — actionStop: 탐지·길안내 중지 2 — obstacleLevel1 장애물: low alert 3 — obstacleLevel2 장애물: medium alert 4 — obstacleLevel3 장애물: high alert ▶ 우선순위 규칙 • 높은 우선순위 진동 진행 중 → 낮은 우선순위 요청은 스킵 • 새 요청이 동등·상위 우선순위 → 기존 진동 즉시 중단 후 새 패턴 실행 • 동일 효과 쿨다운 이내 재호출 → 스킵 ▶ 쿨다운 (동일 효과 최소 재실행 간격) • buttonTap: 100ms • actionStart: 400ms • actionStop: 400ms • obstacleLevel1: 700ms (저위험 — 스팸 방지 강화) • obstacleLevel2: 600ms • obstacleLevel3: 450ms (긴급 — 더 자주 허용) ▶ 발생 시점별 효과 • 버튼 터치 (전 화면 공통) → buttonTap • 탐지·길안내 시작 → actionStart • 탐지·길안내 중지 → actionStop • 장애물 이벤트 alertLevel == 'high' → obstacleLevel3 • 장애물 이벤트 alertLevel == 'medium' → obstacleLevel2 • 장애물 이벤트 alertLevel == 'low' → obstacleLevel1 ▶ 전역 조건 • 진동 강도 설정 0% → 모든 진동 비활성화 (강도 값 자체를 확인 후 즉시 반환) • iOS → 패턴(duration)만 적용	완료

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

	Android (진폭 제어 지원 기기) → 사용자 설정 강도(0~100%)를 0~255 범위로 선형 변환 후 적용	
권한 여부 설정 및 확인	- 앱 최초 권한 온보딩 시트 1 회 자동 표시 (SharedPreferences permission_onboarding_shown 키) - 필수 권한: 카메라 (장애물 탐지/길찾기), 위치 (길찾기) - 선택 권한: 마이크 (음성 목적지 입력), 알림 (앱 공지 및 중요 알림) - 거부해도 다음 권한으로 진행; 최종 결과는 각 기능 시작 버튼의 requestForFeature()가 처리 - iOS에서는 권한 요청 시점과 방식이 Android와 달라 permission_handler 플러그인의 iOS 전용 처리 분기를 적용 - 기능에 필수적인 권한 영구 거부 상태에서 기능 시작 시도 시 권한 필요 안내 및 시스템 설정 화면으로 이동하는 "설정 열기" 버튼을 AlertDialog로 제공	완료
접근성 중심 UI	- WCAG 2.1 준수: 명도 대비 4.5:1 이상, 최소 18pt 글자 크기 - 큰 버튼 중심, 아이콘+텍스트 병행, Android TalkBack 지원 - 장애물 탐지 화면: 세로→가로(랜즈스케이프) 변경 (카메라 시야각 확보) - 홈 화면: 장애물 탐지·스마트 길찾기·통합 모드·설정 4개 버튼 구성 [계획] ① 세로(포트레이트) 모드 기반 UI 설계 → 카메라 시야각이 좁아 좌/우 접근 장애물 탐지 성능 저하 문제 ② 기존 홈 화면 : 장애물 탐지, 스마트 길찾기(장애물 탐지 + 길찾기), 설정 3개의 기능 버튼 → 성북시각장애인복지관 테스트 결과 장애물 탐지+길찾기 기능 동시 제공시 초기 사용에 인지 부담을 느낄 수 있다는 피드백 [변경] ① 장애물 탐지 화면을 가로(랜즈스케이프) 모드로 전환 - AI 모델이 양측 객체를 동시에 인식 가능한 구조 확보 - 단순 UI 변경이 아닌 탐지 정확도·안전성 향상을 위한 기능적 개선 ② 장애물 탐지만 / 길찾기만 / 장애물 탐지 + 길찾기 (통합 모드) 분리 - 홈화면: 장애물 탐지·스마트 길찾기·통합 모드·설정 4개 버튼 구성 - 사용자가 상황에 따라 원하는 기능만 단독 실행 가능	완료 (변경)
서버 기반 AI 추론	- 앱이 0.5 초마다 JPEG 압축 프레임 이미지를 HTTP multipart/form-data로 Spring Boot 백엔드에 전송 - Spring Boot 백엔드는 인증된 userId와 captured_at을 함께 FastAPI AI 서버로 multipart/form-data 방식으로 중계 - FastAPI AI 서버는 프레임을 기반으로 장애물 탐지 수행 - AI 서버는 추론 결과를 Spring Boot의 이벤트 수신 API로 JSON 형태로 콜백	완료

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

	• Spring Boot 는 user_id 기준으로 해당 사용자에게만 WebSocket(STOMP) 메시지를 전송 • 앱은 DetectionWsService 에서 STOMP 연결, 구독, 재연결, 해제를 관리하고 탐지 결과 및 연결 상태를 탐지 화면 UI 에 실시간 반영	
카카오 소셜 로그인	• 카카오 OAuth2.0 + JWT 기반 사용자 인증 • Refresh Token 기반 자동 재발급 구조 • 카카오톡 앱 설치 여부에 따라 앱 로그인/웹 로그인 자동 분기 • 발급된 액세스 토큰·리프레시 토큰을 flutter_secure_storage(iOS Keychain / Android Keystore)에 암호화 저장 • API 호출 시 401 응답 수신하면 토큰 재발급 후 자동 재시도, 재발급 실패 시 강제 로그아웃 처리	완료
사용자 설정 기능	• 안내 문장 길이 및 빠르기·진동 강도 개인화 • 개인화 내용 및 TTS 안내 여부 Amazon RDS user_settings 테이블에 저장 • 알림/효과음 여부 SharedPreferences 기반의 로컬 저장 관리 • 안내 문장 길이(간결/보통/상세) 세그먼트 버튼, 안내 빠르기(0.5~7.0 배속, 0.25 단위 스텝퍼), 진동 강도(0~100%, 10% 단위 스텝퍼)로 구성 • 안내 문장 길이, 안내 문장 빠르기, 진동 강도, 음성 안내 여부 설정 변경 즉시 서버에 PATCH 요청으로 동기화 • 푸시 알림·효과음 설정값은 기기 로컬(SharedPreferences) 저장 및 시스템 권한 상태 실시간 반영	완료
즐거찾기·최근 검색	• 즐거찾기 목록 조회/추가/삭제, 최근 검색 조회/삭제 • Kakao Local API 연동, favorite_places·recent_search_places 테이블 관리 • 장소 검색 결과 무한 스크롤(80px 여유 도달 시 다음 페이지 자동 로드) • 즐거찾기 편집 모드 토글로 삭제 버튼 노출, 최근 검색 전체 삭제 지원 • 길찾기 화면 내 저장된 장소 최대 2 개·최근 장소 최대 4 개 바로 표시	완료

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

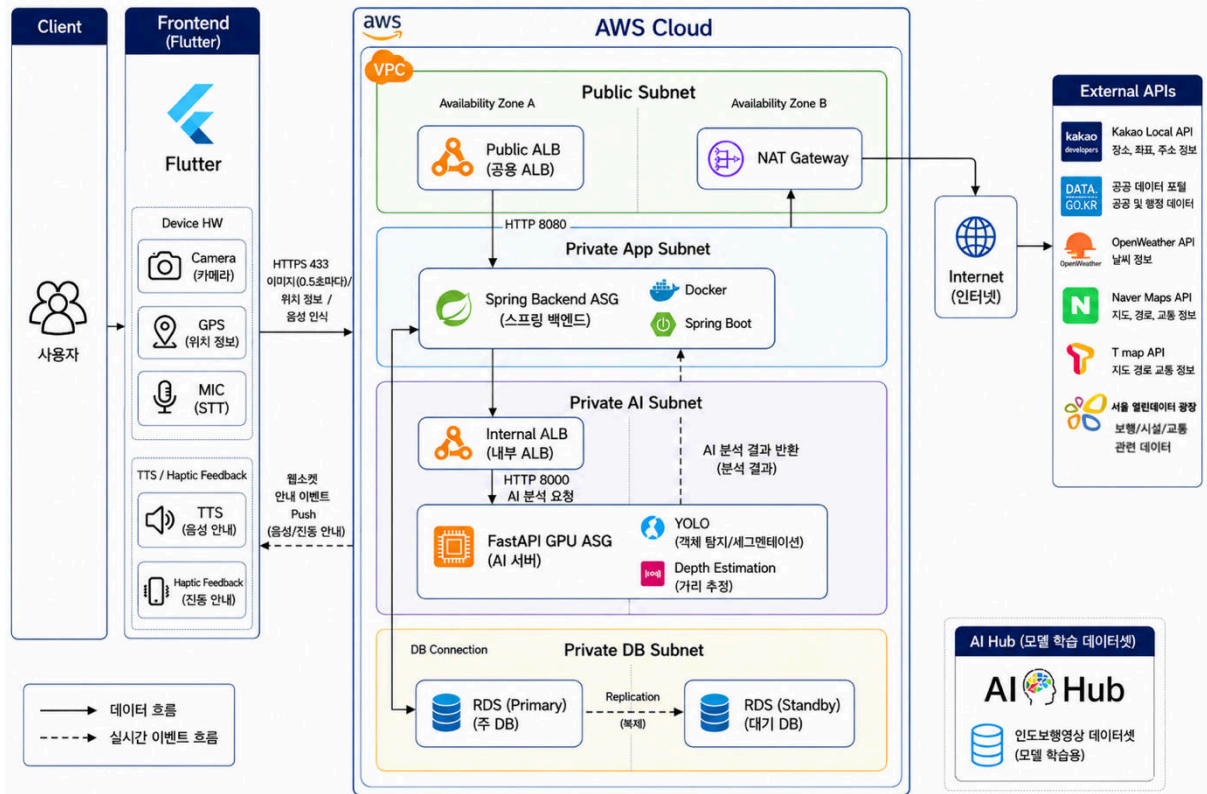
2.2.3 시스템 비기능(품질) 요구사항

요구사항	우선 순위	세부 내용 및 달성 근거	달성 여부
안정성	1	• 신뢰도 낮은 경우 "주의하세요."·"잠시 멈춰주세요" 경고 중심 안내 제공 • 위험 상황 시 음성 + 진동 동시 제공 • 흰지팡이 등 기존 보조 수단과 병행 사용 전제로 설계 • 네트워크 불안정 시 비정상 종료 없이 기본 안내 제공	달성
정확성	2	• 주요 보행 위험 객체 80% 이상 탐지 정확도 (YOLOv11s F1=0.7167 달성) • 보행 가능 영역 분할 IoU 0.7 이상 (YOLO seg mask mAP50=0.768 달성) • 진행 방향 전방 위험 객체 오탐보다 미탐 최소화 우선 설계	달성
실시간 성능	3	• 카메라 이미지 입력 후 0.4 초 이내 탐지 결과 제공 • YOLOv11s Avg Inference 13.07ms, FastAPI 추론 1.2ms 확인 • 회피 안내 문장 탐지 시점 기준 0.7 초 이내 음성 출력	달성
보안 및 개인 정보	4	• 카메라 영상·위치 정보 HTTPS 암호화 통신 전송 • 영상 이미지 미저장 구조 (추론 결과만 전달) • 위치 정보 서비스 제공 목적 외 미저장, 사전 동의 기반 처리	달성
사용성	5	• 안내 문장 1~2 문장 이내 짧고 명확한 표현 제공 • 동일 위험 상황 일정 시간 내 반복 안내 제한 적용	달성

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

2.2.4 시스템 구조 및 설계도

전체 시스템 아키텍처



최종 시스템 아키텍처는 계획서 구조를 기반으로 하되, 두 차례의 멘토링 피드백을 거쳐 AI 모델 구조가 최종 확정되었다.

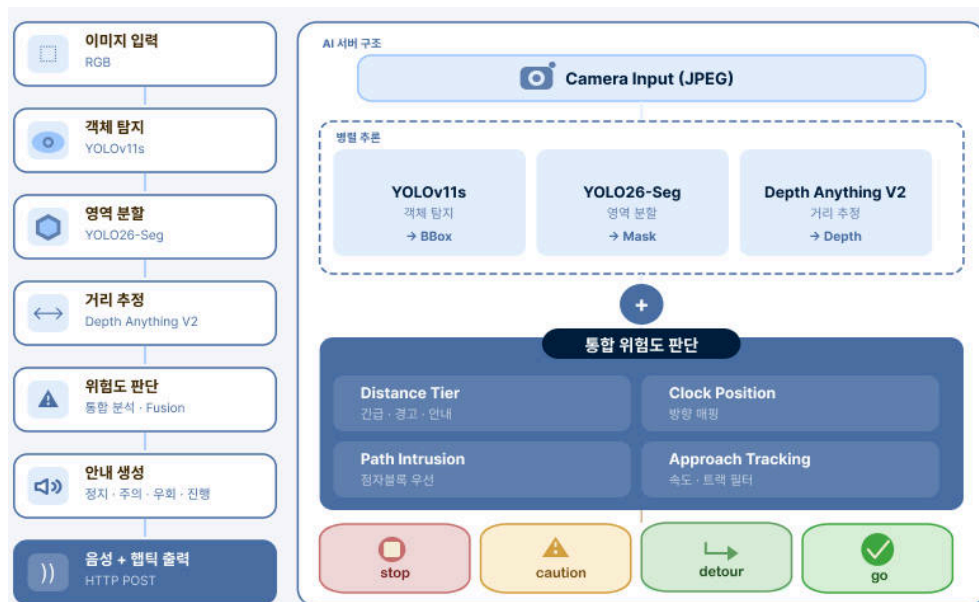
프론트 아키텍처 변경 이력

구분	최초 계획	1 차 멘토링 후	2 차 멘토링 후 (최종)
프론트 통신 구조	카메라 프레임 캡처 → Spring 백엔드 → FastAPI AI 서버 전송 → 장애물 탐지 결과 → Spring 백엔드 → WebSocket 실시간 수신 (서버 기반 분산 추론 구조)	TFLite 변환 모델을 Flutter 앱 내에서 직접 실행하는 온디바이스 추론 구조로 전환 (클라이언트 단독 처리, 서버 의존 제거)	초기 계획의 서버 기반 분산 추론 구조로 복귀. FastAPI AI 서버 추론 결과를 Spring 백엔드 서버를 통해 STOMP over WebSocket 으로 실시간 전송하는 방식으로 최종 확정
변경 사유	-	FastAPI AI 서버와의 네트워크 연결 불안정 및 응답 지연으로 실시간 탐지 신뢰성 저하 → 서버	TFLite 변환 후 모바일 기기의 연산 자원 한계로 추론 속도 저하 확인. 온디바이스 단독

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

		의존도 제거 및 응답 지연 최소화를 위한 온디바이스 전환 검토	처리만으로는 실시간 장애물 안내 요구사항 충족 불가 판단 → 서버 기반 구조 복귀 및 WebSocket 실시간 채널로 안정성 확보
비고	계획서 기준 구조	1 차 멘토링 반영	2 차 멘토링 반영

AI 파이프라인



AI 모델 아키텍처 변경 이력

구분	최초 계획	1 차 멘토링 후	2 차 멘토링 후 (최종)
AI 모델 구조	YOLOv11s (객체 탐지) + SegFormer (영역 분할) 이중 모델	YOLO segmentation 단일 모델로 통합	YOLOv11s (객체 탐지) + YOLO26-seg (영역 분할) 이중 모델 (최종 확정)
변경 사유	-	SegFormer 연산량 과다, 이중 모델 배포 복잡도 문제	단일 모델의 보행영역 분할 정밀도 한계 확인. YOLO 계열로 통일하여 배포 일관성 유지
비고	계획서 기준 구조	1 차 멘토링 반영	2 차 멘토링 반영

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

구분	최초 계획	1 차 멘토링 후	2 차 멘토링 후 (최종)
AI 모델 구조	YOLOv11s (객체 탐지) + SegFormer (영역 분할) 이중 모델	Depth Estimation 기반 거리 추정 도입 실험	Depth Anything V2 Metric Outdoor 거리 추정 이중/융합 구조 최종 확정
변경 사유	-	bbox 크기 기반 거리 판단만으로는 실제 장애물 거리 추정에 한계 발생	-
비고	계획서 기준 구조	1 차 멘토링 반영	2 차 멘토링 반영

기타 변경 사항

장애물 탐지 화면: 세로(포트레이트) → 가로(랜즈스케이프) 모드로 변경 (카메라 시야각 확보, 좌/우 방향 장애물 탐지 성능 향상)

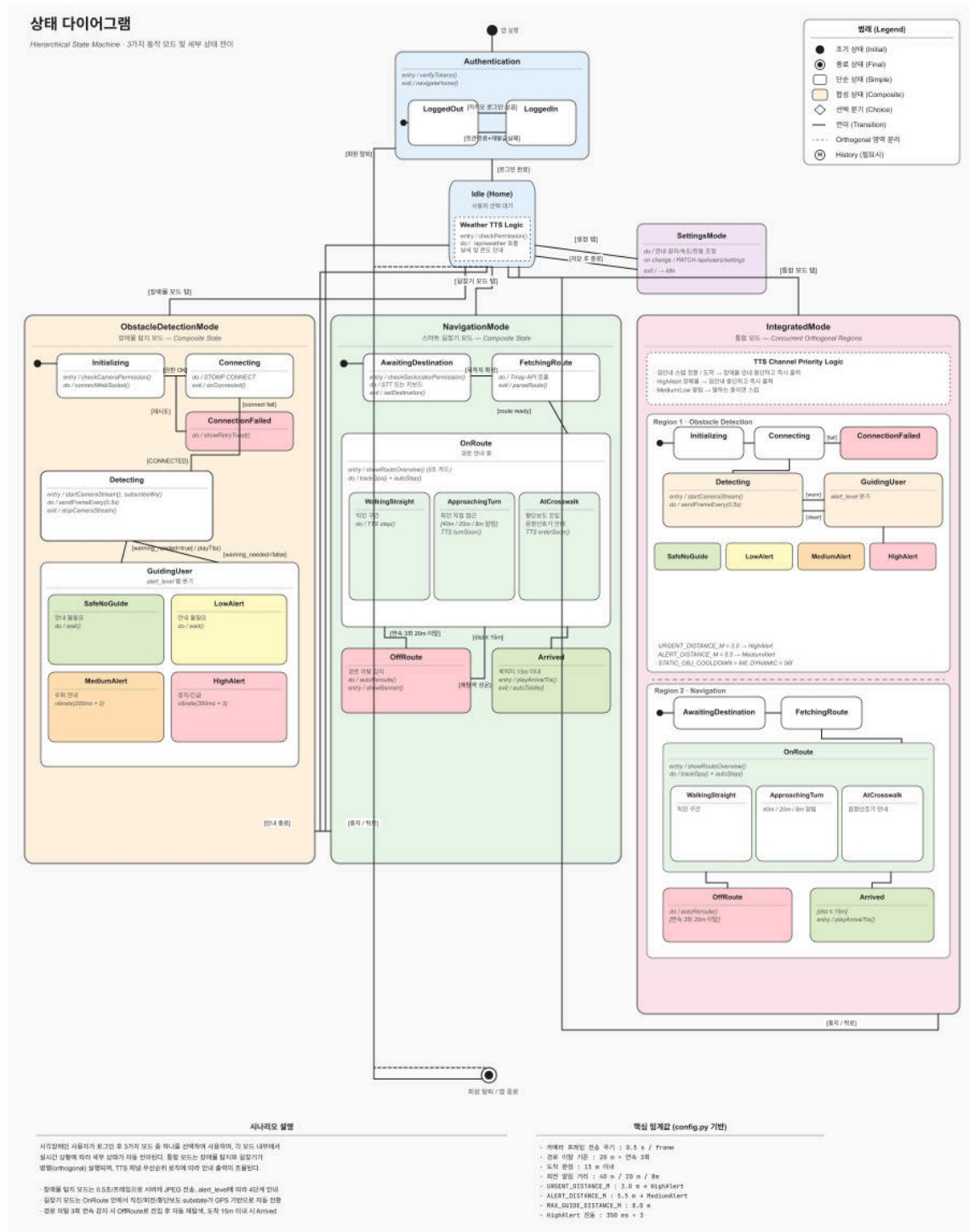
최종 아키텍처 구성요소

- 클라이언트: Flutter 앱 (Android APK/ABB, iOS IPA) — 카메라, GPS, 마이크
- 백엔드 서버
 - AWS VPC Private App Subnet
 - Spring Boot (Java 21) + Docker
 - Launch Template + Auto Scaling Group 기반 자동 배포
 - Public ALB 를 통해 HTTPS(443) 요청 처리
 - Amazon RDS(MySQL)와 Private Network 통신
- AI 서버
 - AWS VPC Private AI Subnet
 - FastAPI (Python 3.11) + Docker
 - YOLOv11s 기반 객체 탐지
 - YOLO26-seg 기반 보행영역 분할
 - Depth Anything V2 Metric Outdoor 기반 거리 추정
 - GPU 기반 EC2 Auto Scaling Group 구성
 - Internal ALB 를 통해 Spring Backend 서버에서만 접근 가능
 - NVIDIA GPU 기반 AI 추론 환경 구성

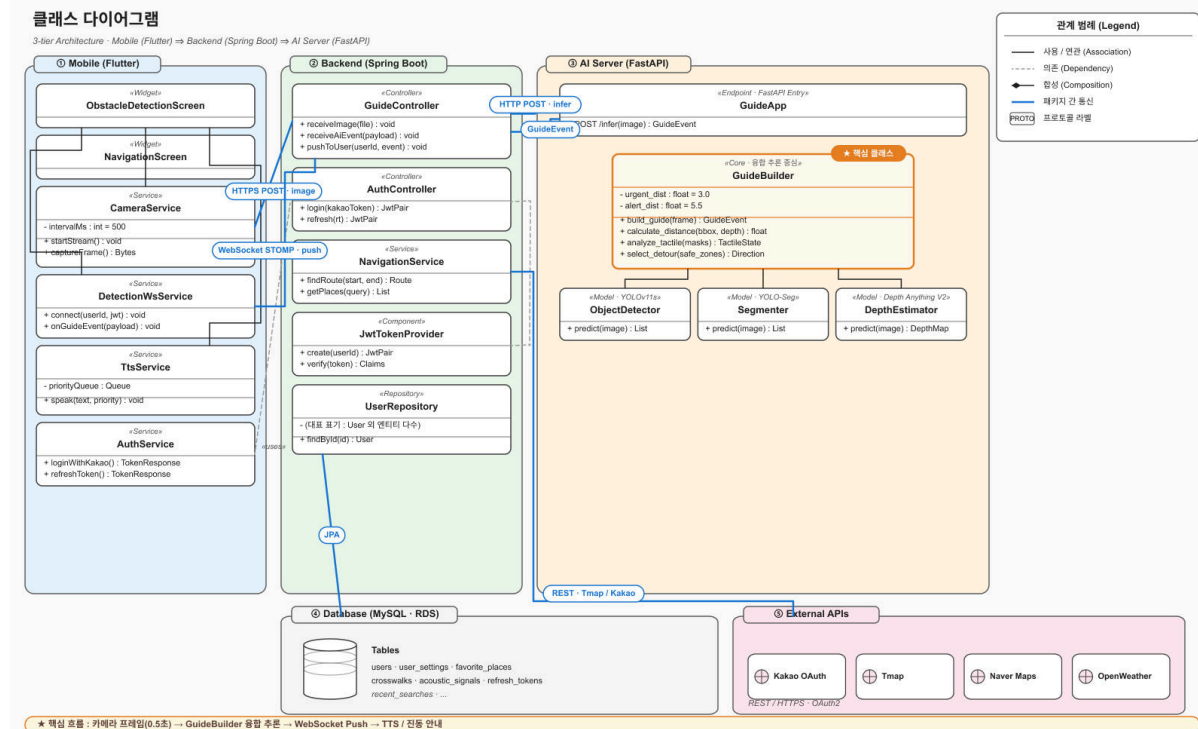
 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

- 외부 API: SK Tmap (경로 탐색), Kakao OAuth2.0/Local (인증·장소 검색), OpenWeatherMap (날씨), Naver Developers API (역지오코딩 보정), 공공 데이터포털 API 및 서울 열린 데이터 광장 (횡단보도 및 음향신호기 공공 데이터 조회)

상태 다이어그램



클래스 다이어그램



전체 동작 흐름

앱이 0.5 초마다 JPEG 압축 이미지를 백엔드(/api/guide/image)로 전송

→ 백엔드가 AI 서버(FastAPI)로 이미지 전달

→ AI 서버가 YOLOv11s로 장애물 탐지, YOLO26-seg 로 보행 가능 영역 분할

→ Depth Anything V2 Metric Outdoor 모델로 depth map 을 생성하고, 탐지된 객체 bbox 하단 영역의 depth 값을 활용하여 사용자와 장애물 간 거리를 추정

→ 추정 거리값에 선형 보정을 적용한 뒤, 3m 이내 긴급, 5.5m 이내 경고, 8m 이내 안내 가능 거리로 위험도를 구분

→ 객체 탐지 결과, 보행 가능 영역 분할 결과, 거리 추정 결과를 Rule-Based 알고리즘에서 통합하여 안내 문장 생성

→ JSON 결과를 백엔드(/api/guide/event)로 반환

→ 백엔드가 WebSocket(STOMP)을 통해 특정 사용자에게 실시간으로 탐지 결과를 Push.

데이터베이스는 Private Subnet(DB)의 Amazon RDS(MySQL)로 구성되며, 사용자 정보·즐거찾기·최근 검색·서비스 이용 기록을 저장한다.

주요 테이블: users, user_settings, refresh_tokens, favorite_places, recent_search_places.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

2.2.5 활용/개발된 기술

분류	기술/도구	활용 내용	채택 및 사용 이유
프론트엔드	Flutter / Dart	크로스플랫폼 모바일 UI 개발 (Flutter SDK 3.35.5, Dart 3.9.2 이상, Android SDK 36.1.0, iOS 13.0 이상)	Android 와 iOS 를 동시에 개발할 수 있는 크로스플랫폼 프레임워크로 개발 효율성을 높이고 유지보수를 단순화하기 위해 채택하였다. 실시간 카메라 스트림, 음성 안내, 지도 UI 등 다양한 기능을 하나의 코드베이스에서 통합 구현할 수 있다는 장점이 있었다.
	flutter_tts / speech_to_text	TTS 음성 안내, 음성 입력(STT)	시각장애인의 화면 접근 한계를 고려하여 음성 기반 인터페이스를 제공하기 위해 사용하였다. TTS 를 통해 경로 및 위험 정보를 음성으로 안내하고, STT 를 통해 목적지 입력 등 일부 기능을 음성 기반으로 사용할 수 있도록 구현하였다.
	vibration / stomp_dart_client	햅틱 피드백, WebSocket STOMP 실시간 통신	장애물 탐지와 같은 즉각적인 위험 상황을 시각 대신 촉각으로 전달하기 위해 햅틱 피드백을 적용하였다. 또한 AI 서버의 추론 결과를 실시간으로 전달하기 위해 WebSocket 기반 STOMP 통신을 사용하였다.
	kakao_flutter_sdk_user	카카오 소셜 로그인 연동	별도의 회원가입 절차 없이 간편 로그인을 지원하여 사용자 접근성을 높이기 위해 채택하였다. 국내 사용자 친화성이 높고 모바일 환경에서 활용성이 높다는 점을 고려하였다.
	geolocator / camera	GPS 위치 정보 수집 및 카메라 스트림 처리	GPS 기반 실시간 위치 정보를 수집하여 보행 경로 안내 기능에 활용하였다. 또한 카메라 스트림 데이터를 실시간으로 처리하여 AI

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

			장애물 탐지 및 보행 환경 분석 기능에 사용하였다.
	http / http_parser	백엔드 REST API 통신, 멀티파트 이미지 전송	카카오 로그인 토큰 발급, 경로 탐색, 장소 검색, 횡단보도 정보 조회 등 서버와의 모든 REST API 통신에 사용하였다. 또한 카메라 프레임 이미지를 AI 서버로 전송할 때 멀티파트 폼 데이터 형식으로 전송하기 위해 http_parser 를 함께 활용하였다. JWT 토큰 만료 시 자동 재발급 후 재시도하는 인증 로직도 이 라이브러리를 기반으로 구현하였다.
	flutter_secure_storage	JWT 액세스/리프레시 토큰 보안 저장	카카오 소셜 로그인을 통해 발급된 JWT 토큰을 안전하게 저장하기 위해 사용하였다. 일반 SharedPreferences 와 달리 Android Keystore 및 iOS Keychain 을 활용한 암호화 저장소를 제공하여 토큰 탈취 위험을 낮출 수 있다. 로그인 상태 유지 및 자동 로그인 기능 구현에 활용하였다.
	audioplayers / shared_preferences	UI 효과음 재생, 사용자 설정 및 앱 상태 로컬 저장	TTS 와 별개로 버튼 터치음, 기능 시작·종료음 등 효과음을 재생하기 위해 audioplayers 를 사용하였으며, TTS 와 동시 재생이 가능하도록 AudioContext 를 구성하였다. shared_preferences 는 효과음 활성화 여부, 알림 수신 여부 등 사용자 설정 및 권한 온보딩 표시 여부를 기기 로컬에 저장하는 데 활용하였다.
백엔드	Spring Boot (Java 21)	RESTful API 서버, 비즈니스 로직 (JDK 21, Gradle, IntelliJ IDEA)	안정적인 서버 구조와 높은 생산성을 제공하는 프레임워크로 RESTful API 및 비즈니스 로직 구현에 활용하였다. Java 21

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

			기반의 최신 기능을 적용하여 유지보수성과 확장성을 확보하였다.
	Spring Security + JWT	사용자 인증, Refresh Token 기반 자동 재발급	사용자 인증 및 인가를 안전하게 처리하기 위해 사용하였다. JWT 기반 인증 방식을 통해 Stateless 구조를 유지하고, Refresh Token 을 활용하여 로그인 세션을 안정적으로 유지할 수 있도록 구현하였다.
	AWS Application Load Balancer (ALB)	HTTPS(443) 처리, 트래픽 분산, Reverse Proxy	Public ALB 를 통해 외부 HTTPS 요청을 수신하고 Spring Backend 서버로 트래픽을 전달하기 위해 사용하였다. Health Check 기반으로 정상 서버에만 요청을 전달할 수 있으며, 향후 서버 확장 시 트래픽 분산 구조를 지원하기 위해 채택하였다. 또한 Internal ALB 를 통해 AI 서버를 Private Network 내부에서만 접근 가능하도록 구성하였다.
	AWS Auto Scaling Group (ASG)	백엔드 및 AI 서버 자동 생성·복구	Spring Backend 서버와 FastAPI GPU 서버를 Launch Template 기반 Auto Scaling Group 으로 구성하여 서버 장애 발생 시 자동 복구가 가능하도록 설계하였다. 또한 컨테이너 기반 서버 환경을 일관성 있게 자동 배포할 수 있도록 활용하였다.
	Amazon ECR (Elastic Container Registry)	Docker 이미지 저장 및 배포	Spring Boot 및 FastAPI 서버 이미지를 중앙 저장소 형태로 관리하기 위해 사용하였다. GitHub Actions 기반 CI/CD 환경에서 Docker 이미지를 빌드 후 ECR 에 업로드하고, EC2 서버가 이를 자동으로 pull 하여 배포하도록 구성하였다.
	AWS Systems Manager	환경 변수 및 API Key	DB 접속 정보, JWT Secret Key,

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

Parameter Store	관리	외부 API 인증 키 등 민감한 환경 변수를 안전하게 관리하기 위해 사용하였다. EC2 UserData 에서 Parameter Store 값을 읽어 .env 파일을 자동 생성하도록 구성하여 운영 편의성과 보안성을 높였다.
AWS VPC / Private Subnet	네트워크 보안 및 내부망 분리	Backend 서버, AI 서버, RDS 를 모두 Private Subnet 에 배치하여 외부에서 직접 접근할 수 없도록 구성하였다. Public ALB 를 외부 진입 지점으로 사용하고, Internal ALB 를 통해 내부 AI 서버와 통신하도록 설계하여 보안성과 네트워크 분리 구조를 강화하였다.
WebSocket (STOMP)	AI 분석 결과 실시간 앱 전달	AI 추론 결과를 앱에 실시간으로 전달하기 위해 적용하였다. 일반 HTTP polling 방식보다 지연이 적고 실시간성이 뛰어나 사용자 보행 중 즉각적인 피드백 제공에 적합하였다.
Spring Data JPA + MySQL	Amazon RDS DB 접근 ORM	객체 중심의 데이터 처리를 통해 개발 생산성을 높이기 위해 ORM 기반 JPA 를 사용하였다. 사용자 정보, 경로 기록, 설정 데이터 등을 MySQL 기반으로 안정적으로 관리하였다.
Nginx + Docker + AWS EC2	리버스 프록시, 컨테이너 배포 (Ubuntu 22.04 LTS)	서비스 배포 및 운영 환경을 구축하기 위해 사용하였다. Docker 를 통해 서버와 AI 환경을 컨테이너화하여 배포 일관성을 확보하였고, Nginx 를 리버스 프록시로 활용하여 트래픽 관리 및 보안성을 강화하였다.
Swagger (Springdoc OpenAPI)	API 명세서 자동 생성 및 Swagger UI 제공	프론트엔드·백엔드 협업 과정에서 API 명세를 효율적으로 공유하기 위해 사용하였다. API 테스트 및 문서화를 자동화하여 개발

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

			생산성과 협업 효율을 높였다.
AI	YOLOv11s (Ultralytics)	실시간 장애물 객체 탐지 (PyTorch, Google Colab/Local GPU/CUDA)	실시간 장애물 탐지를 위해 경량화와 추론 속도를 고려하여 채택하였다. 보행 환경에서 빠른 객체 인식이 중요하므로, 엣지 디바이스 환경에서도 비교적 안정적인 성능을 제공하는 모델을 활용하였다.
	YOLO26-Seg	보행 가능 영역·점자블록·횡단보도 분할	단순 객체 탐지를 넘어 보행 가능 영역, 점자블록, 횡단보도 등 영역 단위의 정보를 제공하기 위해 사용하였다. 이를 통해 사용자 주변 환경을 보다 직관적으로 분석하고 안전한 보행 경로 판단에 활용하였다.
	Depth Anything V2 Metric Outdoor	객체별 거리 추정 및 위험 거리 판단 (PyTorch, Hugging Face, CUDA)	객체 탐지만으로는 실제 거리 판단에 한계가 있어 사용하였다. Depth map 을 생성한 뒤 객체 bbox 하단 영역의 깊이값을 활용하여 장애물과 사용자 간 거리를 추정하였다. FastDepth, LiteMono 보다 객체 윤곽과 거리 차이가 더 선명하고 안정적으로 나타나 최종 모델로 채택하였다. 또한 선형 보정을 적용하여 실제 거리와의 오차를 줄이고, 가까운 위험 객체만 우선 안내하는 데 활용하였다.
	FastAPI (Python 3.10+)	AI 추론 서버 (AWS EC2 Private Subnet, Docker)	AI 모델 추론 서버를 경량·고속 환경으로 구축하기 위해 사용하였다. Python 기반 AI 모델과의 연동이 용이하고 비동기 처리 성능이 우수하여 실시간 추론 처리에 적합하였다.
	OpenCV / NumPy / Pandas	이미지 전처리 및 데이터 분석	이미지 전처리, 프레임 분석, 데이터 처리 과정에 활용하였다. OpenCV 를 통해 영상 데이터를 가공하고, NumPy 와 Pandas 를

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

			통해 추론 결과 및 데이터 분석 작업을 효율적으로 수행하였다.
외부 API	SK OpenAPI (Tmap)	보행자 전용 경로 탐색	보행자 전용 경로 탐색 기능을 제공하기 위해 사용하였다. 자동차 중심이 아닌 실제 보행 환경 기반 경로 안내가 가능하여 시각장애인 보행 서비스 목적에 적합하였다.
	Kakao OAuth2.0 / Local	소셜 로그인 및 장소 검색	소셜 로그인 기능과 장소 검색 기능 구현에 활용하였다. 국내 지도 및 장소 데이터 활용성이 높아 사용자 친화적인 위치 검색 경험을 제공할 수 있었다.
	OpenWeatherMap API	날씨 정보 조회	날씨 정보를 제공하여 우천·악천후 상황에서 사용자 보행 안전성을 높이기 위해 활용하였다. 실시간 날씨 데이터를 기반으로 보행 시 주의 정보를 함께 제공할 수 있도록 구현하였다.
	Naver Developers API	역지오코딩 보강	카카오 로컬 API의 주소 변환 한계를 보완하여 좌표 기반 도로명 주소 정확도를 높이기 위해 사용하였다.
	공공 데이터 포털	전국 횡단보도 데이터 조회	전국 단위의 횡단보도 정보를 확보하여 보행 경로 내 횡단보도 안내 기능에 활용하기 위해 사용하였다.
	서울 열린데이터 광장	서울 횡단보도 및 음향신호기 데이터 조회	서울 지역의 횡단보도와 음향신호기 정보를 수집하여 횡단보도 접근 및 음향신호기 존재 여부 안내에 활용하기 위해 사용하였다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

2.2.6 현실적 제한 요소 및 그 해결 방안

분류	제한 요소	해결 방안 및 결과
하드웨어	모바일 기기 성능에 따른 온디바이스 AI 추론 속도 저하	AI 추론을 백엔드 서버에서 처리하고, 앱은 카메라 전송·결과 표시만 담당하는 클라이언트-서버 구조로 구성하여 해결
	스마트폰 카메라 성능 및 촬영 환경(조명·날씨)에 따른 탐지 정확도 저하	다양한 환경 테스트, 이미지 전처리·모델 튜닝으로 성능 개선. 야간·역광 환경은 향후 보완 과제
	GPS 위치 정확도 저하로 경로 안내 오차 발생	SK OpenAPI 맵매칭 기술과 GPS 데이터 결합, 경로 전환 임계값 넉넉하게 설정
소프트웨어	스마트폰-서버 간 네트워크 지연으로 실시간성 저하 (LTE 환경 왕복 지연)	JPEG 압축 후 전송 전처리 최적화, 중요 프레임만 선택적 전송하는 스킵 프레임 전략 적용
	Flutter-AI 간 API 연동 비동기 처리 및 에러 핸들링 복잡도 증가	Swagger UI 기반 API 명세서 작성하여 프론트·백엔드 병행 개발, 공통 에러 코드 정의
	STOMP 구조적 한계 및 감지 불가, 유령 장애물 잔류	- Stale Sweep 메커니즘 도입: Map 구조로 수신 시각을 기록하고, 1 초 주기로 탐색하여 8 초간 갱신 없는 데이터 자동 제거 - 실시간성 및 신뢰도 확보: 유령 장애물을 차단해 화면 실시간성을 높이고, 서버 수정 없이 클라이언트 로직만으로 데이터 무결성 보장
	통합·길찾기 모드가 동일 페이지에서 GPS 스트림을 공유 → 탭 전환 시 이전 스트림이 정리되지 않아 TTS·거리 계산 동작 X	→ 탭 전환 시 이전 스트림이 정리되지 않아 TTS·거리 계산 동작 X하면 이동 전 기존 GPS 연결을 완전히 끊고 새 화면으로 전환. GPS 상시 스트림 → 필요할 때만 위치를 받아오는 방식으로 변경
	POINT 좌표 기준 직선 거리 판정 → 정상 보행 중에도 이탈 오판. 지나온 구간 재검사 및 목적지 근처 GPS 흔들림으로 불필요한 재탐색 반복	POINT → LINE 선분 수직 거리 기준으로 전환. 경로 인덱스 단방향 전진으로 지나온 구간 검사 제외. 목적지 50m 이내 이탈 감지 비활성화
	단일 API 기반 주소 변환만으로는 현재 위치의 도로명 주소 정확도 저하	Kakao Local API를 기본으로 사용하되 Naver Maps API를 보조 API로 결합. 후보 주소는 다시 지오코딩해 원래 좌표와의 거리를 비교, 주소가 없을 경우 행정구

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

		역 및 근처 장소 검색으로 fallback 처리
	YOLO 가 화면 가장자리에 잘리거나(Edge) 다른 물체에 가려진(Occluded) 객체의 탐지 정확도가 낮아, 보행 환경에서 위험 객체 검출 누락 우려	- Edge/Occluded 케이스 자동 식별 후 학습 노출 빈도 2~3 배로 오버샘플링 - 2 단계 Curriculum 학습 적용: Stage 1 전체 데이터 학습(lr=1e-3) → Stage 2 Hard 케이스 미세조정(lr=3e-4)으로 약점 보완
	YOLOv11s + SegFormer 이중 구조의 시스템 복잡도 증가 (1 차 멘토링 피드백)	- 1 차 멘토링: YOLO seg 단일 모델로 전환하여 복잡도 및 연산량 문제 해소 - 2 차 멘토링: 단일 모델의 보행 영역 분할 정밀도 한계 확인 → YOLOv11s(탐지) + YOLO26-seg(영역 분할) 이중 구조로 최종 확정. SegFormer 대신 YOLO 계열로 통일하여 배포 일관성 확보
	Depth Estimation 기반 거리 추정 정확도 한계 및 실제 거리 오차 발생	Depth Estimation 미적용 시에는 bbox 크기만으로 거리를 판단해, 먼 객체를 가까운 위험 요소로 오인하거나 가까운 작은 장애물을 놓치는 문제발생. 이를 해결하기 위해 Depth Anything V2 Metric Outdoor 모델을 적용하고, bbox 하단 영역의 depth 값과 선형 보정을 활용해 실제 거리 추정 정확도를 개선. 그 결과 가까운 위험 객체를 더 정확히 선별할 수 있었고, 불필요한 원거리 음성 안내를 줄일 수 있었음.
인프라 / 배포	CUDA·PyTorch 기반 GPU Docker 이미지 용량 증가로 no space left on device 오류 발생	초기에는 루트 디스크(/) 용량 부족 문제로 판단했으나, 실제 원인은 Docker와 containerd의 저장 경로 분리 구조에 있었음. Docker Root Dir을 /opt/dlami/nvme/docker로 이동하고, containerd snapshotter 경로도 NVMe 영역으로 이동 및 symbolic link 적용하여 대용량 AI 이미지의 pull·extract가 정상 동작하도록 개선
	containerd snapshot overlay 가 루트 디스크 (/var/lib/containerd)를 지속적으로 사용하여 extract 실패 발생	Docker 저장 경로 변경만으로 해결되지 않아 containerd overlayfs snapshot 저장소까지 NVMe로 분리. CUDA/PyTorch 기반 5GB+ GPU 이미지를 안정적으로 배포할 수 있는 구조 확보
기타	실제 보행 환경의 다양한 장애물 완벽 인식 어려움	다양한 데이터셋 학습, 탐지 신뢰도 임계값 보수적 설정 + Fail-safe 경고 음성 정책 적용

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

	시각장애인 대상 UI 검증을 위한 실제 사용자 테스트 확보 어려움	WCAG 2.1 가이드라인 준수 설계, 팀원 안대 착용 시나리오 내부 테스트 수행. 성북시각장애인복지관, 서울시립노원시각장애인복지관 방문 테스트 수행
--	--	---

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

2.2.7 결과물 목록

대분류	소분류	기능/내용	형식	기술 문서
모바일 앱 - 주요 기능	홈 화면	- 장애물 탐지·스마트 길찾기·통합 모드·설정 큰 버튼 4개 - 접근성 메인 화면 (오렌지/남색 고대비)	Flutter	X
	장애물 탐지 화면	- 카메라 스트림 기반 실시간 탐지 결과 표시 (가로모드) - TTS·진동 출력, 실시간 감지 활성화 표시	Flutter	X
	길찾기 화면	- 목적지 음성 입력(STT), 구간별 방향 TTS 경로 안내, 남은 거리·경로 진행 표시 - 성북시각장애인복지관 피드백 반영 : 단독/통합 선택형 구조로 제공	Flutter	X
	통합 모드 화면	- 장애물 탐지 + 길찾기 동시 제공 화면 - 경로 안내 중 실시간 장애물 탐지 병행 수행 - 성북복지관 테스트 피드백 반영: 단독/통합 선택형 구조로 제공	Flutter	X
	설정 화면	안내 문장 길이 및 빠르기, 진동 강도 개인화 설정, TTS 음성 안내 및 효과음 활성화 토글, 푸시 알림 수신 설정	Flutter	X
모바일 앱 - 기타 화면	스플래시 / 자동 로그인 화면	앱 시작 시 저장된 JWT 토큰 유효성 자동 확인, 유효 시 홈으로, 만료 시 재발급 후 이동, 실패 시 로그인 화면으로 분기	Flutter	X
	회원가입 / 로그인 화면	카카오 소셜 로그인, 사용 가이드 진입 버튼	Flutter	X
	장애물 탐지 진행 화면	- 탐지된 장애물 카드 목록 실시간 표시(물체명·거리·위험도), 8초 stale 타임아웃으로 오래된 탐지 자동 제거 - WS 연결 상태 표시, 탐지 수 카운터, 가로 화면 강제	Flutter	X
	길찾기 진행 화면	- 구간별 방향 TTS 경로 안내 - 총 거리, 예상 시간 표시 - GPS 실시간 추적, 경로 이탈 감지 및 재탐색, 도착 자동 감지, 횡단보도 음성신호기 정보 안내, 가로 화면 강제	Flutter	X

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

	통합 모드 진행 화면	- 길찾기 진행 화면에 장애물 탐지를 병행 수행 - 경로 안내 TTS와 장애물 탐지 TTS·진동 동시 출력 - 카메라 스트림 및 WebSocket STOMP 연결 유지	Flutter	X
	저장된 장소 관리 화 면	즐거찾기 목록 조회·추가·삭제, 별칭 및 카테고리 설정, 장소 주소 검색 연동	Flutter	X
	사용방법 화면	기본 조작·길찾기·위험 탐지·설정·음성 명령 5개 튜토리얼 제공, 시각장애인 대상 앱 사용 안내	Flutter	X
	계정 관리 화면	사용자 정보 확인, 로그아웃, 회원 탈퇴	Flutter	X
AI 모델	장애물 탐지모델	YOLOv11s 기반 보도 위 장애물 탐지	YOLO (Ultralytics)	O (5.9 참조)
	보행영역 분할모델	YOLO26-seg 기반 점자블록·보행가능영역·횡단보도 분할	YOLO seg	O (5.9 참조)
	거리 추정 모델	Depth Anything V2 Metric Outdoor 기반 객체별 거리 추정. 탐지된 객체의 bbox 하단 영역에서 depth 값을 추출하고, 선형 보정을 적용하여 사용자와 장애물 간 실제 거리를 계산	Depth Estimation	O (5.9 참조)
AI 서버	FastAPI 추론 서버	이미지 수신 → YOLOv11s 탐지 + YOLO26-seg 분할 → Rule-Based 안내 문장 생성 → JSON 반환	FastAPI (AWS EC2)	X
백엔드	Spring Boot 서버	이미지 수신·AI 서버 중계·WebSocket Push, Nginx 리버스 프록시	Spring Boot(AWS EC2)	O (5.7 참조)
	경로 탐색 API	SK OpenAPI 호출·GeoJSON 파싱·안내 텍스트 추출 REST API	Spring Boot	O (5.7 참조)
	데이터 베이스	Amazon RDS(MySQL): users·user_settings·refresh_tokens·favorite_places·recent_search_places·crosswalks·acoustic_signals·crosswalk_acoustic_signal_mappings	Amazon RDS	O (5.10 참조)
산출물	Android	최종 길벗 모바일 애플리케이션 (Android 9 이상)	Android	O

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

APK		APK	(5.1 참조)
API 명세서	프론트-백엔드 간 통신 규격 문서 (Swagger UI)	문서	O (5.7 참조)
테스트 보고서	시스템 통합 테스트 결과 및 성능 측정 보고서	문서	O (5.6 참조)

시연 영상: <https://youtu.be/Z8eKX4Aa4Dc>

2.3 기대효과 및 활용방안

1) 시각장애인 및 저시력자의 독립적인 이동 능력 향상

사용자가 스마트폰 카메라와 인공지능 기반 안내 기능을 활용하여 주변 보행 환경을 실시간으로 인식하고, 이를 음성 및 진동 형태로 전달받을 수 있도록 설계하였다. 타인의 지속적인 도움 없이도 스스로 목적지까지 이동할 수 있는 자율성과 이동의 자유를 확대하는 데 기여한다.

2) 보행 중 충돌 및 낙상 사고 예방

인공지능이 실시간으로 위험 요소를 탐지하고 즉각적으로 경고 및 방향 안내를 제공함으로써 충돌이나 낙상과 같은 보행 사고를 예방한다. 탐지 신뢰도 임계값을 보수적으로 설정하고 Fail-safe 경고 음성 정책을 적용하여 안전 중심 설계를 채택하였다.

3) 스마트폰 기반 높은 접근성 확보

별도의 고가 장비가 아닌 일상적인 스마트폰을 기반으로 구현되어 접근성과 활용성이 높다. 애플리케이션 형태로 제공되어 소프트웨어 업데이트와 기능 확장이 용이하다.

향후 확장 가능 분야

- 노약자 보행 보조 시스템: 장애물 회피·낙상 위험 예방·안전한 경로 안내를 고령자 대상으로 확장
- 공공 안전 서비스: 보행 위험 요소 수집·분석을 통한 지역 단위 안전 정보 제공 시스템
- 지자체 및 공공 서비스 연계: 교통약자 이동 지원 정책·스마트시티 기반 보행 안전 인프라와 결합

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

3 자기평가

한여진

길벗 프로젝트에서 나는 모바일 앱 프론트엔드 개발 전반을 담당하였다. 주요 담당 영역은 길찾기 기능 전체, STT 기능, 설정 화면 내 계정 및 개인정보처리방침 관련, 그리고 사용자 인터페이스를 고려한 UI 디자인이었다. 특히 사용자가 실제로 서비스를 이용하는 과정에서 불편함 없이 자연스럽게 사용할 수 있도록 화면 흐름과 인터랙션을 설계하는 데 집중하였다.

프로젝트에서는 길찾기 관련 핵심 화면과 기능을 직접 구현하였다. 저장된 장소 및 최근 장소 화면, 음성·텍스트 입력 위젯, 실시간 위치 안내 위젯, 길찾기 안내 시작 및 진행 화면, 위치 정보 공유 기능, Nav Bar, 설정 페이지 등 실제 사용자가 자주 접하는 주요 화면과 컴포넌트를 담당하였다. 또한 단순히 화면을 개별적으로 구현하는 데 그치지 않고, 여러 화면에서 공통적으로 사용되는 버튼, 입력창, 안내 위젯 등의 요소를 컴포넌트화하여 재사용 가능하도록 설계하였다. 이를 통해 UI 일관성을 유지하고 유지보수 효율성을 높일 수 있었으며, 이후 기능 추가나 수정 과정에서도 빠르게 대응할 수 있도록 구조를 구성하였다.

가장 많은 시간과 노력을 들인 부분은 실시간 길안내 기능이었다. 사용자가 실제로 이동하면서 현재 위치를 기반으로 다음 행동을 자연스럽게 안내받을 수 있도록 GPS 위치를 지속적으로 추적하고, 현재 단계를 자동으로 전환하는 로직을 직접 설계하고 구현하였다. 처음에는 GPS 값을 그대로 활용했지만, 실외 테스트를 진행하면서 실내 환경과 달리 위치 오차가 크게 발생하는 상황이 빈번하다는 것을 확인하였다. 이로 인해 엉뚱한 위치에서 단계가 전환되거나 경로 이탈이 잘못 감지되는 문제가 발생했고, 이를 해결하기 위해 일정 수준 이상의 오차를 가진 위치 정보는 무시하는 필터링 로직을 추가하였다. 또한 경로 이탈 감지 방식도 단순 반경 체크에서 벗어나, 경로 선분과 사용자의 실제 거리를 계산하는 방식으로 개선하였다. 목적지 근처에서는 작은 오차만으로도 재탐색이 반복되는 문제가 있었는데, 목적지 인근에서는 이탈 감지를 비활성화하는 예외 처리를 추가하여 해결하였다. 이처럼 실제로 걸어보지 않으면 발견하기 어려운 문제들이 많았고, 원인을 분석하고 수정하는 과정을 반복하며 기능의 완성도를 높여 나갔다.

음성 안내 시스템을 설계하는 과정도 많은 고민이 필요했다. 길안내 음성과 장애물 경고 음성이 동시에 재생되면 사용자가 내용을 제대로 인지하기 어렵다는 문제를 발견하였고, 이를 해결하기 위해 채널별 우선순위를 부여하여 중요한 안내가 덜 중요한 안내를 자연스럽게 대체하도록 구조를 설계하였다. 또한 남은 거리에 따라 안내 멘트를 다르게 분기하고, 이전 단계의 거리 정보가 잠시 다음 단계 화면에 표시되는 버그처럼 상태 관리와 타이밍 처리 과정에서 발생하는 세부적인 문제들도 하나씩 수정하였다. 어느 거리에서 어떤 안내를 제공해야 사용자가 가장 자연스럽게 행동할 수 있는지에 대한 정답은 없었기 때문에, 실제 테스트를 반복하며 기준을 지속적으로 조정하였다.

횡단보도 구간에서 음성신호기 정보를 안내에 반영하는 기능 역시 예상보다 복잡한 작업이었다. 처음에는 단순한 API 연동 정도로 생각했지만, 실제 구현 과정에서는 경로의 각 단계에 해당하는 횡단보도 정보를 미리 조회하여 캐싱해두고, 단계 전환 시점에 자연스럽게 안내로 연결되는 흐름을 설계해야 했다. 이 과정에서 서버 응답 구조가 예상과 달라 파싱 로직을 수정해야 했고, 기능 구현 이후에도 스텝 전환 시점과 TTS 재생 타이밍이 어긋나는 문제를 추가로 수정하였다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

STT 기능 구현 과정에서는 사용자의 음성을 안정적으로 인식하고 길찾기 흐름과 자연스럽게 연결하는 부분에 집중하였다. 단순히 음성을 텍스트로 변환하는 것을 넘어, 사용자의 발화가 실제 서비스 흐름에서 끊김 없이 이어질 수 있도록 상태 관리와 예외 상황 처리에 신경 썼다. 또한 실제 사용 환경에서는 주변 소음이나 발화 속도에 따라 인식 결과가 달라지는 경우가 많았기 때문에, 여러 상황에서 테스트를 반복하며 인식 흐름을 개선하였다.

설정 화면에서는 계정 관리와 개인정보처리방침 관련 기능을 구현하였다. 사용자가 필요한 정보를 직관적으로 확인할 수 있도록 화면 구조를 구성하였고, 접근성과 가독성을 고려해 UI 를 설계하였다. 또한 프로젝트 전반의 UI 디자인에도 참여하며, 단순히 화면을 구현하는 것을 넘어 실제 사용자의 이용 흐름과 편의성을 고려한 인터페이스를 만드는 데 집중하였다. 특히 시각장애인을 위한 서비스라는 점을 고려하여 버튼 크기, 정보 배치, 음성 안내 흐름 등 사용성이 중요한 요소들을 지속적으로 개선하였다.

이번 프로젝트를 진행하며 가장 크게 느낀 점은 개발 환경과 실제 사용 환경 사이에 큰 차이가 존재한다는 점이었다. 에뮬레이터나 실내 테스트에서는 정상적으로 동작하던 기능도 실외 환경에서는 전혀 다른 문제를 발생시키는 경우가 많았고, 이러한 차이를 줄여 나가는 과정이 서비스의 완성도를 결정한다는 것을 직접 경험할 수 있었다. 또한 시각장애인을 위한 서비스를 만든다는 것은 단순히 기능을 구현하는 것을 넘어, 사용자의 상황과 행동을 구체적으로 상상하며 설계해야 하는 과정이라는 점도 깊이 느낄 수 있었다. 아직 부족한 부분은 남아 있지만, 실제로 동작하는 서비스를 끝까지 구현하고 개선해본 경험은 앞으로의 개발 과정에서도 큰 밑거름이 될 것이라고 생각한다.

김예지

길벗 프로젝트에서 인공지능 모델 개발 중 보행 환경 세그멘테이션 모델 개발과 AI 서버 구현을 담당하였다. 주요 담당 영역은 보행 가능 영역, 점자블록, 주의 노면, 차도, 횡단보도 등 보행 환경을 구분하는 세그멘테이션 모델 학습과 추론 결과를 서비스에서 활용할 수 있도록 FastAPI 기반 AI 서버로 연결하는 작업이었다. 단순히 이미지를 분류하거나 모델을 학습시키는 데 그치지 않고, 시각장애인의 실제 보행 상황에서 어떤 노면 정보를 인식해야 안전한 안내로 이어질 수 있는지 고민하며 AI 기능을 서비스 흐름에 맞게 구현하는 데 집중하였다.

프로젝트 초반에는 보행 중 위험 요소를 판단하기 위해 객체 탐지 모델과 세그멘테이션 모델을 함께 사용하는 구조를 검토하였다. 이 중 내가 담당한 부분은 보행 가능 영역과 점자블록, 차도, 횡단보도 등 노면 정보를 구분하는 세그멘테이션 모델이었다. 처음에는 SegFormer 기반 모델을 활용하여 보행 환경을 분리하는 실험을 진행하였고, 기존 데이터셋의 라벨을 프로젝트 목적에 맞게 다시 정리하였다. 보도, 점자블록, 주의 노면, 차도, 횡단보도처럼 실제 보행 안내에 필요한 핵심 클래스를 중심으로 데이터 구조를 재구성하였으며, 서비스에서 필요한 정보가 무엇인지 먼저 정의한 뒤 이에 맞게 학습 방향을 설정하였다.

모델 개발 과정에서는 여러 시행착오를 겪으며 구조를 지속적으로 개선하였다. 초기에는 SegFormer 를 활용해 보행 가능 영역을 분리하는 방식을 실험하였지만, 멘토링을 통해 스마트폰 기반 실시간 서비스에서는 무거운 모델 구조가 속도와 배포 측면에서 부담이 될 수 있다는 점을 확인하였다. 이에 따라 이후에는 YOLO 기반 segmentation 모델을 중심으로 구조를 전환하고, 보행 안내에 꼭 필요한 클래스만 남기는 방향으로 모델을 단순화하였다. 최종적으로는 walkable surface, braille guide blocks,

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

caution surface, roadway, crosswalk 의 5 개 클래스로 재정의하여 모델을 학습하였다. 학습 결과 전체 Mask mAP50 은 0.730, Mask mAP50-95 는 0.548 을 기록하였고, 특히 보행 가능 영역은 Mask mAP50 0.832, 점자블록은 0.776 의 성능을 보여 실제 안내 기능에 활용 가능한 가능성을 확인할 수 있었다.

모델 성능뿐 아니라 실제 사용 가능성을 판단하기 위한 추론 속도와 구조 최적화도 함께 고려하였다. 길벗은 사용자가 걷는 중에 보행 환경을 인식해야 하는 서비스이기 때문에, 정확도뿐 아니라 지연 시간도 중요한 요소였다. 이에 따라 입력 해상도별 추론 속도를 비교하고, 모델 크기와 이미지 크기에 따른 FPS 차이를 실험하였다. 또한 멘토님의 조언에 따라 엣지 디바이스 및 온디바이스 적용 가능성도 검토하였고, TFLite 변환과 양자화 가능성을 일부 시도하였다. 다만 실제 프로젝트 결과물에서는 안정적인 기능 구현과 서버 연동을 우선하여 온디바이스 배포까지는 진행하지 않았고, 서버 기반 추론 구조를 중심으로 최종 프로토타입을 완성하였다. 이 과정에서 단순히 모델을 가볍게 만드는 것만으로는 충분하지 않으며, 실제 서비스 환경에서는 출력 형태, 후처리, 연동 안정성까지 함께 고려해야 한다는 점을 배웠다.

또한 나는 세그멘테이션 결과를 실제 안내에 활용할 수 있도록 후처리 로직을 설계하였다. 예를 들어 단순히 점자블록이 화면에 있는지 여부를 판단하는 것이 아니라, 점자블록 영역이 얼마나 안정적으로 인식되는지, 보행 가능 영역과 차도·횡단보도가 어떻게 구분되는지 확인하며 결과를 해석하였다. 특히 점자블록과 보행 가능 영역은 시각장애인의 이동 판단에 직접적인 영향을 주는 정보이기 때문에, 클래스별 마스크가 실제 화면에서 어떻게 나타나는지 반복적으로 확인하고 시각화 결과를 비교하였다.

AI 서버 구현 같은 경우, FastAPI 기반으로 이미지 분석 API 를 개발하고, 앱이나 백엔드에서 이미지를 전달하면 세그멘테이션 추론 결과와 보행 환경 정보를 반환할 수 있도록 구성하였다. 초기에는 모델 추론 결과를 그대로 확인하는 수준에서 시작했지만, 이후 서비스 연동을 위해 응답 구조를 정리하고, 백엔드와 Flutter 앱에서 활용할 수 있는 형태로 결과를 반환하도록 개선하였다. 또한 디버깅 과정에서 Swagger 응답뿐 아니라 서버 터미널에서도 오류 로그를 확인할 수 있도록 출력 구조를 정리해, 연동 과정에서 발생하는 문제를 빠르게 파악할 수 있도록 하였다.

캡스톤 프로젝트를 하며 가장 크게 느낀 점은, 인공지능 모델의 성능 수치가 곧바로 서비스의 완성도를 의미하지는 않는다는 점이었다. 처음에는 높은 정확도의 모델을 만드는 것이 가장 중요하다고 생각했지만, 실제 서비스에서는 어떤 클래스를 구분해야 하는지, 모델 결과를 어떤 기준으로 해석해야 하는지, 그리고 사용자가 실제로 도움을 받을 수 있는 형태로 어떻게 연결해야 하는지가 더 중요하다는 것을 알게 되었다. 특히 시각장애인을 위한 서비스는 일반적인 데모 환경과 달리 작은 오탐이나 지연도 사용자에게 불편이나 위험으로 이어질 수 있기 때문에, 모델 성능뿐 아니라 응답 시간, 안내 기준, 예외 상황 처리까지 함께 고려해야 했다.

최종 결과물의 사용 가능성을 평가해보면, 기능적 측면에서는 보행 가능 영역, 점자블록, 주의 노면, 차도, 횡단보도 등 핵심 보행 환경을 인식하고, 이를 서버를 통해 서비스와 연동할 수 있는 기본 구조를 구현했다는 점에서 프로젝트 목표를 달성했다고 판단한다. 또한 여러 모델 구조를 비교하고, 최종적으로 서비스 제약에 맞추어 단일 segmentation 모델 중심으로 발전시켰다는 점에서도 의미가 있었다. 다만 실제 상용 서비스 수준으로 사용하기 위해서는 다양한 날씨, 조도, 보행 환경, 카메라 각도에서의 추가 검증과 스마트폰 환경에서의 최적화가 더 필요하다. 따라서 현 단계의 결과물은

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

완성된 상용 서비스라기보다는, 길벗 서비스의 핵심 가능성을 보여주는 기능성 프로토타입에 가깝다고 평가할 수 있다.

그럼에도 불구하고 길벗은 실제 사용자의 문제에서 출발하여, 필요한 인공지능 기능을 정의하고, 데이터셋 구성부터 모델 학습, 추론 결과 확인, 서버 연동까지 하나의 서비스 흐름으로 구현해냈다는 점에서 충분한 의의가 있다. 나는 이번 프로젝트를 통해 단순히 모델을 학습하는 개발자에서 나아가, 사용자의 상황을 이해하고 기술적 선택을 서비스 가치와 연결해 판단하는 경험을 할 수 있었다. 여러 번의 구조 변경과 시행착오가 있었지만, 그 과정을 통해 더 현실적인 방향을 선택하고 개선해 나갔다는 점에서 스스로 의미 있는 성장을 이루었다고 생각한다.

양나래

길벗 프로젝트에서 백엔드 개발과 AI 기능 연동, 그리고 AWS 서버 배포 및 관리를 주로 담당하였다. 프로젝트에서는 단순히 API 를 구현하는 것에 그치지 않고, AI 모델이 실제 서비스 흐름 속에서 안정적으로 동작할 수 있도록 데이터 처리와 응답 구조를 설계하는 데 집중하였다. 또한 서비스 운영 환경까지 고려하여 AWS 서버 배포와 관리 업무를 함께 수행하며 전체 시스템이 안정적으로 동작할 수 있도록 구성하였다.

프로젝트에서는 AI 모델 연동 및 응답 처리 기능을 구현하였으며, 사용자의 입력 데이터를 처리하고 결과를 반환하는 API 개발을 담당하였다. 특히 AI 모델의 결과를 단순히 반환하는 것이 아니라, 실제 서비스 흐름 안에서 자연스럽게 연결될 수 있도록 데이터 전처리와 결과 최적화 로직을 설계하였다. 또한 실시간 기능과 사용자 인터랙션 과정에서 발생하는 여러 상황을 고려하여 예외 처리 로직과 상태 관리 기능도 함께 구현하였다. 프로젝트 전반의 기능 테스트와 오류 수정 과정에도 지속적으로 참여하며 서비스의 안정성과 완성도를 높이는 데 집중하였다.

가장 많은 시간과 노력을 들인 부분은 AI 모델의 응답 속도와 안정성을 개선하는 작업이었다. 초기에는 모델 응답 속도가 느려 사용자 경험이 저하되는 문제가 발생하였다. 특히 AI 기능 특성상 응답 지연이 길어질 경우 서비스 사용 흐름 자체가 끊기는 문제가 있었기 때문에 이를 해결하는 과정이 중요했다. 문제를 분석한 결과 서버 환경의 성능 한계가 주요 원인 중 하나라는 점을 확인하였고, 이후 AWS EC2 환경을 GPU 기반 서버로 변경하여 처리 속도를 개선하였다. 단순히 서버 사양만 높이는 것이 아니라, 실제 서비스 운영 시 발생하는 비용까지 고려하여 적절한 인스턴스를 선택하고 서버 구조를 최적화하는 과정도 함께 진행하였다. 이를 통해 응답 속도를 개선하면서도 운영 비용을 일정 수준 관리할 수 있도록 시스템을 구성하였다.

또한 예상하지 못한 입력값이나 예외 상황에서 오류가 발생하는 문제도 지속적으로 수정하였다. 실제 테스트 과정에서는 개발 단계에서 예상하지 못했던 다양한 입력 패턴이 발생하였고, 일부 상황에서는 AI 모델 결과가 정상적으로 반환되지 않거나 API 흐름이 중단되는 문제가 있었다. 이를 해결하기 위해 입력 검증 과정을 강화하고 예외 처리 로직을 추가하여 서비스가 비정상적인 상황에서도 안정적으로 동작할 수 있도록 개선하였다. 특히 단순히 오류를 해결하는 데 그치지 않고, 어떤 상황에서 문제가 발생하는지 로그를 분석하고 재현 테스트를 반복하며 원인을 찾는 과정에 많은 시간을 투자하였다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

AWS 서버 배포와 관리 과정 역시 중요한 경험이었다. 처음에는 단순히 기능만 정상적으로 실행되면 된다고 생각했지만, 실제 서비스 환경에서는 안정성과 비용, 그리고 유지보수 편의성까지 함께 고려해야 한다는 점을 배우게 되었다. 서버 배포 이후에도 지속적으로 모니터링을 진행하며 성능 문제를 확인하였고, 필요에 따라 서버 설정과 구조를 수정하며 안정적인 운영 환경을 구축하기 위해 노력하였다.

프로젝트를 수행하면서 가장 인상 깊었던 부분은 서비스의 성능과 안정성을 유지하는 일이 생각보다 훨씬 복합적인 과정이라는 점이었다. 초기에는 기능 구현 자체에 집중했지만, 실제로 여러 기능을 연결하고 테스트를 반복하는 과정에서 응답 속도, 서버 자원 사용량, 예외 상황 처리처럼 사용자에게 직접 영향을 주는 요소들이 서비스 품질에 매우 큰 영향을 준다는 것을 느낄 수 있었다. 특히 AI 기능은 입력 데이터나 서버 환경에 따라 결과와 처리 시간이 달라지는 경우가 많았기 때문에, 단순히 모델을 연동하는 것만으로는 충분하지 않았고 안정적으로 동작할 수 있도록 지속적인 개선 작업이 필요했다.

또한 AWS 서버를 직접 배포하고 운영하면서 개발 단계에서는 크게 고려하지 않았던 운영 비용과 서버 성능 사이의 균형도 중요하다는 점을 배우게 되었다. 응답 속도를 개선하기 위해 GPU 서버 환경으로 변경하는 과정에서는 단순히 성능만 높이는 것이 아니라 실제 운영 환경에서 효율적으로 유지할 수 있는 구조를 고민해야 했다. 이 과정에서 로그 확인, 성능 모니터링, 오류 분석 등을 반복하며 문제 원인을 찾고 개선하는 경험을 할 수 있었다.

협업 측면에서도 많은 것을 배울 수 있었다. 기능 간 연결 과정에서 예상하지 못한 충돌이나 수정 사항이 자주 발생했기 때문에, 팀원들과 진행 상황과 변경 사항을 꾸준히 공유하는 과정이 매우 중요했다. 이번 프로젝트를 통해 단순한 기능 개발을 넘어 실제 서비스 운영과 유지보수까지 고려하는 경험을 할 수 있었고, 이러한 경험은 앞으로 더 안정적이고 완성도 높은 서비스를 개발하는 데 큰 도움이 될 것이라고 생각한다.

이일환

길벗 프로젝트에서 나는 인공지능 모델 개발 중 객체 탐지 모델 개발, Depth Estimation 기반 거리 추정, 그리고 이를 종합하여 사용자에게 전달할 위험도 판단 및 안내 문장 생성 로직 설계를 담당하였다. 단순히 모델을 학습하고 결과를 출력하는 데 그치지 않고, 시각장애인 사용자가 실제 보행 중에 받게 되는 안내가 안전하고 실용적이어야 한다는 점을 가장 중요하게 생각하며 작업을 진행하였다.

객체 탐지 모델은 YOLOv11s 를 기반으로 파인튜닝하였다. 초기에는 일반적인 학습 방식으로 진행하였지만, 실제 보행 환경에서 촬영된 이미지를 분석해보니 카메라 프레임 가장자리에서 일부만 보이는 차량이나 사람, 다른 물체에 의해 가려진 객체가 굉장히 많이 등장한다는 것을 확인하였다. 그러나 일반적인 학습 방식에서는 이러한 잘린 객체나 가려진 객체에 대한 인식 성능이 충분히 확보되지 않았고, 이는 보행 보조 서비스에서 매우 치명적인 문제가 될 수 있다고 판단하였다. 예를 들어, 모서리에서 접근 중인 차량을 인식하지 못한다면 사용자에게 실질적인 위험이 될 수 있기 때문이다. 이를 해결하기 위해 학습 데이터 구성 단계에서 edge-touching 차량을 포함한 이미지에는 +2 회, occluded 객체를 포함한 이미지에는 +1 회의 oversampling 을 적용하여, 모델이 잘린 객체와 가려진 객체에 대해 더 민감하게 반응할 수 있도록 학습 전략을 설계하였다. 또한 2 단계 학습 구조를

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

적용하여, 1 차로 전체 데이터에 대해 학습한 뒤 2 차에서는 hard sample 중심으로 fine-tuning 하는 방식으로 모델의 견고성을 높였다.

모델 성능뿐만 아니라 실제 사용 가능성을 판단하기 위한 추론 속도와 구조 최적화도 함께 고려하였다. 길벗은 사용자가 걷는 중에 보행 환경을 실시간으로 인식해야 하는 서비스이기 때문에, 단순히 탐지 정확도만 높다고 해서 좋은 모델이라고 할 수 없었다. 정확도뿐 아니라 지연 시간이 사용자 경험에 직접적인 영향을 주기 때문에, 입력 해상도, 모델 크기, FPS, 최대 지연 시간 등을 함께 비교하며 최적의 균형점을 찾기 위해 노력하였다. 다양한 모델 후보(YOLOv8n, YOLOv11n, YOLOv11s, RT-DETR-L)를 비교 실험한 결과, YOLOv11s 가 F1 0.7167, Risk Recall 0.6998, 평균 추론 시간 13.07ms, FPS 76.51 로 정확도와 실시간성을 모두 충족하는 가장 균형 잡힌 모델임을 확인하고 최종적으로 선정하였다.

가장 어려웠던 부분은 객체 탐지 결과를 어떻게 실제 위험 판단으로 연결할 것인가 하는 문제였다. 처음에는 객체 탐지 모델의 bbox 크기만으로도 충분히 안내 음성을 생성할 수 있을 것이라고 생각하였다. bbox 가 크면 가까이 있는 객체이고, 작으면 멀리 있는 객체라는 단순한 가정에서 출발한 것이다. 그러나 실제로 테스트를 진행해보니 이 가정은 생각보다 자주 깨졌다. 예를 들어 멀리 있지만 크기 자체가 큰 버스나 트럭은 화면 안에서 bbox 가 크게 잡혀 사용자에게 가까운 위험으로 잘못 안내되었고, 반대로 사용자 바로 앞에 있는 작은 볼라드나 가까이 접근한 자전거처럼 정작 위험한 객체는 bbox 크기가 작아 안내 대상에서 빠지는 경우가 발생하였다. 즉, "큰 객체 = 가까운 객체"라는 단순한 공식만으로는 시각장애인이 실제로 위험을 인지해야 하는 상황을 제대로 판단할 수 없다는 것을 직접 확인하게 되었다.

이를 해결하기 위해 Depth Estimation 모델을 추가로 도입하기로 결정하였다. FastDepth 와 LiteMono, Depth Anything V2 Metric Outdoor 모델을 직접 비교 실험한 끝에 객체별 깊이 차이를 가장 안정적으로 표현하는 Depth Anything V2 Metric Outdoor 모델을 최종 선정하였다. 그러나 모델을 적용한 것만으로는 실제 거리값과 추정값 사이의 오차가 여전히 컸다. 모델이 출력하는 raw depth 값을 그대로 사용하면 어떤 구간에서는 지나치게 가깝게, 어떤 구간에서는 지나치게 멀게 추정되는 경우가 있었다. 이를 해결하기 위해 실제 환경에서 다양한 거리에 객체를 두고 측정한 결과를 수집한 뒤, 모델이 추출한 거리값과 실제 거리값이 최대한 일치하도록 직접 계산하여 선형 보정식을 도출하였다. $\text{거리} = \text{raw depth} * \text{scale factor} + \text{offset}$ 형태의 보정식을 적용하였고, 최종적으로 DEPTH_SCALE_FACTOR = 0.6515, DEPTH_OFFSET_M = -0.4844 값을 사용하여 보정하였다. 또한 bbox 전체 영역의 depth 를 그대로 사용할 경우 차량 하단의 도로면이나 배경 픽셀이 함께 섞이는 문제가 있었기 때문에, 객체 하단 10% 영역에서도 가장 가까운 25% 픽셀만 선별하여 중앙값을 사용하는 방식으로 개선하였다. 이 과정에서 단순히 모델을 가져다 적용하는 것을 넘어, 실제 환경에서의 측정과 분석을 바탕으로 모델 출력을 직접 보정하는 경험을 할 수 있었다.

거리 추정 기능을 추가한 이후에도 안내 문장 생성 로직을 설계하는 과정에서 예상하지 못한 예외 상황이 많이 등장하였다. 차량이 화면에 보이지만 실제로는 차도 위에 있어 사용자의 보행 경로와 무관한 경우, 객체가 가까이 있지만 진행 방향과 반대 방향에 있어 위험도가 낮은 경우, 멀리 있는 큰 객체가 크게 탐지되어 실제보다 가깝게 추정되는 경우, 동일한 객체가 매 프레임마다 반복적으로 안내되어 사용자에게 피로감을 주는 경우 등 단순한 규칙만으로는 처리하기 어려운 상황들이 끊임없이 나타났다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

이러한 예외 상황들을 처리하기 위해 여러 단계의 후처리 로직을 설계하였다. 차도 위 객체에 대해서는 segmentation 결과의 road mask 와 bbox 를 비교하여 보행 경로와 겹치지 않는 경우 위험도를 낮추도록 하였다. 작은 bbox 에 대해서는 화면 높이 대비 비율이 15% 이하인 경우 최소 신뢰 거리 기준(8m)을 적용하여 과도하게 가깝게 판단되는 문제를 보완하였다. 반복 안내 문제는 동적 객체와 정적 객체에 서로 다른 cooldown frame 을 적용하였다. 이렇게 하나하나의 예외 상황을 직접 마주하고 해결해 나가는 과정에서, 실제 서비스가 동작하는 환경은 개발자가 미리 상상하는 것보다 훨씬 다양하고 복잡하다는 점을 깊이 체감하였다.

이러한 예외 상황들을 발견하고 해결할 수 있었던 가장 큰 이유는 실제 환경에서 반복적으로 테스트를 진행했기 때문이었다. 처음에는 정적인 테스트 이미지와 영상만으로 충분할 것이라고 생각했지만, 실제로 카메라를 들고 다양한 시간대와 장소에서 직접 걸어 다니며 테스트해보니 책상 앞에서는 전혀 예상하지 못했던 문제들이 끊임없이 나타났다. 낮과 저녁, 맑은 날과 흐린 날, 사람이 많은 거리와 한적한 골목, 인도가 좁은 구간과 넓은 구간 등 환경이 바뀔 때마다 모델의 반응이 달라졌고, 그에 따라 안내 문장이 어색하거나 부정확하게 출력되는 상황이 발생하였다. 거리 임계값(3m, 5.5m, 8m), cooldown frame 수치, 작은 객체 보정 기준 등은 모두 한 번에 정해진 것이 아니라 수십 번의 실외 테스트를 통해 점차 조정된 결과였다. 또한 안내 음성이 실제 보행 속도와 맞지 않거나, 사용자가 이미 객체를 지나친 뒤에 안내가 출력되는 등 시간적 요소와 관련된 문제들도 직접 걸어보지 않으면 발견하기 어려웠다. 이 과정을 통해 AI 서비스는 모델이 "동작하는 것"과 "사용자에게 실제로 도움이 되는 것" 사이에 큰 간극이 있으며, 그 간극을 좁히는 것은 결국 실제 환경에서의 반복적인 테스트라는 점을 깊이 체감하였다.

이번 프로젝트를 통해 가장 크게 배운 점은, AI 서비스에서는 모델의 성능 수치뿐만 아니라 지연 시간이 사용자 경험에 매우 중요한 영향을 미친다는 사실이었다. 아무리 정확한 모델이라도 응답이 늦으면 사용자가 이미 위험에 노출된 뒤에 안내가 도착하게 되고, 이는 시각장애인을 대상으로 하는 서비스에서는 치명적인 문제가 된다. 이번에는 서버 기반 추론 구조로 안정성을 확보하는 방향을 선택하였지만, 결국 보행 보조와 같은 실시간성이 중요한 서비스에서는 네트워크 지연이 없는 온디바이스 구조가 더 적합하다고 생각한다. 따라서 추후에는 현재 모델을 경량화하고 양자화하여 스마트폰에서 직접 추론 가능한 온디바이스 모델로 발전시키는 것을 목표로 하고 싶다. 이를 통해 네트워크 환경에 영향을 받지 않으면서도 더 빠른 응답을 제공할 수 있는 서비스로 길벗이 발전할 수 있을 것이라고 기대한다.

전체적으로 이번 프로젝트는 단순히 모델을 학습하는 경험을 넘어, 실제 사용자의 안전과 직접 연결되는 AI 서비스를 설계하고 구현하는 경험을 할 수 있었다는 점에서 매우 의미 있었다. 모델 선정, 거리 보정, 예외 처리 로직 설계까지 모든 과정에서 "이 안내가 사용자에게 실제로 도움이 될 수 있는가"를 기준으로 판단하며 작업하였고, 이 과정에서 기술적 의사결정을 사용자 가치와 연결해서 생각하는 능력을 키울 수 있었다고 생각한다.

전예찬

프로젝트 길벗에서 Spring Boot 기반 백엔드 서버 설계 및 REST API 개발을 담당했다. 특히 위치 및 장소 조회, 주소 변환, 위치 기반 데이터 처리 구조와 횡단보도 및 음향신호기 안내 구조 설계를 중심으로 개발을 수행했다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

본 프로젝트는 시각장애인 및 보행약자의 안전한 보행을 지원하는 서비스이므로, 단순히 데이터를 조회하는 API 보다 사용자의 현재 위치와 목적지를 정확하게 해석하고, 보행 안내에 필요한 정보를 안정적으로 제공하는 백엔드 구조가 중요하다고 판단했다. 이에 따라 장소 검색과 주소 변환 기능을 보행 안내의 기반 기능으로 보고, 카카오 로컬 API 를 활용해 장소 검색, 지오코딩, 역지오코딩 기능을 구현했다.

개발 과정에서 카카오 로컬 API 만으로는 역지오코딩 정확도에 한계가 있었다. 특히 특정 좌표에서 도로명 주소가 충분히 정확하게 반환되지 않아 지번 주소와 도로명 주소가 혼용되거나 행정동 수준의 주소로 반환되는 문제가 발생했다. 이를 해결하기 위해 Naver Maps API 를 함께 활용하는 방향으로 구조를 확장했고, 카카오와 네이버의 응답 결과를 비교해 더 적절한 주소를 선택할 수 있도록 위치 기반 데이터 처리 로직을 개선했다. 이 과정에서 단순히 외부 API 를 호출하는 것에 그치지 않고, 응답 데이터의 신뢰도와 실제 사용자 위치 사이의 거리 오차를 함께 고려해야 한다는 점을 확인했다.

인공지능 모델이 영상 기반으로 횡단보도를 탐지하지만, 실제 환경에서는 조명, 가림, 촬영 각도 등에 따라 오탐 또는 미탐이 발생할 수 있다고 판단했다. 이를 보완하기 위해 백엔드에서는 사용자의 현재 위치를 기준으로 주변 횡단보도 정보를 조회하고, 보행 경로와 결합하여 횡단보도 접근 여부를 확인할 수 있는 기능을 설계했다. 즉, AI 탐지 결과에만 의존하지 않고 위치 기반 공공데이터를 함께 활용하여 횡단보도 안내의 신뢰성을 높이고자 했다. 특히 TMAP 보행 경로에서 횡단보도 후보를 추출하고, 자체 DB 의 횡단보도 및 음향신호기 데이터와 매칭해 사용자가 횡단보도에 접근했을 때 안전 정보를 받을 수 있도록 설계했다.

다만 공공데이터 중 일부는 좌표값이 누락되어 있었고, 이러한 데이터를 임의로 보정하여 DB 에 저장할 경우 실제 위치와 다른 횡단보도 정보가 포함되어 데이터 정합성이 떨어질 수 있다고 판단했다. 특히 시각장애인의 보행 안내에서는 잘못된 위치 정보가 오히려 안전에 영향을 줄 수 있으므로, 좌표가 없는 데이터는 우선 DB 저장 대상에서 제외했다. 대신 좌표 기반 공공데이터로 확인 가능한 횡단보도는 백엔드에서 보완하고, 좌표 기반으로 확인하기 어려운 상황은 AI 기반 탐지가 함께 보완하는 구조로 보았다. 한편 서로 다른 데이터셋 간 좌표 체계와 주소 정보가 일치하지 않는 문제에 대해서는 거리 기반 매칭, 행정구역 정보 비교 등을 함께 고려하는 방향으로 개선했다.

이번 프로젝트에서 나는 위치 기반 백엔드 기능의 정확성과 안정성을 높이는 역할을 수행했다. 시각장애인의 보행 보조를 돕는 서비스인 만큼 위치 정확도가 사용자의 안전과 직접적으로 연결된다고 생각하여, GPS 오차와 외부 API 응답 오차, 공공데이터의 좌표 누락 및 데이터셋 간 차이를 줄이기 위해 여러 방법을 시도했다. 캡스톤 개발 기간 안에 모든 오차를 충분히 줄이지 못한 점은 아쉬웠지만, 정확도 문제를 해결하기 위해 다양한 API 와 데이터 처리 방식을 비교하고 개선 방향을 찾아보는 경험을 할 수 있었다. 이를 통해 백엔드 개발에서 기능 구현뿐 아니라 데이터의 신뢰성, 예외 상황, 실제 사용자 환경까지 함께 고려해야 한다는 점을 배울 수 있었다.

황연주

Flutter 기반 프론트엔드 개발을 중심으로 서비스 기획 및 개발 기능 분배, UI/UX 설계, 접근성 설계, 권한 아키텍처 구축, Git 협업 체계 정립(ISSUE/PR 템플릿 제작)을 수행하였으며, 프로젝트 초기 환경 세팅 및 앱 전체 컬러-텍스트 디자인 시스템 정의, 카카오 소셜 로그인 및 자동 로그인(AuthGate), Title Bar, 회원탈퇴, 앱 아이콘 및 앱 이름 설정, 장애물 탐지, 온디바이스 추론(TFLite) 연결, 통합

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

모드, 음성(TTS) 및 진동·효과음 안내, 설정 관리, 권한 관리, 날씨 안내 TTS, 반응형 UI(기종별 화면 크기 대응), 스크린리더(TalkBack) 접근성 세부 구현, iOS/Android 배포 환경 설정을 담당하였다.

프로젝트 초기에는 시각장애인의 실제 보행 환경과 기존 보행 보조 서비스의 한계를 조사하며 문제를 정의하였다. 단순히 “AI 기능을 구현하는 것”이 아니라, 실제 사용자가 안전하고 예측 가능하게 사용할 수 있는 서비스를 만드는 것을 핵심 목표로 설정하였다. 또한 필요한 기능들을 세부 단위로 리스트업하고, 프론트엔드 팀원 간 역할과 작업 범위를 명확히 분배하기 위한 업무 관리 시트를 작성하였다. 이를 통해 기능별 우선순위와 담당 영역을 체계적으로 관리할 수 있었으며, 병렬 개발 과정에서 발생할 수 있는 작업 중복이나 누락을 최소화하였다. 아울러 프로젝트 규모가 커질 것을 고려하여 컴포넌트, 화면, 서비스, 모델 등의 역할을 기준으로 Flutter 프로젝트 구조를 체계적으로 설계하였다. 공통 컴포넌트와 기능별 파일을 분리하고, 재사용 가능한 구조로 설계함으로써 유지보수성과 확장성을 높였으며, 이후 기능 추가 및 협업 과정에서도 코드 탐색과 개발이 용이한 환경을 구축하였다. 이러한 초기 구조화 작업은 프론트엔드 개발 생산성을 높이고, 프로젝트가 복잡해진 이후에도 일관된 개발 흐름을 유지하는 기반이 되었다.

UI/UX 설계 단계에서는 Figma 를 활용하여 전체 화면 구조와 기능 흐름을 설계하였다. 시각장애인을 주 사용자로 상정하였기 때문에 일반적인 시각 중심 UI 설계 방식이 아니라, 스크린 리더 사용 환경과 음성 기반 인터랙션을 중심으로 화면 구조를 설계하였다. 버튼 영역을 충분히 확보하고 터치 탐색 시 오동작 가능성을 줄이기 위해 화면 구성 요소를 단순화하였으며, Semantics 위젯을 활용해 스크린 리더 접근성을 강화하였다. 또한 실제 시각장애인 사용자를 대상으로 기능 흐름과 음성 안내를 검증받으며 안내 문구, 기능 진입 흐름 등을 지속적으로 수정하였다. 이 과정은 단순히 기능이 동작하는지 확인하는 수준을 넘어, 실제 사용자가 서비스를 안전하고 자연스럽게 사용할 수 있는지를 검증하는 경험이었다.

프론트엔드 구현에서는 Flutter 기반 전체 앱 구조를 설계하고 구현하였다. 사용자 인증 기능에서는 카카오 OAuth 기반 로그인 및 회원가입 흐름을 구현하여 사용자가 별도의 회원 정보 입력 없이 간편하게 서비스를 이용할 수 있도록 하였다. 카카오 로그인 성공 후 발급되는 사용자 인증 정보를 백엔드 서버와 연동하여 자체 액세스 토큰 및 리프레시 토큰을 발급받는 구조를 구현하였으며, 앱 재실행 이후에도 로그인 상태가 유지될 수 있도록 토큰 저장 및 자동 로그인 흐름을 설계하였다. 또한 토큰 만료 상황을 고려하여 재발급 로직과 예외 처리 구조를 구현함으로써 인증 상태가 예기치 않게 해제되는 문제를 방지하고 안정적인 사용자 세션 관리를 가능하게 하였다. 홈 화면은 기존 3 개 기능 구조에서 통합 모드가 추가된 4 개 기능 구조로 재설계하였으며, 시각장애인 사용자가 화면 전체를 탐색하기 쉽도록 Expanded 위젯을 활용하여 버튼 영역을 균등하게 배치하였다.

장애물 탐지 기능에서는 실시간 카메라 스트리밍과 AI 서버 연동 구조를 구현하였다. 카메라 프레임을 서버로 전송하고 WebSocket 을 통해 AI 추론 결과를 수신하는 구조를 설계하였으며, alert_level 기반의 위험 등급 체계를 구성하여 위험 수준에 따라 진동과 TTS 안내를 차등 제공하도록 구현하였다. 또한 일정 시간 이상 갱신되지 않은 탐지 결과를 자동 제거하는 primaryObjectId 를 활용한 stale 처리 로직을 추가하여 오래된 탐지 정보가 사용자에게 혼란을 주지 않도록 하였다. 또한 개발 과정에서는 장애물 탐지 기능의 동작 여부를 신속하게 검증하고, 앱 → 서버 → AI 추론으로 이어지는 전체 파이프라인의 디버깅 효율을 높이기 위해 DebugOverlay 기능을 구현하였다. 이를 통해 현재 서버로 전송되고 있는 실시간 카메라 프레임을 앱 내부에서 직접 확인할 수 있도록 하였으며, 카메라 입력 상태, 프레임 전송 여부 등을 시각적으로 검증할 수 있는 디버깅 환경을 구축하였다. 이는 AI 모델

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

자체의 문제와 네트워크·프론트엔드 전송 과정에서 발생하는 문제를 구분하여 분석하는 데 실질적인 도움이 되었다.

개발 과정에서 가장 많은 설계적 고민이 필요했던 영역은 권한 관리였다. 초기 구현에서는 기능 진입 시마다 시스템 권한 팝업이 반복적으로 출력되거나, 권한 거부 시 기능이 조용히 중단되는 문제가 존재하였다. 이는 일반 사용자보다 시각장애인 사용자에게 훨씬 큰 혼란을 줄 수 있는 문제라고 판단하였다. 이를 해결하기 위해 권한 요청 시점을 앱 최초 실행 시의 온보딩 과정으로 일원화하고, 이후에는 snack bar 또는 시스템 팝업 대신 현재 권한 상태만 확인하여 권한이 거부된 경우 기능별 안내 AlertDialog 를 출력하는 구조를 설계하였다. 또한 장애물 탐지는 카메라 권한, 스마트 길찾기는 위치 권한, 통합 모드는 카메라와 위치 권한을 각각 독립적으로 검사하도록 분리하였다. 이 과정에서 speech_to_text 패키지의 초기화 시점 때문에 앱 시작과 동시에 마이크 권한 팝업이 출력되는 문제를 발견하여 실제 마이크 버튼을 누르는 시점으로 초기화 위치를 변경하는 등 플랫폼 동작 원리에 기반한 여러 엡지 케이스를 직접 해결하였다.

설정 화면에서는 OS 권한 상태와 앱 내부 설정 상태 간의 불일치 문제를 해결하기 위해 앱 생명주기 기반 상태 동기화 구조를 구현하였다. WidgetsBindingObserver 를 활용하여 앱 복귀를 감지하고, 낙관적 UI 패턴을 적용하여 사용자가 토글을 변경했을 때 즉각적인 시각적 피드백을 제공하면서도 실제 권한 상태에 따라 최종 상태를 확정하도록 설계하였다. 또한 서버에서 관리하는 설정값과 앱 내부 로컬 설정값을 명확히 분리하여 SharedPreferences 기반의 로컬 저장 구조를 구현함으로써 네트워크 오류 상황에서도 사용자 설정이 유지되도록 구성하였다.

기술 구현 외에도 팀 전체의 Git 협업 체계를 직접 설계하고 운영하였다. 브랜치 전략을 master → front-develop / back-develop / ai-develop → feature/{담당분야}/{기능명} 구조로 설계하여 프론트엔드와 백엔드, AI 작업 영역을 분리하고 병렬 개발 시 충돌을 최소화하였다. 또한 커밋 메시지 컨벤션([FEAT], [FIX], [REFACTOR], [DOCS] 등)을 도입하고, 이슈 및 PR 템플릿을 직접 제작하여 팀원들이 일정한 형식으로 작업 내용을 기록하고 리뷰할 수 있도록 가이드하였다. 이를 통해 코드 리뷰 효율이 높아졌으며, 브랜치 충돌 및 작업 이력 추적 문제를 효과적으로 줄일 수 있었다.

프로젝트 중반에는 온디바이스 AI 구조의 가능성을 검토하기 위해 TensorFlow Lite 기반 추론 구조를 직접 탐색하였다. Flutter 연동까지 실제 구현을 진행하였으나, 모바일 기기 성능 편차와 실시간 추론 속도 한계, 서비스 특성상 요구되는 안정성을 고려했을 때 온디바이스 구조가 적합하지 않다는 결론에 도달하였다. 이후 팀 논의를 통해 서버 기반 추론 구조로 복귀하였으며, 이를 통해 기술 도입 자체보다 서비스 요구사항에 적합한 구조를 선택하는 판단 능력의 중요성을 경험할 수 있었다.

최종적으로 본 프로젝트의 결과물은 단순 프로토타입 수준을 넘어 실제 사용 가능성을 고려한 통합형 보행 보조 서비스로 발전하였다. 접근성 기반 UI 설계, 음성 중심 인터랙션, 실시간 AI 추론 연동, 안정적인 권한 관리 흐름, 상태 지속성 구조 등을 통해 시각장애인이 보다 안전하고 일관된 경험 속에서 서비스를 사용할 수 있는 기반을 마련하였다. 실제 사용자를 대상으로 한 검증 과정에서도 기능 흐름과 접근성 측면에서 긍정적인 피드백을 확인할 수 있었다.

종합적으로 본 프로젝트를 통해 본인은 단순히 기능을 구현하는 개발자가 아니라, 사용자 맥락을 기반으로 문제를 정의하고 기술적 결정을 내리며 협업 구조까지 설계할 수 있는 엔지니어로 성장할 수 있었다고 평가한다. 특히 접근성 중심 UI/UX 설계, 권한 시스템 및 앱 생명주기 이해, 실시간 AI 추론 연동, 협업 프로세스 구축 경험은 실무에서도 직접적으로 활용 가능한 역량이라고 판단한다. 또한

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

기술을 무조건 도입하기보다 서비스 목적과 실제 사용자 경험을 기준으로 적절성을 판단하고 구조를 선택했던 경험은 본 프로젝트를 통해 얻은 가장 큰 성장이라고 생각한다.

4 참고 문헌

번호	종류	제목	출처	발행년도	저자	기타
1	가이드라인	Web Content Accessibility Guidelines (WCAG) 2.1	W3C	2018	W3C Web Accessibility Initiative	링크
2	가이드라인	Accessible Typography Guidelines for Inclusive Mobile App Design	MoldStud	2025	Vasile Crudu & MoldStud Research Team	링크
3	논문	The design of auditory user interfaces for blind users	<i>Proceedings of the second Nordic conference on Human-computer interaction</i> (pp. 149-156)	2002	H. Donker, P. Klante, P. Gorny	링크
4	기술문서	SK open API 보행자 경로 안내	<i>SK open API</i>	-	SK open API	링크
5	논문	VisionGPT: LLM-Assisted Real-Time Anomaly Detection for Visual Navigation	<i>arXiv preprint arXiv:2403.12415.</i>	2024	Hao Wang, Jiayou Qin, Ashish Bastola, Xiwen Chen, John Suchanek, Zihao Gong, Abolfazl Razi	링크
6	논문	Segformer: Simple and Efficient Design for Semantic Segmentation with	<i>Advances in neural information processing systems, 34, 12077-12090.</i>	2021	Xie, E., Wang, W., Yu, Z., Anandkumar, A., Alvarez, J. M., & Luo, P.	링크

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

		Transformers				
7	기술문서	Ultralytics YOs 살펴보기	<i>Ultralytics</i>	2024	Ultralytics	링크
8	기술문서	Huggingface Segformer	<i>Huggingface</i>	2021	Hugging Face	링크
9	기술문서	Kakao Developers 문서	<i>Kakao Developers</i>	-	Kakao	링크
10	기술문서	Naver Maps 문서	<i>Naver Cloud Platform</i>	-	Naver	링크
11	기술문서	DepthAnything- V2	<i>github</i>	2024	DepthAnything	링크
12	논문	DepthAnything- V2	<i>arxiv.org/abs/2406.09414</i>	2024	Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, Hengshu	링크

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

5 부록

5.1 사용자 매뉴얼

설치 가이드

- Android 9 이상의 스마트폰에서 제공된 길벗 APK 파일을 다운로드한다.
- 설정 > 보안 > 알 수 없는 소스 허용 후 APK 를 설치한다.
- 앱 실행 후 카메라, 마이크, 위치 권한을 허용한다.
- MAC 환경에서 IPA 파일을 AltStore 또는 Xcode 를 통해 iOS 기기에 설치한다.
- 앱 실행 후 카메라, 마이크, 위치 권한을 허용한다.

주요 기능 사용 방법

- 로그인: 앱 실행 후 카카오 로그인 버튼을 탭하여 카카오 계정으로 인증한다.
- 장애물 탐지 시작: 홈 화면에서 '장애물 탐지' 버튼을 탭한다. 카메라를 전방으로 향하게 하면 실시간 탐지가 시작된다. 장애물 감지 시 음성 안내 및 진동으로 알림을 받는다.
- 스마트 길찾기: '스마트 길찾기' 버튼을 탭한 후 음성으로 목적지를 말하거나 검색창에 입력한다. 구간별 방향 안내가 TTS 로 제공된다. (단독 실행 - 장애물 탐지 미포함)
- 통합 모드: '통합 모드' 버튼을 탭하면 경로 안내와 장애물 탐지를 동시에 제공한다. 목적지 입력 후 보행 중 실시간 장애물 감지 및 경로 안내를 함께 받을 수 있다.
- 설정: '설정' 화면에서 안내 문장 길이와 빠르기, 진동 강도를 조절할 수 있다.
- TalkBack 지원: Android TalkBack을 활성화하면 모든 UI 요소에 음성 안내가 제공된다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

테스트 가이드

길벗 테스트 가이드 - 애플리케이션 화면 설명

1. 시작 화면



"시작하기" 버튼을 눌러 로그인합니다. 카카오톡 로그인을 지원합니다. 로그인 성공시 홈 화면으로 이동됩니다.

"사용 방법" 버튼을 누르면 사용 가이드를 볼 수 있습니다.

2. 홈 화면



목적지 없이, 보행 중 장애물을 탐지하여 안내합니다.

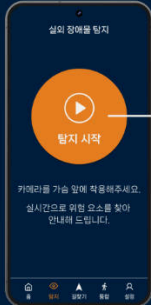
목적지를 입력하면 경로를 안내합니다.

목적지를 입력하면 경로를 안내하고, 이동 중 장애물도 함께 알려줍니다.

안내 메시지 상세도와 빠르기, 진동 강도를 개인에 맞게 설정할 수 있습니다.

길벗 테스트 가이드 - 애플리케이션 화면 설명

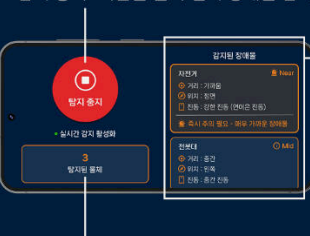
3. 실외 장애물 탐지 화면



"탐지 시작" 버튼을 눌러 실외 장애물 탐지 모드를 시작합니다.

카메라를 가슴 높이에 맞춰 고정시켜주세요.

4. 실외 장애물 탐지 중 화면



"탐지 중지" 버튼을 눌러 실외 장애물 탐지를 종료합니다.

감지된 장애물 목록
가까운 거리: 강한 연속 진동
중간 거리: 중간 진동
먼 거리: 약한 진동

탐지된 물체의 개수가 표시됩니다.

길벗 테스트 가이드 - 애플리케이션 화면 설명

5. 길찾기&통합모드 시작 전 화면



목적지를 입력합니다. 키보드 입력, 마이크 입력 모두 지원합니다.

목적지를 선택했다면 "안내 시작"을 누릅니다.

저장된 장소 목록 화면으로 이동합니다.

자주가는 목적지를 저장할 수 있습니다. 해당 장소를 눌러 목적지를 선택합니다.

최근 방문한 목적지가 표시됩니다.

6. 저장된 장소 목록 화면



저장된 장소들을 모두 볼 수 있는 화면입니다.

장소를 누르면 목적지가 선택되고, 자동으로 이전 화면으로 돌아갑니다.

"편집하기" 버튼을 눌러 저장된 장소를 편집할 수 있습니다.

길벗 테스트 가이드 - 애플리케이션 화면 설명

7. 저장된 장소 수정 화면



"편집하기"를 눌렀을 때 화면입니다. 저장된 장소를 삭제하거나, 추가할 수 있습니다.

"삭제" 버튼을 눌러 장소를 삭제합니다.

"추가" 버튼을 누르면 장소 추가 화면으로 이동합니다.

"저장하기" 버튼을 눌러 편집을 마칩니다.

8. 저장된 장소 추가 화면



목적지를 입력합니다. 키보드 입력, 마이크 입력 모두 지원합니다.

장소 이름을 입력합니다. 키보드 입력, 마이크 입력 모두 지원합니다. (중복된 이름은 사용할 수 없습니다.)

장소를 대표할 카테고리 아이콘을 선택합니다.

모든 값을 입력하면 "추가하기" 버튼이 활성화됩니다. 버튼을 눌러 장소 저장을 완료합니다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

길벗 테스트 가이드 - 애플리케이션 화면 설명

9. 길찾기&통합모드 중 화면

*길찾기 전용 모드에서는 장애물 탐지 결과가 제공되지 않습니다.

남은 거리, 예상 소요 시간, 경로 상태 등을 보여줍니다.



이동 방향을 안내합니다.

위험 요소 / 장애물을 알려줍니다.

"안내 중지" 버튼을 눌러 길 안내를 종료합니다.

10. 설정 화면 (1)

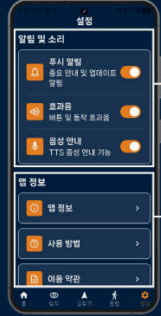


클릭하여 "계정 화면"으로 이동합니다.

안내 문장 길이와 빠르기, 진동 강도를 선호도에 맞게 조절할 수 있습니다.

길벗 테스트 가이드 - 애플리케이션 화면 설명

11. 설정 화면 (2)



알림 및 소리 활성화 여부를 선택합니다.

앱 관련 정보들을 확인할 수 있습니다.

- 앱 정보
- 사용 방법
- 이용 약관
- 개인정보 처리 방침

12. 계정 화면



현재 로그인 정보를 확인할 수 있습니다.

"로그아웃" 버튼을 눌러 로그아웃합니다.

"회원 탈퇴" 버튼을 눌러 계정을 삭제합니다.

5.2 운영자 매뉴얼

서버 환경

- 백엔드 서버
 - AWS EC2 Auto Scaling Group (Private App Subnet)
 - Amazon Linux 2023
 - Docker + Spring Boot (Java 21)
 - Public ALB 를 통해 80/443 요청 수신 후 Spring Boot 8080 포트로 전달
 - Launch Template + UserData 기반 자동 배포 구조
- AI 서버
 - AWS EC2 GPU Auto Scaling Group (Private AI Subnet)
 - Deep Learning AMI (Ubuntu 22.04 기반)
 - Docker + FastAPI (Python 3.11)
 - Internal ALB 를 통해 Spring 서버에서만 접근 가능
 - NVIDIA GPU 기반 AI 추론 환경 구성
- 데이터베이스
 - Amazon RDS MySQL
 - Private DB Subnet 배치

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

- Public Access 비활성화
- Spring 서버에서만 접근 가능

배포 설정

- 백엔드, AI 서버
 - Docker 이미지 빌드 후 Amazon ECR 업로드
 - Auto Scaling Group 의 EC2 가 ECR 이미지 자동 pull 및 실행
- 환경 변수
 - AWS Systems Manager Parameter Store 기반 .env 관리
 - EC2 UserData 에서 Parameter Store 값을 읽어 .env 파일 자동 생성
- API 인증: SK OpenAPI, OpenWeatherMap API, Tmap API, 서울 열린데이터 광장, 공공 데이터포털, 카카오 OAuth2.0/Local API, 네이버 Developers API,
- API 문서: Swagger UI (<https://gilbeot.p-e.kr/swagger-ui/index.html>) 접근 가능

모니터링

- 탐지 결과는 JSON 형태로 백엔드에 전달되며, 영상 이미지는 서버에 저장되지 않는다.
- 이상 상황 발생 시 Spring Boot 로그 및 FastAPI 로그를 통해 원인을 확인한다.

5.3 배포 가이드

Android APK/ABB 배포

- Flutter apk 빌드: flutter build apk --release --dart-define-from-file=.env.json
- 빌드 산출물: build/app/outputs/flutter-apk/app-release.apk
- 배포: 직접 APK 파일 공유
- Flutter abb 빌드: flutter build appbundle --dart-define-from-file=.env.json
- 빌드 산출물: build/app/outputs/flutter-apk/app-release.abb
- 배포: Google Play Console 을 통한 배포

iOS IPA 배포

- Flutter ipa 빌드: flutter build ipa --release --dart-define-from-file=.env.json
 - 빌드 산출물: build/ios/ipa/Runner.ipa
 - 배포: Xcode Organizer / TestFlight 또는 App Store Connect 를 통한 배포
- 추가로 iOS 는 배포 전에 다음 과정들이 필요..

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

- Apple Developer 계정 등록
- Xcode Signing & Capabilities 설정
- Provisioning Profile / Certificate 설정
- App Store Connect 앱 등록

백엔드 서버 배포

- Docker 이미지 빌드: ./gradlew clean bootJar 후 docker build -t spring-api .
- ECR 업로드: spring-api:latest 이미지를 Amazon ECR 에 push
- 배포 방식: Spring Launch Template + Auto Scaling Group 사용
- 실행 방식: EC2 UserData 에서 ECR 이미지 pull 후 docker compose up -d 실행
- 트래픽 흐름: Public ALB 80/443 포트 → Spring Boot 8080 포트 전달
- 배치 위치: Spring EC2 는 Private App Subnet 에 배치
- 보안 그룹: Public ALB 는 80/443 허용, Spring EC2 는 ALB 에서 오는 8080 만 허용
- 환경 변수: SSM Parameter Store 에서 .env 생성
- 내부 통신: Spring 서버는 Internal ALB 를 통해 AI 서버 호출, RDS 는 Private DB Subnet 에서만 접근

AI 서버 배포

- Docker 이미지 빌드: docker build -t fastapi-gpu .
- ECR 업로드: fastapi-gpu:latest 이미지를 Amazon ECR 에 push
- 배포 방식: FastAPI GPU Launch Template + Auto Scaling Group 사용
- 실행 방식: EC2 UserData 에서 ECR 이미지 pull 후 docker compose up -d 실행
- FastAPI 실행: uvicorn app.main:app --host 0.0.0.0 --port 8000
- 트래픽 흐름: Spring 서버 → Internal ALB → FastAPI GPU ASG 8000 포트
- 배치 위치: FastAPI GPU EC2 는 Private AI Subnet 에 배치
- 보안 그룹: FastAPI EC2 는 Internal ALB 에서 오는 8000 포트만 허용
- 외부 접근: Public Internet 에서 직접 접근 불가
- GPU 구성: GPU 인스턴스는 Deep Learning AMI 기반으로 생성하고 Docker 에서 NVIDIA GPU 사용

5.4 현장 조사 및 인터뷰

본 프로젝트 팀은 서비스의 실제 필요성과 사용자 관점을 이해하기 위해 세 차례에 걸쳐 시각장애인 복지관을 방문하였다. 1차 방문(성북시각장애인복지관)에서는 시각장애 체험과 서비스 필요성, 활용 의향에 대한 의견 수집 및 실제 사용자 요구사항 조사를 진행하였고, 2차 방문(성북시각장애인복지관)에서는 개발한 앱을 활용한 사용자 테스트 및 기능 구현의 적절성, 사용 편의성, 아쉬운 점 및 개선사항에 대한 피드백을 수집하였고, 3차 방문(서울시립노원시각장애인복지관)에서는 완성된 앱을 시각장애인 사용자 3명에게 직접 시연하고 실제 사용 경험 기반의 최종 만족도 및 피드백을 수집하였다.

5.4.1 1차, 2차 방문 - 성북시각장애인복지관 (기획 단계 + 외부 사용자 테스트)

서비스 기획 단계에서 시각장애인의 실제 이동 환경과 정보 인식 방식을 직접 체험하고 요구사항을 도출하기 위해 방문하였으며, 이후 완성된 앱을 대상으로 외부 사용자 테스트도 함께 진행하였다.



이론 교육 및 인식 개선 활동

- 정보 인식 경험: 안대 착용 후 청각만으로 강의를 청취하며 시각 정보 의존도 확인
- 보조공학 이해: 점자 및 보조공학 기기 활용법, 시각장애인 안내 에티켓 습득
- 주요 시사점: 이동 지원은 단순한 물리적 배려를 넘어 정확한 상황 정보 제공이 필수적임을 확인

보행 체험 및 시연 테스트

- 흰지팡이와 점자블록에 의존한 실외 보행을 통해 기술적 해결 과제 도출
- 보행의 어려움: 점자블록의 단절, 방향 전환 구간에서의 감지 지연 등 실제 보행 난이도 체감
- 안내 정보의 역할: 장애물 유무뿐 아니라 현재 위치와 진행 방향에 대한 지속적인 피드백이 필수적임

1 차 방문 - 사용자 인터뷰 요구사항 및 반영 사항

구분	주요 요구사항 및 인사이트	본 프로젝트 반영 사항
최우선 가치	접근성보다 '정확도' 우선: 잘못된 안내는 사고 직결 (예: 횡단보도 오안내)	탐지 신뢰도 임계값 보수적 설정, Fail-safe 경고 음성 정책 적용

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

장애물 대응	유도블록 없는 구간 다수, 돌발 장애물 실시간 탐지 기능 필수	YOLOv11s 실시간 탐지 + YOLO seg 보행가능영역 분석으로 구현
알림 방식	단순 경고음보다 음성 안내 + 진동 복합 제공이 가장 효과적	flutter_tts TTS + vibration 진동 패턴 결합 멀티모달 안내 방식 구현
안내 체계	"10 시 방향, 3m 앞" 등 시계 방향+구체적 거리 결합 가이드 선호	bbox 중심 좌표 기반 시계 방향 + 가까움/중간/멀리 거리 추정 적용
UI / UX	뒤로가기 버튼 오작동 방지, 개인별 시력을 고려한 고대비 테마 커스텀 필요	오렌지/남색 고대비, WCAG 4.5:1 이상 명도 대비, 설정 화면 개인화 지원

현장 조사를 통해 시각장애인 보행 보조 서비스는 '정밀한 위치 보정'과 '직관적인 방향 가이드'가 결합되어야 실효성을 가질 수 있음을 확인하였다. 이를 본 프로젝트의 알고리즘 및 인터페이스 설계의 핵심 원칙으로 반영하였다.

2 차 방문 - 시각장애인 대상 테스트 결과

성북시각장애인복지관과 협력하여 저시력 시각장애인 참여형 시나리오 기반 기능 테스트를 진행하였다. 직접 시각장애인 직원분이 시연을 진행하였으며, 장애물 탐지·길찾기 기능의 사용성을 검증하였다.

항목	피드백 유형	세부 내용	개선 방향	반영 여부
정보 제공 방식	개선 필요	- 장애물·방향·주변 객체 등 정보량 과다로 혼란 발생 - 음성이 겹치거나 끊기는 문제 발생 - 불필요한 정보 사례: 건너편 버스, 멀리 있는 표지판 등	- 핵심 위험 요소 중심으로 정보 축소 - 정보의 '양'보다 '정확성' 중심 설계	O
기능 구성	개선 필요	- 장애물 탐지 + 길찾기 동시 제공으로 실제 보행 시 인지 부담 큼 - 지팡이 + 앱 + 길찾기 동시 사용은 현실적으로 어려움	- 기능 선택형 구조 필요 - 장애물 탐지만 - 길찾기만 - 통합 모드	O
음성 안내 UX	개선 필요	- 음성 겹침 및 중간 끊김 발생 - 우선순위 혼재로 정보 전달력 저하	- 음성 우선순위 명확화 - 안내 빈도 조절 - 중요한 정보만 선별 전달	O

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

방향 안내 방식	개선 필요	- '10 시/11 시 방향' 등 세밀한 방향 표현은 체감 차이 적음	- 회피 범위를 더 크게 표현 예: 10 시 장애물 → 12 시 방향으로 회피 안내	O
장애물 탐지 품질	개선 필요	- 안내 지연 발생 (이미 지나간 후 알림) - 불필요한 객체까지 탐지 (원거리 객체 포함)	- 탐지 거리 근거리 중심으로 제한 - 보행 경로 기준 필터링 적용 - 실시간성 개선	O
설정 UI 조작 방식	개선 필요	- 슬라이더 방식은 시각장애인이 정확한 값 입력이 어려운 조작 방식 - 드래그 조작은 TalkBack 환경에서 조작성 낮음	- 안내 문장 길이·재생 속도·진동 강도 항목의 슬라이더를 버튼 방식으로 전환 - 안내 문장 길이: '간결·보통·상세' 3 단계 탭 버튼 - 재생 속도·진동 강도: '-/+' 버튼으로 대체	O
접근성·시스템 충돌	확인 필요	- TalkBack/VoiceOver와의 충돌 가능성 - 통화·음악 사용 시 앱 간섭 우려	- 다양한 상황에서의 동작 테스트 - 접근성 기능과의 호환성 검증	O (확인 결과 문제 X)
GPS 정확도	구조적 한계	- 위치 오차로 잘못된 경로 안내 발생 예: 출구 위치 오류, 불필요한 횡단보도 안내	- GPS 자체 한계이나 사용자 신뢰도에 영향 - SK Tmap 맵매칭 고도화로 보완 검토	-
사용자 설정	긍정 평가	- 속도·볼륨·진동 조절 기능 긍정적 평가	- 색상 및 대비 설정 옵션 추가 제안	

성북복지관 테스트 핵심 개선 과제 (우선순위)

순위	개선 과제	개선 여부
1	불필요한 정보 제거 및 필터링 — 핵심 위험 요소 중심, 근거리 객체 우선 탐지	O
2	음성 안내 우선순위 및 충돌 해결 — 음성 겹침 방지, 중요도 기반 안내 순서 확립	O
3	장애물 탐지 범위 및 정확도 개선 — 탐지 거리 근거리 제한, 보행 경로 기준 필터링	O

4	기능 분리 구조 도입 — 장애물 탐지 전용 / 길찾기 전용 / 통합 모드 선택형 구조	O
5	실시간 처리 성능 개선 — 안내 지연 최소화, 이미 지나간 객체 알림 방지	O
6	설정 UI 접근성 개선 — 슬라이더 → 버튼 방식 전환으로 시각장애인 정확한 조작 가능	O

5.4.2 3 차 방문 - 서울시립노원시각장애인복지관 (앱 완성 후 사용자 테스트)



구분	내용
방문 목적	완성된 길벗 앱의 접근성·사용성 검증, 실제 시각장애인 사용자 피드백 수집, 음성 안내 및 UI 개선 방향 도출
인터뷰 대상	시각장애인 사용자 3인
진행 방식	앱 직접 시연 및 기능별 사용자 반응 수집, 음성 속도 변경(0.5 배·0.75 배·1 배속) 시연 진행

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

UI 및 접근성 관련 피드백

항목	사용자 피드백	향후 계획
색상 대비	검은 배경 + 흰 글씨는 잘 보임. 흰 배경 + 검은 글씨는 잘 보이지 않음. 단순히 글자를 크게 하는 것이 해결책은 아님	고대비 테마 유지 (오렌지/남색), 배경색 선택 기능 향후 검토
글자 크기	시야가 좁은 사용자는 너무 작아도, 너무 커도 한 번에 인식이 어려움. 사용자마다 선호 크기가 다름	사용자 맞춤 글자 크기 조절 기능 우선순위 상향
TalkBack 연동	일부 사용자는 TalkBack 을 항상 사용하지 않음. 음성 속도가 빠르게/느리게 느껴지는 사용자 모두 존재	TalkBack 음성 속도와 앱 내 TTS 속도 연동 기능 검토
UI 직관성	버튼에 기능 설명이 제공되는 점 긍정적 평가. '버튼 버튼 버튼'처럼 의미 없는 UI 가 아닌 점 호평	현행 유지 (접근성 레이블 정책 계속 적용)

길찾기 기능 관련 피드백

항목	사용자 피드백	향후 계획
음성 속도 조절	0.5 배·0.75 배·1 배속 시연에 긍정적 반응. 속도 조절 기능 자체가 다양한 사용자 특성을 고려한 적절한 기능이라는 의견	현행 유지, TalkBack 연동 추가 검토 (개선 우선순위 4 순위)
횡단보도 정보	음향신호기 존재 여부 안내 기능에 매우 긍정적 반응. 작은 횡단보도에 신호등이 없는 경우 판단이 어려워 "그 기능만 있어도 굉장히 도움이 될 것 같다"는 의견	신호등 존재 여부 안내 기능 개선 1 순위로 반영
기존 서비스 한계	실제 위치와 반대 방향 안내, 어느 인도를 걷는지 정확하지 않은 문제 언급. 길벗의 신호등·횡단보도 정보 활용에 대한 기대감 표현	SK Tmap 경로 정확도 개선 및 실제 인도 위치 반영 검토

장애물 탐지 기능 관련 피드백

항목	사용자 피드백	향후 계획
거리 정보 제공	"1 시 방향에 블라드" 방향 안내 외에 대략적 거리(3m·5m·7m 등)도 함께 제공하면 긴급 상황 판단 및 심리적 여유 확보 가능	거리 정보 음성 안내 추가 개선 2 순위로 반영

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

공중 장애물 탐지	흰지팡이는 바닥 장애물 감지 가능하나 파라솔·간판·줄 등 공중 장애물이 오히려 더 위험함. 장애물 높이 구분(하단·중간·머리 높이) 및 "머리 높이 장애물" 유형 안내 필요	공중 장애물 탐지 클래스 추가 및 높이 기반 안내 개선 3 순위로 반영
-----------	--	---

진동 피드백 관련 피드백

항목	사용자 피드백	향후 계획
현행 진동 방식	가까운 장애물일수록 강한 진동, 먼 장애물은 약한 진동 방식에 긍정적 반응	현행 유지
스마트워치 연동	스마트워치·스마트밴드 연동 가능 여부 문의. 장애물 탐지 시 스마트폰을 손에 들고 사용하므로 현재 방식도 충분하다는 반응	향후 웨어러블 연동 기능 장기 로드맵 검토

3 차 방문 종합 인사이트 및 향후 개선 우선순위

순위	개선 항목	사용자 인사이트	향후 계획
1	신호등 존재 여부 안내	실제 보행 중 신호등 유무 판단이 가장 어려운 문제. 작은 횡단보도의 무신호 여부 확인 불가	신호등 탐지 정확도 고도화, 유/무신호 횡단보도 구분 안내 추가
2	장애물 거리 정보 음성 제공	방향 안내만으로는 긴급 상황 판단 어려움. 거리 정보로 심리적 여유와 대응 시간 확보 가능	bbox 기반 거리 추정 고도화 및 음성 안내 문장에 거리 정보 추가
3	공중 장애물 탐지 강화	흰지팡이로 감지 불가한 파라솔·간판·줄 등 공중 장애물이 실제로 더 위험. 높이 구분 안내 필요	탐지 클래스에 공중 장애물 추가, 높이 기반(하단·중간·머리 높이) 유형 안내 구현
4	TalkBack 음성 속도 연동	사용자가 익숙한 TalkBack 속도를 그대로 사용할 수 있도록 연동 요청	TalkBack API 연동 방안 검토
5	사용자 맞춤 접근성 설정	글자 크기·배경색·색상 대비 등 개인별 시야 특성이 다르므로 맞춤 설정 중요	설정 화면에 글자 크기·테마 커스텀 옵션 추가 검토

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

3 차 방문을 통해 완성된 앱에 대한 전반적인 긍정적 평가(UI 직관성, 기능 설명 명확성, 음향신호기 안내 유용성)를 확인하였으며, '실제 보행 중 불안감을 줄여주는 정보' 제공이 단순 기능 추가보다 중요함을 재확인하였다. 도출된 개선 우선순위를 향후 고도화 계획에 반영할 예정이다.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

5.5 멘토링

5.5.1 1차 멘토링

1. 멘토링 당시 구현 상태

- 프론트엔드에서 0.5초 단위로 촬영한 이미지를 서버에 전송하고, YOLO + Segmentation 조합으로 추론 후 결과를 앱에 반환하는 API 통신 구조가 구현 완료된 상태
- AI는 객체 탐지와 보행 영역 추론을 조합하여 위험 여부를 판단하고, 결과를 프론트로 전달
- 안내 문장은 현재 룰베이스(rule-based) 방식으로 생성 중

2. 멘토 피드백 및 질의응답

<사전 질문 질의 응답>

Q1. 실시간성 확보와 모델 최적화 우선순위

질문 내용
"저희는 스마트폰 카메라 이미지가 0.5초 단위로 서버에 들어오고, YOLO + Segmentation 조합으로 추론 중인데 전체 응답 시간이 중요합니다. 정확도를 너무 많이 희생하지 않으면서 속도를 줄이려면 어떤 최적화 우선순위로 접근하는 게 좋을가요? 입력 해상도 축소, 경량 모델 변경, 프레임 스킵, 후처리 단순화 중 무엇부터 보는 게 좋을지 궁금합니다. 또한 VLM 사용은 어떨지, 현재 0.7초 딜레이 개선 방법도 궁금합니다."
멘토 답변
• 현재 latency 80ms는 나쁜 수준은 아니나, 실시간 안전 안내 목적상 10ms 수준을 추천. • 실시간 처리의 기준은 30fps 기준(0.03초/프레임) 수준. • 딜레이 감소를 위한 고려사항: 1. AI 모델 병렬 처리 도입 2. YOLO + Segmentation 두 모델을 사용하는 구조를 재검토하여 하나의 모델로 통합 가능한지 검토 3. Transformer 기반 Segformer는 대신 CNN 기반 Segmentation 모델 사용 권장 4. 모델 양자화(Quantization): 32bit → 8bit 변환으로 속도 및 메모리 최적화 VLM(Vision Language Model)은 모바일 환경에 최적화되어 있지 않아 사용을 권장하지 않음

Q2. 한국 보행 환경 특화 데이터셋 및 후처리 전략

질문 내용
"한국 보행 환경은 점자블록, 공사구간, 좁은 골목처럼 예외가 많아서 세그멘테이션 성능 확보가 어렵습니다. 이럴 때 모델 고도화보다 데이터셋 설계나 후처리 규칙 결합이 더 중요한지 궁금합니다."
멘토 답변
• 한국 보행 환경은 미국 등 해외와 달리 점자블록, 공사구간, 좁은 골목 등 특수한 엣지 케이스가 많음. 이러한 요소들은 일반 모델로는 처리가 어려움. • YOLO는 파인튜닝이 비교적 어려운 반면, Segmentation 모델은 파인튜닝이 수월하므로 CNN 기반 Segmentation 모델을 선택하여 파인튜닝하는 것을 권장. • 모델만으로 해결이 어려운 케이스는 알고리즘 기반 후처리 규칙을 추가로 결합하는 방식이 현실적.

	결과보고서		
	국민대학교	프로젝트 명	길벗
	소프트웨어학부	팀 명	리트리버
	캡스톤 디자인 I	Confidential Restricted	Version 1.0 2026-MAY-14

- 자율주행자동차 개발에서도 유사하게 접근하는 사례가 많으며, 이 프로젝트 역시 같은 방향이 적합.

Q3. 횡단보도 위 차량 통과 시 인식 끊김 문제

질문 내용

"Segmentation을 할 때 횡단보도에서 차가 지나다니면 갑자기 인식이 끊기는 일이 발생하는데, 이런 부분들은 어떻게 처리하면 좋을지 궁금합니다."

멘토 답변

- 이는 Occlusion 문제로, 모델만으로 해결하기 어려운 영역
- 횡단보도와 같이 일시적으로 지면이 가려지는 상황에서는 **별도 조건 처리 로직**을 추가하는 것이 필요.
- 객체 추적(Object Tracking) 기술을 도입하면 인식의 **연속성 유지 가능**
- 자전거 인식 문제(예: 자동차 뒤에 실린 자전거를 90도 각도에서 자전거로 인식하지 못하는 문제)처럼, **알고리즘 단의 예외 처리**가 필요한 경우가 많음.

Q4. AWS 클라우드 사용 관련 조언

질문 내용

"최적화 관련해서 AWS에 올릴 때 조언을 받을만한 것이 있을까요?"

멘토 답변

- 현재 API 통신을 위해 클라우드(AWS EC2)를 사용 중인 것은 이해하나, **핵심 추론 처리를 클라우드에 의존하는 구조는 장기적 서비스 운영 측면에서 재고 권장**.
- 보행 안전 시스템 특성상 네트워크 지연이나 연결 불안정이 발생하면 안정성에 영향이 있을 수 있음.
- 서버 사이즈는 **단계별로 스펙을 올려가며 적절한 크기를 찾는 방식**을 권장.
- 핵심 판단 처리를 **엣지 디바이스**에서 수행하는 방향으로 전환 검토를 권장.

<추가 질문 질의응답>

Q5. 엣지 디바이스 적용 시 디바이스 성능 편차 대응

질문 내용

"엣지 디바이스 적용 시 사용자의 디바이스에 따라 지연 속도가 천차만별일 텐데, 고려해야 할 사항은 무엇인가요?"

멘토 답변

- 디바이스마다 성능이 다르므로, 각 디바이스에 맞는 최적화가 필요.
- 설계 원칙은 **가장 낮은 사양의 하드웨어에서 정상 동작하도록 설계**하면, 그보다 좋은 기기에서는 자연스럽게 잘 동작함.
- 모델을 최대한 작게 만들어 **메모리와 컴퓨팅 자원을 최소화**하는 것이 핵심.
- 모델 양자화(Quantization)**를 적용하면 메모리와 속도 양면에서 효과적이며, 32bit → 8bit 전환 시 속도 약 2배 향상 및 성능 저하도 거의 없음. 4bit, 2bit 변환도 가능하나 아직 연구 단계이며, 성능을 고려하였을 때는 8bit를 권장.
- 엣지 적용 시 STT/TTS 또한 클라우드 의존 없이 **엣지 친화적인 경량 모델**로 구성하는 방향 권장.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

Q6. 안내 문장 생성 방식 개선

질문 내용
"음성 안내 문장을 현재 룰베이스(rule-based)로 생성 중인데 모든 상황에 적합한 문장 생성이 어렵습니다. 해결 방안은 무엇인가요?"
멘토 답변
• 룰베이스 방식은 한계와 제약이 많아 모든 상황에 대응하기 어려움. • 최신 트렌드는 소형 언어 모델(LLM) 또는 Vision Language Model(VLM)을 활용한 생성 방식. • 단, LLM 사이즈가 크고 속도가 느리므로 특정 태스크에 맞게 파인튜닝한 경량 모델 사용이 현실적. ◦ 예: GPT-2(수백 MB 수준), LLaMA 1B 모델 등 오픈소스 소형 모델 적용 가능. ◦ 1B 이하 모델도 특정 태스크 전용으로 파인튜닝하면 스마트폰(4GB RAM 기준)에서 충분히 동작 가능. • Google Colab 환경에서 파인튜닝 실험을 시작하는 것을 권장.

4. 기술 심화 방향 제안

멘토는 다음과 같은 기술 과제에 대해 제시하였다.

기술 과제	내용
2D → 3D 거리 추정	현재는 방향만 제공 중. 거리 추정까지 구현하면 완성도 대폭 향상
객체 추적(Tracking)	딥러닝과 다른 접근이 필요하며, 인식 연속성 확보에 필요
가려짐(Occlusion) 처리	차량 통과 등 일시적 가림 상황 대응 → 알고리즘 처리
한국 환경 특화 데이터셋	점자블록, 공사구간, 횡단보도 등 추가 파인튜닝 필요(현재 구현 중인 기술 과제)
모델 (Quantization)	경량화 시 32bit → 8bit 적용, 엣지 디바이스 최적화

5. 진로 및 연구 관점 조언

- 멘토는 이 프로젝트가 컴퓨터 비전, 엣지 AI, 경량화, 2D→3D 추론, 객체 추적 등 **다양한 연구 주제로 확장 가능**하다고 평가하였다.
- 대학원 진학을 고려하는 경우, **이 프로젝트 자체가 좋은 연구 주제**가 될 수 있으며, 논문 또는 독자적 연구로 이어질 수도 있다고 조언하였다.
- 어플리케이션 기능 구현에만 집중하지 않고, **기술적 난제 해결에 집중**하면 프로젝트 완성도와 연구 가치를 높일 수 있음을 강조하였다.

6. 종합 결론

멘토는 본 프로젝트가 유사 주제 팀 대비 높은 완성도를 보이고 있다고 평가하였다. 주요 제언은 다음과 같다.

1. **실시간성 확보**: 현재 0.5초 전송 구조에서 10ms 수준의 추론 목표로 전환 검토
2. **모델 단순화**: YOLO + Segmentation 이중 구조를 단일 모델로 통합 가능성 검토
3. **엣지 디바이스 전환**: 핵심 추론을 클라우드가 아닌 기기 내부에서 처리하는 구조로 방향 전환 검토
4. **모델 양자화**: 32bit → 8bit 변환으로 속도·메모리 최적화 (성능 저하 미미)

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

5. 기술 심화: Tracking, Occlusion 처리, 2D→3D 거리 추정 등 추후 연구적 가치가 있는 topic 제시



5.5.2 2차 멘토링

1. 멘토링 당시 구현 상태

- 지면 인식 모델(YOLO Segmentation)과 객체 탐지 모델을 분리하여 병렬 또는 순차 실행하는 구조로 전환
- YOLO nano/small 모델 기준 단일 프레임 추론 시간 약 0.08초 수준 측정
- TFLite 변환 및 INT8 양자화 적용 시 약 0.16초 수준 측정 (Colab A100 기준)
- Lite-Mono tiny 모델(파라미터 약 2.2M) 기준 추론 시간 약 0.02초 수준 측정
- 스마트폰 실행 시 객체 탐지 추론 시간이 클라우드 대비 약 0.7~0.8초 수준으로 증가하는 문제 확인
- 카카오 단일 API 사용 시 GPS 오차가 크게 발생하여 카카오+네이버 병행 사용 후 오차가 작은 값을 선택하는 로직으로 개선

2. 멘토 피드백 및 질의응답

<사전 질문 질의 응답>

Q1. 지면 모델과 객체 탐지 모델의 통합 방향

질문 내용

"엣지 디바이스 환경에서 객체 모델과 노면 모델을 완전히 단일화하는 것이 현실적으로 적절한 것일까요? 아니면 2개의 모델을 병렬 또는 순차적으로 사용하는 구조도 충분히 가능한지 궁금합니다."

멘토 답변

- 두 모델을 아키텍처 수준에서 통합하려면 내부 구조를 깊이 분석해야 하므로, **석사 과정 기준으로도 1~2개월이 소요되는 작업임.**
- 남은 기간을 고려하면 단일 통합 모델 구현은 **현실적이지 않으며**, 두 모델을 분리하여 병렬 실행하는 방식을 우선 권장함.
- 병렬 실행이 어렵다면 순차 실행으로 구성하되, **최종 발표에서는 통합 실패가 아닌 현실적인 시스템 구조 선택으로 설명하는 것이 적절함.**

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

Q2. 단일화 시도의 적정 수준

질문 내용
"단일 모델 구성이 어렵다면 어느 수준까지 단일화를 시도하는 것이 적절할지 궁금합니다."
멘토 답변
- 객체 탐지(Object Detection)와 지면 인식(Semantic Segmentation)은 태스크 자체가 달라 통합이 쉽지 않음. - 현재 일정에서는無理하게 통합을 시도하기보다, 여름 방학 이후 시간적 여유가 생겼을 때 재 시도하는 것이 좋음.

Q3. 거리 추정 모델 추가 시 연산량 및 실시간성

질문 내용
"지면 모델과 객체 탐지 모델에 2D→3D 거리 추정 모델까지 추가했을 때 전체 연산량과 실시간성을 고려하면 구조적으로 무리가 없는지 궁금합니다."
멘토 답변
- CNN 기반 거리 추정 모델이라면 전체 연산량과 실시간성에 큰 문제 없음. - Transformer 기반 모델은 연산 부담이 크므로 주의가 필요함. - 거리 추정 기능은 반드시 포함하는 것이 바람직함.

Q4. 엣지 환경에서의 우선순위

질문 내용
"엣지 환경에서 실제 서비스 수준으로 가져가기 위해 우선순위를 뒤야 할 부분이 무엇이 있을까요?"
멘토 답변
- 엣지 디바이스 적용이 현실적으로 어렵다고 판단된 만큼, 최소 기능이 동작하는 파이프라인을 클라우드 환경에서 먼저 완성하는 것이 우선임. - 전체 시스템이 동작함을 확인한 후, 병목이 발생하는 부분을 파악하고 순차적으로 최적화하는 방향이 적절함.

<추가 질문 질의응답>

Q5. 추론 속도 측정 및 발표 방식

질문 내용
"입력 해상도를 2개로 나눠서 측정했습니다. 960이랑 640이랑 수치 차이가 별로 없어서 정리해보았는데, 이 정도 차이면 의미 있는 차이로 봐도 되는지, 그리고 발표할 때 이 수치를 어떻게 제시하면 좋을지 궁금합니다."
멘토 답변
- 실제 보행 보조 서비스는 카메라 프레임을 한 장씩 처리하므로 FPS보다 단일 프레임 기준 latency(ms/frame)로 제시하는 것이 더 타당함. - 평균값만 제시하면 특정 프레임에서 시간이 튀는 경우 왜곡될 수 있으므로, median latency, 변동 폭, outlier 여부를 함께 측정할 것을 권장. preprocessing / inference / postprocessing 시간을 분리하여 측정해야 함. 960, 640 외에 320 해상도도 실험하여 정확도와 latency를 비교할 것을 제안함. 640과 960의 차이가 측정 오차 범위 안에 있을 수 있으므로, 단일 결과만으로 속도 개선은 단정하지 말고 100장 이상 프레임 기준으로 반복 측정할 것을 권장함.

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

Q6. TFLite 변환 및 엣지 디바이스 실행

질문 내용
"TFLite로 변환할 때 전체 이미지 말고 crop한 사진도 같이 넣어봤습니다. 지면 이미지는 하단 절반만 봐도 될 것 같아서 해봤는데, 이 방향이 맞는지 궁금합니다. 그리고 지금 Colab A100으로 돌렸을 때 INT8 기준으로 0.16초 정도 나왔는데 이 수치를 발표에서 써도 될지 궁금합니다."
멘토 답변
- Colab A100 기준 속도와 실제 스마트폰에서의 속도는 완전히 다르므로, A100 기준 수치만 제시하면 설득력이 떨어질 수 있음. 실제 스마트폰에서의 속도 제시 필요. - 엣지 서비스를 목표로 한다면 Android 스마트폰 기준 실험 수치를 최소한 일부라도 확보해야 함. - TFLite는 Android 환경에서 실행 가능하므로 직접 스마트폰 테스트를 권장. - 현재 당장 최적화가 어렵다면 "클라우드에서는 동작하고, 엣지에서는 향후 최적화 예정"이라는 구조로 설명하는 것이 현실적임.

Q7. 거리 추정 모델 성능 문제

질문 내용
"3미터 정도 거리를 판단하는 게 중요한데, 인터넷에 있는 데이터셋이 대부분 차량에서 찍은 이미지라 3미터 거리에 맞는 데이터가 별로 없습니다. 그래서 그 거리 객체를 잘 탐지 못하고 있는데, FastDepth 써봤더니 나무 기둥 같은 건 잡는데 사람이나 차는 형체를 못 잡고, Lite-Mono는 바로 쓰면 성능이 잘 안 나오는 상황입니다. 모델 규모를 키우면 해결되는지, 아니면 다른 방법이 있는지 궁금합니다."
멘토 답변
- 거리 추정 모델은 tiny 모델만 고집할 필요 없으며, 스마트폰 기준 수십 MB~100MB 안팎 모델도 최적화 시 실행 가능하므로 정확도 확보를 위해 모델 크기 확대를 고려할 수 있음. end-to-end workflow가 먼저 자연스럽게 동작하는 것이 우선. - 최종 발표에서는 "현재는 클라우드 환경에서 전체 workflow 구현, 엣지는 개발 중"이라고 설명 가능.

Q8. 스마트폰 기준 디바이스 설정

질문 내용
"스마트폰마다 사양이 다 다른데, 기기를 하나 정해두고 그 사양에 맞춰서 개발을 진행하면 되는지 궁금합니다."
멘토 답변
- 모든 스마트폰을 대상으로 최적화할 필요 없이, 기준 기기 하나를 정하는 것이 합리적임. - 출시 후 약 2년 된 Android 기기를 기준으로 삼으면, 해당 기기에서 동작할 경우 최신 기기에서는 더 잘 동작할 가능성이 높음. - iOS는 동시에 고려하기보다 Android + TFLite 기반으로 좁히는 것을 권장.

Q9. GPS 오차 및 역지오코딩 문제

질문 내용
"GPS 오차 때문에 실제 위치가 아닌 건너편으로 인식돼서 불필요한 횡단보도 이동이 안내되는 문제가 생기고 있습니다. 처음에는 카카오 단일 API를 썼는데 오차가 너무 커서, 지금은 카카오랑 네이버를 같

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

이 쓰고 돌 중에 오차가 적은 값으로 전달하는 로직을 만들었습니다. GPS 오차를 더 줄일 수 있는 방법이 있는지 궁금합니다."

멘토 답변

- 한국에서는 Google Maps의 위치 정확도가 제한적이므로(약 10m 이상 오차 가능), **카카오맵**이나 **네이버지도**를 활용하는 것이 더 적합함.
- 현재 **카카오+네이버 병행 사용 후 오차가 작은 값을 선택하는 로직은 올바른 방향임.**
- GPS 오차 자체는 스마트폰, 통신사, 지도 서비스, 주변 환경에 영향을 받으므로 팀이 완전히 해결하기는 어려움.
- 완전한 해결보다는 **카카오/네이버/구글 지도 API의 공식 정확도 스펙을 근거 자료로 제시하고, 오차 사례 및 개선 방향을 함께 설명하는 것이 심사에서 더 설득력 있음.**

4. 기술 심화 방향 제안

멘토는 다음과 같은 기술 과제에 대해 제시하였다.

기술 과제	내용
모델 분리 실행 구조	지면 인식(Segmentation)과 객체 탐지(Detection)는 태스크 특성이 달라 단기 통합이 어려우므로, 병렬 또는 순차 실행 구조로 구성
추론 속도 측정 방식 개선	FPS 대신 단일 프레임 기준 ms/frame으로 측정하며, preprocessing / inference / postprocessing 시간을 분리하여 median latency 및 변동 폭 함께 제시
입력 해상도 실험	960, 640 외 320 해상도까지 실험하여 정확도와 latency 비교. 100장 이상 반복 측정으로 통계적 유의성 확보
거리 추정 모델 확장	tiny 모델에 한정하지 않고, 스마트폰 기준 수십 MB~100MB 안팎 모델까지 고려하여 3m 이내 거리 추정 정확도 확보
개발 디바이스 기준 설정	출시 후 약 2년 된 Android 기기 하나를 기준으로 삼아 최적화 진행
클라우드 기반 end-to-end 데모	엣지 최적화 전까지는 클라우드(A100 등) 환경에서 전체 workflow가 동작함을 시연하고, 엣지에서의 latency 한계와 향후 개선 계획을 수치 기반으로 제시

5. 진로 및 연구 관점 조언

- 멘토는 현재 프로젝트가 사회적 의미와 기술적 의미를 모두 갖추고 있다고 평가하였다.
- 현재 엣지 디바이스에서의 완전한 경량화 구현은 현실적으로 어렵더라도, Colab A100 기반으로 두 모델을 실행하여 실시간에 가깝게 동작하는 데모를 완성하는 것이 우선이라고 조언하였다.
- 심사 과정에서 엣지 디바이스 미적용에 대한 질문이 들어올 경우, 현재 개발 중이며 엣지 디바이스에서의 latency 및 accuracy 문제를 인지하고 있고 향후 개선할 예정이라고 답하면 충분히 납득시킬 수 있다고 하였다.

6. 종합 결론

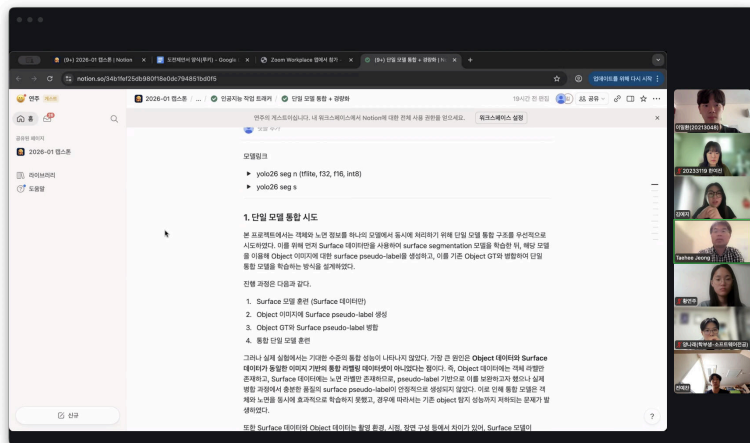
멘토는 남은 약 2주 기간 동안 모든 기능을 완벽하게 구현하려 하기보다, 전체 workflow가 동작하는 것을 명확히 보여주고 한계와 향후 개선 방향을 근거 있게 제시하는 것이 중요하다고 강조하였다. 주요

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

제언은 다음과 같다.

- **모델 분리 실행:** 지면 인식 모델과 객체 탐지 모델은 분리하여 병렬 또는 순차 실행으로 구성
- **속도 지표 개선:** FPS 대신 단일 프레임 기준 ms/frame으로 측정하고, preprocessing / inference / postprocessing 분리 측정
- **적절한 통계 기준으로 수치 확보:** Colab A100 수치만으로는 설득력 부족하므로 Android 스마트폰 기준 실험 수치를 최소한 일부라도 제시
- **클라우드 기반 데모:** A100 환경에서 end-to-end workflow 전체가 동작함을 시연
- **GPS/역지오코딩 한계 설명:** API 공식 정확도 스펙과 실험 사례를 근거로 제시하고 향후 개선 방향 언급

향후 계획: TFLite 최적화, INT8 양자화, 입력 해상도 조정, Android 기준 기기 최적화 등을 개선 방향으로 제시



 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

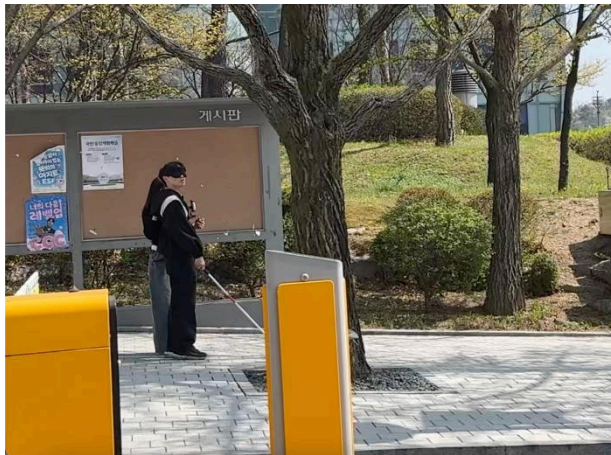
5.6 테스트 케이스

대분류	소분류	기능	테스트 방법	기대 결과	테스트 결과
장애물 탐지	실시간 탐지	카메라로 전방 장애물 실시간 탐지	카메라를 전방 향하게 하여 사람·자전거·킥보드 등 장애물 앞 1~3m 위치	화면에 bbox 표시 및 "12 시 방향에 사람이 있습니다" 음성 출력	성공
	회피 방향 안내	탐지 장애물에 대한 최적 방향 안내	정면에 장애물 배치 후 탐지 실행	"오른쪽으로 이동하세요" 등 시계 방향 기반 안내 문장 음성 출력	성공
신호등 인식	신호 상태 인식	보행자 신호등 색상 인식 및 안내	신호등 앞에서 카메라로 신호등 촬영	"초록불입니다. 건너도 됩니다." / "빨간불입니다. 멈추세요." 음성 출력	성공
길찾기	경로 탐색	목적지 입력 후 경로 안내	음성으로 목적지 입력 후 안내 시작 버튼 탭	SK Tmap 보행자 경로 계산 후 구간별 방향 TTS 안내 출력	성공
	경로 재탐색	경로 이탈 시 자동 재탐색	경로 이탈 후 대기	자동 재탐색 후 새 경로 안내 시작	성공
인증	카카오 로그인	카카오 소셜 로그인 및 JWT 발급	앱 실행 후 카카오 로그인 버튼 탭	카카오 인증 완료 후 JWT 발급, 메인 화면 진입	성공
	토큰 재발급	Refresh Token 기반 자동 재발급	Access Token 만료 상태에서 API 호출	Refresh Token 으로 Access Token 자동 재발급 후 정상 응답	성공
설정	안내 설정 변경	음성/진동 설정 개인화	설정 화면에서 진동 강도·음성 속도·문장 길이 변경	변경 설정 즉시 적용 후 이후 안내에 반영	성공
접근성	TalkBack 지원	Android TalkBack 화면 리더 지원	Android TalkBack 활성화 후 앱 사용	모든 UI 요소에 콘텐츠 설명 레이블 음성 안내	성공

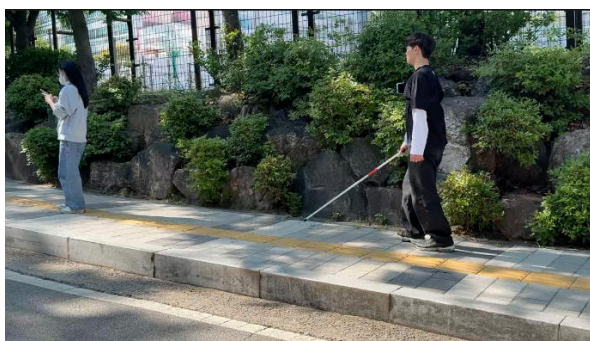
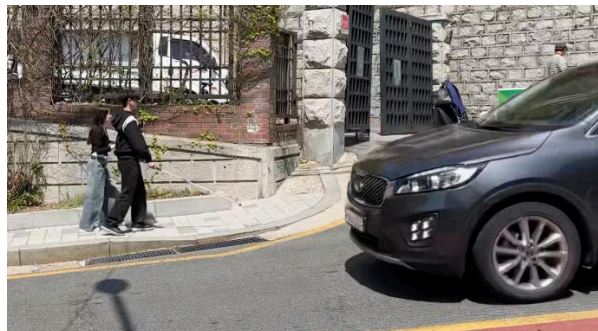
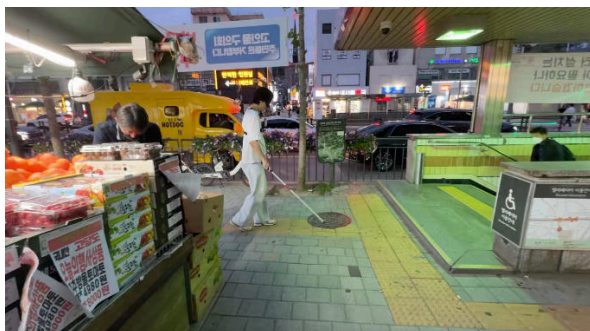
 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

	고대비 UI	저시력자 명도 대비 기준 충족	UI 색상 대비 측정 (오렌지-남색)	텍스트-배경 명도 대비 4.5:1 이상 (WCAG AA 기준) 충족	성공
	안대 착용 내부 테스트	시각 정보 차단 상태 사용성 검증	팀원 안대 착용 후 장애물 탐지·길찾기 시나리오 수행	음성·진동만으로 기능 동작 확인	성공
시스템	네트워크 안정성	네트워크 불안정 시 비정상 종료 없음	네트워크 차단 후 앱 실행 및 기능 사용	앱 비정상 종료 없이 기본 안내 메시지 제공	성공
AI 서버	이미지 전송 주기	0.5 초 주기 이미지 전송 및 추론 응답	앱에서 0.5 초마다 JPEG 이미지 전송	AI 서버 수신 후 0.4 초 이내 탐지 결과 WebSocket 으로 앱 수신	성공

* 테스트 활동 사진



 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14



 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

5.7 백엔드 API 명세서

프론트엔드와 백엔드 간 통신 규격 문서이다.

Swagger UI(<https://gilbeot.p-e.kr/swagger-ui/index.html>)에서 실시간 확인 및 테스트 가능하다.

기능	Method	URL	설명
카카오 로그인	POST	/api/auth/kakao	카카오 OAuth2.0 로그인 및 JWT 발급
토큰 재발급	POST	/api/auth/reissue	Refresh Token → Access Token 재발급
로그아웃	POST	/api/auth/logout	Refresh Token 삭제
내 프로필 조회	GET	/api/users/me	로그인 사용자 정보 조회
회원 탈퇴	DELETE	/api/users/me	사용자 계정 삭제
설정 조회	GET	/api/users/settings	사용자 개인화 설정 조회
설정 변경	PATCH	/api/users/settings	안내 스타일, 진동 강도 설정 수정
장소 검색	GET	/api/places/search	키워드로 장소 검색 (Kakao Local)
근처 장소 목록	GET	/api/places/nearby	현재 위치 기준 주변 장소 조회
지오코딩	GET	/api/places/geocode	주소 → 좌표 변환

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

역지오코딩	GET	/api/places/reverse-geocode	좌표 → 주소 변환
즐거찾기 목록	GET	/api/favorites	즐거찾기 목록 조회
즐거찾기 추가	POST	/api/favorites	장소 즐거찾기 등록
즐거찾기 삭제	DELETE	/api/favorites/{favoriteId}	특정 즐거찾기 삭제
최근 검색 목록	GET	/api/recent-searches	최근 검색 리스트 조회
최근 검색 삭제	DELETE	/api/recent-searches/{id}	특정 검색 기록 삭제
경로 계산	GET	/api/navigation/route	SK Tmap 보행자 경로 탐색 및 안내 텍스트 추출
이미지 전송	POST	/api/guide/image	앱 → 백엔드 이미지 업로드 (0.5 초 주기)
AI 이벤트 수신	POST	/api/guide/event	AI 서버 → 백엔드 탐지 이벤트 전달
이벤트 Push	WebSocket	-	백엔드 → 앱 실시간 탐지 결과 전송 (STOMP)
날씨 조회	GET	/api/weather?lat={lat}&lon={lon}	OpenWeatherM ap API 날씨 정보 조회
근처 횡단보도	GET	/api/crosswalks/nearby?latitude={lat}&longitude={lon}&radius Meters={m}	현재 위치 기준 주변 횡단보도

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

조회			조회, 거리순 정렬, 좌표 없는 데이터는 제외 가능
횡단보도 단일 조회	GET	/api/crosswalks/{crosswalkCode}	특정 횡단보도 상세 정보 조회, 매칭된 음성신호기 정보 포함

5.8 데이터셋 상세 클래스 정보

본 프로젝트에서 활용한 AIHub 인도 보행 영상 데이터셋의 객체 클래스 및 노면 클래스 상세 구성이다.

장애물 탐지용 객체 클래스

분류	객체 클래스	비고
이동체	· person (사람) · car (승용차, 승합차) · bus (버스) · truck (트럭) · bicycle (자전거) · motorcycle (오토바이, 스쿠터) · stroller (유모차) · wheelchair (휠체어) · dog (개) · cat (고양이) · carrier (리어카, 손수레) · movable_signage (이동식 홍보 전시물/안내판) · scooter (변속 기어 없는 이륜차, 두 발로 탈 수 있는 기구)	보행 환경에서 이동하는 모든 객체 포함 AIHub 인도 보행 영상 bbox 데이터 활용
고정체	· tree_trunk (가로수 기둥) · potted_plant (화분) · traffic_light (신호등) · traffic_sign (교통 표지판) · pole (대/기둥) · bench (2 인 이상 앉을 수 있는 기구) · chair (1 인이 앉을 수 있는 기구) · table (탁자) · stop (버스/택시 정류장) · kiosk (한쪽이 열린 점포) · fire_hydrant (소화전) · parking_meter (주차요금정산기) · bollard (볼라드) · barricade (바리케이드) · power_controller (전력제어함) · traffic_light_controller (신호등제어기)	보행 중 마주치는 고정 장애물 포함 볼라드·바리케이드 등 보행 위험 요소 중점 학습

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

보행 환경 분석용 노면 클래스 및 속성

노면 클래스	속성	비고
Sidewalk (인도)	· blocks (보도블록) · cement (시멘트) · urethane (우레탄) · asphalt (아스팔트) · soil_stone (흙, 돌, 비포장도로) · damaged (인도의 파손) · other (정의된 인도 속성 외의 경우)	보행 가능 영역 핵심 클래스
braille_guiding_block (점자블록)	· normal · damaged (점자블록 파손)	시각장애인 보행 유도 핵심 클래스 mask mAP50 0.821 달성
roadway (차도)	· normal · crosswalk (횡단보도)	차도 영역 구분으로 보행 위험 판단
alley (이면도로)	· normal · crosswalk (횡단보도) · speed_bump (과속방지턱) · damaged (이면도로 파손)	
bike_lane (자전거도로)	(해당없음)	
caution_zone (주의구역)	· stairs (계단) · manhole (맨홀) · grating (그레이팅) · repair_zone (보수구역) · tree_zone (가로수 한 그루 식재를 위한 바닥면 구역)	낙상·충돌 위험 구역 Fail-safe 경고 음성 정책 적용

위 데이터셋을 기반으로 보행자 전방의 장애물 탐지뿐 아니라 노면 상태와 주의 구역까지 함께 인식하여, 보다 안전한 보행 보조 인공지능 시스템을 구현하였다.

5.9 AI 안내 판단 기준

본 시스템은 객체 탐지 결과만을 기준으로 안내하지 않고, segmentation 기반 보행 가능 영역, 차도 영역, 점자블록 영역과의 overlap 을 함께 고려하여 안내 대상을 선별한다. 점자블록 위 장애물, 점자블록 가림, 점자블록 끊김은 보행 안전에 직접적인 영향을 주므로 일반 객체보다 우선적으로 안내한다. 차량류 객체는 동적 위험도가 높지만, 차도 위 객체까지 모두 안내하면 불필요한 경고가 증가하므로 보행 경로 침범 여부, 접근 속도, 거리, 화면상 크기를 함께 고려한다. 또한 동일한 안내 문장이 짧은 시간 내 반복되지 않도록 10 초 쿨다운을 적용하여 사용자 피로도를 줄인다.

1. 공통 판단 기준

판단 요소	기준	의미
distance	near / mid / far	객체와 사용자 사이의 거리 구간
clock_direction	9 시 ~ 3 시	사용자 기준 객체 위치
motion	static / moved	객체가 정지 상태인지 이동 중인지
on_path	path_overlap ≥ 0.10 and road_overlap < 0.50	보행 가능 영역 위에 있는지
blocks_tactile	tactile_overlap ≥ 0.05	점자블록 위에 객체가 있는지
road_overlap	bbox 하단부와 road_mask 겹침 비율	객체가 차도 위에 있는지
approach_speed	이전 프레임 대비 거리 감소량	객체가 사용자 쪽으로 접근 중인지
priority	1 / 2 / 3 / 4	숫자가 작을수록 위험도가 높음
voice_guide 동일성	같은 문장 10 초 이내 반복 여부	동일 안내 반복 방지

2. 점자블록 판단 기준

점자블록 관련 위험은 가장 우선적으로 판단한다. 일반 객체보다 먼저 안내 대상으로 선정하며, 점자블록 위 객체, 점자블록 가림, 점자블록 끊김을 모두 caution 위험으로 처리한다.

상황	세부 조건	액션	Priority	안내 여부
점자블록 위 객체 감지	blocks_tactile=True	caution	2	안내

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

점자블록 가림 / 중간 occlusion	tactile_status="blocked_or_occluded"	caution	2	안내
점자블록 끊김	tactile_status="broken"	caution	2	안내
점자블록 정상 연결	tactile_status="clear"	go	4	안내 없음
점자블록 없음	tactile_status="no_tactile"	go	4	안내 없음
동일 점자블록 끊김 문장 반복	동일 voice_guide 가 10 초 이내 반복	기존 action 유지, 음성만 제거	기존 priority 유지	silent
문장 구성이 다른 점자블록 안내	예: "전방에 점자블록이 있습니다. 주의..."	기존 action 유지	기존 priority 유지	안내 가능

점자블록 판단 예시

입력 상황	기대 출력
점자블록 위에 블라드가 있음	"주의. 12 시 방향 점자블록 위 블라드, 우회바람."
점자블록이 사람/장애물에 가려져 중간이 비어 있음	"주의. 12 시 점자블록 가림, 오른쪽 우회."
점자블록이 전방에서 끊김	"주의. 전방 점자블록 끊김, 천천히 이동."
동일한 끊김 문장이 10 초 이내 반복됨	음성 안내 없음

3. 정적 장애물 판단 기준

대상 예시: pole, bollard, bench, barricade, tree_trunk, traffic_sign, movable_signage, kiosk, chair, table

정적 장애물은 보행 경로 위에 있는지가 핵심 기준이다. 보행 경로 밖에 있으면 대부분 안내하지 않거나 약한 주의로 처리하고, 보행 경로 위에 있으면 우회 안내를 우선한다.

상황	세부 조건	액션	Priority	안내 여부
처음 등장, 보행 경로 밖	on_path=False, distance=mid/far	go 또는 caution	3~4	대부분 안내 없음
보행 경로 위 정적 장애물	on_path=True	detour	2	안내
중앙 + 가까움 + 우회 가능	clock=12 시, distance=near, safe zone 존재	detour	2	안내
중앙 + 가까움 + 우회 불가	clock=12 시, distance=near, safe zone 없음	caution	2	안내
매우 가까운 정적 장애물	distance < URGENT_DISTANCE_M	caution 또는 stop 검토	1~2	안내
같은 상태 지속	동일 객체, 동일 위험 상태	silent	기존 priority	안내 생략
위험 상태 상승	off_path->on_path, mid->near	caution 또는 detour	상승된 priority	재안내

정적 장애물 판단 예시

입력 상황	기대 출력
왼쪽 벽 옆 블라드, 보행 경로 밖	안내 없음
12 시 방향 보행 경로 위 블라드	"주의. 12 시 블라드, 오른쪽 우회."
12 시 방향 장애물, 좌우 모두 위험	"주의. 12 시 방향에 블라드가 있습니다."
이전에는 경로 밖이었지만 현재 경로 위로 들어옴	재안내

4. 사람 판단 기준

현재 구현 기준에서는 일반 보행 경로 안내 후보에서 사람은 기본적으로 제외하는 정책이 적용된다. 다만 점자블록 위에 있는 사람은 blocks_tactile=True 라면 점자블록 방해 객체로 안내될 수 있다.

상황	세부 조건	액션	Priority	안내 여부
멀리 있는 사람	distance=far, on_path=False	go	4	안내 없음
경로 밖에서 움직이는 사람	motion=moved, on_path=False	go 또는 caution	3~4	기본 안내 없음
점자블록 위 사람	blocks_tactile=True	caution	2	안내
보행 경로 위 가까운 사람	on_path=True, distance=near	정책 선택 필요	2~3	현재는 일반 안내 제외
중앙 + 매우 가까운 사람	clock=12 시, distance=near	caution 또는 stop 검토	1~2	향후 개선 가능
같은 상태 지속	동일 사람, 동일 위치	silent	기존 priority	안내 생략
위험 상태 상승	left/right->center, off_path->on_path, mid->near	caution 또는 detour	상승된 priority	재안내 후보

5. 차량·오토바이·스쿠터·자전거 판단 기준

대상 예시: car, bus, truck, motorcycle, scooter, bicycle

차량류는 동적 위험도가 높지만, 차도 위에 있는 차량까지 모두 안내하면 오탐성 알림이 많아진다. 따라서 road_overlap 이 높고 on_path=False 인 차량은 기본적으로 후보에서 제외하고, 가까움/접근/화면상 큼/보행 경로 침범이 있을 때만 안내한다.

상황	세부 조건	액션	Priority	안내 여부
차도 위 차량, 보행 경로 밖	road_overlap >= 0.50, on_path=False	go	4	안내 없음
보행 경로 위 차량	on_path=True	stop 우선	1	안내
가까운 차량	distance=near	caution 또는 stop	1~2	안내
움직이는 차량	motion=moved, distance=mid/near	caution	2	안내

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

사용자 쪽 접근	approach_speed > threshold	stop	1	안내
화면상 크게 보이는 차량	bbox_area 큼	stop 또는 caution	1~2	안내
우회 방향 불명확	safe zone 없음	stop	1	안내
같은 상태 지속	동일 차량, 동일 위험 상태	silent	기존 priority	안내 생략
위험 상태 상승	far->mid, mid->near, off_path->on_path, center 진입	caution 또는 stop	상승된 priority	재안내

차량 판단 예시

입력 상황	기대 출력
오른쪽 차도 위 차량이 지나감	안내 없음
보행 경로 위로 차량이 진입	"정지! 12 시 차량."
멀리 있던 차량이 가까워짐	재안내
차량이 사용자 쪽으로 빠르게 접근	"정지! 12 시 방향에서 차량이 빠르게 다가오고 있습니다."

6. 최종 액션 기준표

Action	의미	사용 조건
stop	즉시 정지 필요	가까운 동적 객체, 접근 차량, 빨간불, 우회 불가 위험
detour	우회 필요	보행 경로 위 장애물이 있고 안전한 우회 방향이 있을 때
caution	주의 필요	점자블록 위험, 움직이는 객체, 애매한 위험
go	진행 가능	경로상 위험 없음
silent	음성 안내 생략	위험 상태는 유지되지만 이미 안내했거나 동일 문장 쿨다운 중

7. 우선순위 기준표

Priority	위험도	대표 상황	Action
1	매우 높음	접근 차량, 가까운 차량, 빨간불, 정지 필요	stop
2	높음	점자블록 위 장애물, 보행 경로 위 장애물, 우회 필요	caution, detour
3	낮음	파란불, 장면 정보, 약한 주의	go, caution
4	없음	경로상 위험 없음	go, silent

8. 재안내 조건표

아래 조건 중 하나라도 발생하면 silent 상태에서 다시 안내할 수 있다.

변화 유형	재안내 조건	예시
거리 위험 증가	far -> mid, mid -> near	멀리 있던 차량이 가까워짐
움직임 위험 증가	static -> moved	정지해 있던 자전거가 이동 시작
경로 위험 증가	off_path -> on_path	보행 경로 밖 장애물이 경로 안으로 들어옴
방향 위험 증가	left/right -> center	측면 객체가 12 시 방향으로 이동
우선순위 상승	3 -> 2, 2 -> 1	주의에서 정지로 상승
접근 속도 증가	approach_speed threshold 초과	차량이 빠르게 접근
점자블록 상태 악화	clear -> blocked_or_occluded, clear -> broken	점자블록 가림 또는 끊김 발생
일정 시간 후 재등장	exit_ttl 초과 후 재탐지	사라졌던 장애물이 다시 등장
동일 문장 쿨다운 종료	동일 voice_guide 발생 후 10 초 경과	같은 점자블록 끊김 안내 재허용

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

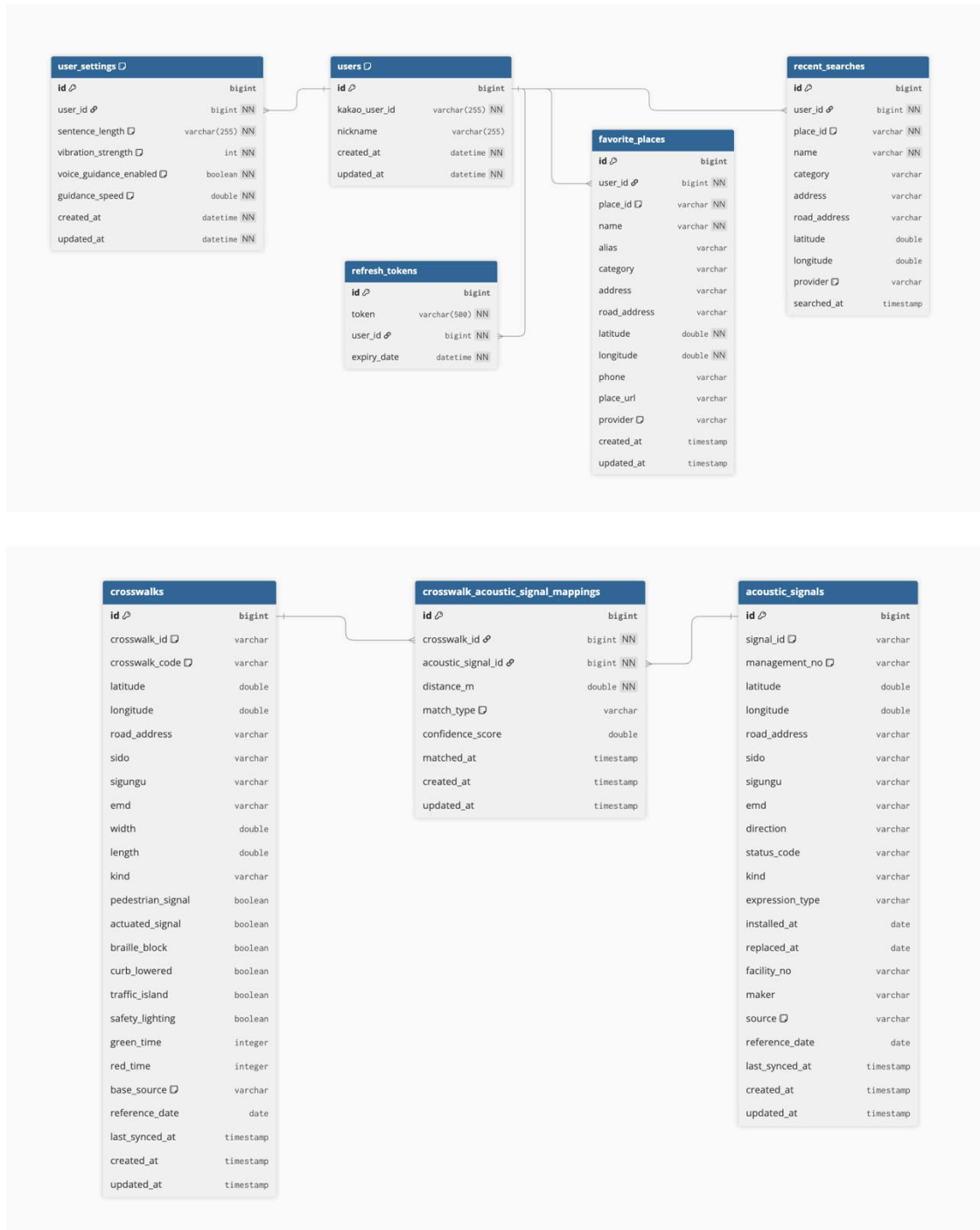
9. 동일 안내문 쿨다운 기준

이전 안내	현재 안내	10 초 이내 처리
주의. 전방 점자블록 끊김, 천천히 이동.	주의. 전방 점자블록 끊김, 천천히 이동.	silent
주의. 전방 점자블록 끊김, 천천히 이동.	전방에 점자블록이 있습니다. 주의. 전방 점자블록 끊김, 천천히 이동.	안내
주의. 12 시 차량.	정지! 12 시 차량.	안내
주의. 12 시 블라드, 오른쪽 우회.	주의. 12 시 블라드, 오른쪽 우회.	silent

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

5.10 DB 스키마

Amazon RDS(MySQL)에 구축된 데이터베이스의 테이블 구조이다. 모든 테이블은 백엔드 서버(Spring Boot)에서만 접근 가능하도록 Private Subnet에 배치하여 보안성을 확보하였다.



 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

5.11 개발 환경 상세

통합 개발 환경

항목	내용	비고
회의록·문서 공유	Notion / Slack	회의록 및 문서 공유
문서 작업	Google Docs, Figma	문서 작업 협업
버전 관리	Git / GitHub	협업 및 코드 관리

프론트엔드 개발 환경

항목	내용	비고
운영체제	macOS / Windows	개발 환경
컴파일 환경	Flutter SDK 3.35.5, Android SDK 36.1.0, Gradle 8.12	모바일 빌드
개발 언어	Dart 3.9.2 이상, null-safety 기반	Flutter 개발
프레임워크	Flutter	크로스플랫폼 UI
IDE	Android Studio / VSCode	개발 도구
UI 디자인	Figma	프로토타이핑
주요 라이브러리	speech_to_text, flutter_tts, camera, geolocator, http, vibration, stomp_dart_client, kakao_flutter_sdk_user, flutter_secure_storage	
접근성 기능	TTS, 진동, 큰 버튼 UI, WCAG 가이드라인 적용	시각장애인 대상
통신 방식	REST HTTP / WebSocket (STOMP)	서버 연동 프로토콜
인증 방식	카카오 OAuth2.0 + JWT	로그인 및 토큰 관리
빌드 타겟	Android / iOS	지원 플랫폼

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

백엔드 개발 환경

항목	내용	비고
운영체제	macOS / Windows	개발 환경
컴파일 환경	JDK 21, Gradle	
개발 언어	Java 21	Spring Boot
프레임워크	Spring Boot	백엔드 프레임워크
IDE	IntelliJ IDEA	개발 도구
실시간 통신	WebSocket (STOMP)	AI 분석 결과 클라이언트 전달
지도 서비스	SK OpenAPI (Tmap)	보행자 경로 탐색
위치 서비스	GPS / Kakao Locals, Naver API	사용자 위치 정보
날씨 서비스	OpenWeatherMap API	날씨 정보
공공 정보	공공 데이터 포털, 서울 열린데이터 광장	횡단보도 및 음성신호기 정보
서버 구축	AWS EC2, AWS ECR, AWS ALB, AWS ASG, Amazon ECR, AWS SSM, AWS VPC, Subnet	클라우드 배포
컨테이너	Docker + Nginx	배포 환경 일관성
데이터베이스	MySQL (Amazon RDS)	데이터 저장
ORM	Spring Data JPA	DB 접근
인증 방식	카카오 OAuth2.0 + JWT	로그인 및 토큰 관리
API 문서	Swagger (Springdoc OpenAPI)	API 테스트 UI

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

인공지능 개발 환경

항목	내용	비고
운영체제	macOS / Windows	개발 환경
컴파일 환경	VSCode / Google Colab	
개발 언어	Python 3.10 이상	AI 모델 학습 및 추론
딥러닝 프레임워크	PyTorch	객체 탐지 모델 학습
객체 탐지 모델	YOLOv11s (Ultralytics)	최종 선정 모델
세그멘테이션 모델	YOLO26 Seg	기존 SegFormer 에서 변경
주요 라이브러리	torch, torchvision, ultralytics, transformers, opencv-python, numpy, pandas	
학습 환경	Google Colab / Local GPU / CUDA	모델 파인튜닝
데이터셋	AIHub 인도 보행 영상 데이터셋, Roboflow pedestrian traffic light	
AI 서버	FastAPI (Python 3.10+), AWS EC2 Private Subnet	추론 서버

실행 환경

구분	항목	내용
모바일 앱	운영체제	Android 9 이상, iOS 13.0 이상
	하드웨어	카메라 및 GPS 센서가 있는 스마트폰
	필수 라이브러리	flutter_tts, speech_to_text, vibration
백엔드 서버	운영체제	Ubuntu 22.04 LTS (AWS EC2 t3.micro)

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

	필수 소프트웨어	Docker, OpenJDK 21, Gradle, Git, Nginx
	필수 라이브러리	Spring Boot, Spring Web, Spring Data JPA, Spring Security, JWT, Lombok
AI 서버	필수 소프트웨어	Python 3.10 이상, FastAPI, Docker
	필수 라이브러리	PyTorch, ultralytics, opencv-python, numpy
	입력 데이터	스마트폰 카메라 영상 이미지 (JPEG 압축)

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

5.12 Git 협업 방식

Git 워크플로우 설계

브랜치 구조

master



1. 각 파트(front/back/ai)는 독립적으로 develop 브랜치를 사용한다.
2. 모든 작업은 해당 develop 브랜치에서 feature 브랜치를 생성하여 진행한다.
3. 작업 완료 후, feature 브랜치는 각 develop 브랜치로 merge한다.
4. 최종적으로 develop 브랜치들을 master로 merge하여 배포한다.

브랜치 네이밍 형식

{type}/{part}/{feature}-{assignee}

1. type: feature, fix, refactor, docs, hotfix

feature/	일반 기능 개발
fix/	일반 버그 수정 (서비스는 돌아가는데 문제 있는 경우)
refactor/	기능 변화 없이 코드 구조 개선
docs/	문서 작업
hotfix/	긴급/치명적 버그 수정 (서비스 장애 수준 → master 기준으로 바로 작업)

2. **part**: front, back, ai
3. **feature**: 기능명 (kebab-case)
4. **assignee**: 개발 담당 구별 시 사용(선택)

브랜치 생성 및 병합 흐름

- 각 develop 브랜치에서 feature 브랜치 생성
- 기능 개발 진행
- develop 브랜치로 PR 및 merge

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

- 최종적으로 master 브랜치로 merge 후 배포

master branch 머지 순서 규칙

front가 의존성이 가장 많으므로, API 기준으로 merge 진행

- back-develop 먼저 merge
- ai-develop merge
- front-develop 마지막 merge

깃허브 커밋 규칙

1. 커밋 유형 지정

Prefix	설명
[INIT]	개발 환경 초기 설정
[STYLE]	스타일 관련 기능(코드 포매팅, 세미콜론 누락, 코드 자체의 변경이 없는 경우)
[DOCS]	문서 수정
[UPLOAD]	이미지 및 파일 단순 업로드
[FEAT]	새로운 기능의 추가
[FIX]	버그 수정
[UPDATE]	기존 기능 보완
[REMOVE]	기능/요소/파일 등 삭제
[RENAME]	파일 또는 폴더 이름 변경
[MOVE]	코드 또는 파일 이동

 국민대학교 소프트웨어학부 캡스톤 디자인 I	결과보고서		
	프로젝트 명	길벗	
	팀 명	리트리버	
	Confidential Restricted	Version 1.0	2026-MAY-14

[COMMENT]	주석 추가 또는 변경
[REFACTOR]	코드 리팩토링
[CHORE]	빌드 업무 수정, 패키지 매니저 수정(ex .gitignore 수정 같은 경우)
[TEST]	테스트 코트, 리팩토링 테스트 코드 추가
[REVERT]	이전 커밋 되돌리기
[BUILD]	빌드 시스템 수정

커밋 메시지 작성: 한국어, 어떤 것에 대한 개발인지 명확하게 알 수 있도록 작성

도입 효과: 14주 동안 충돌 없는 병렬 개발 유지, PR 리뷰 시 변경 범위 명확화, GitHub Actions CI 트 리거 조건 분리